MINISTRY OF EDUCATION AND TRAINING
CAN THO UNIVERSITY
COLLEGE OF INFORMATION AND
COMMUNICATION TECHNOLOGY
DEPARTMENT OF SOFTWARE ENGINEERING

GRADUATION THESIS
BACHELOR OF ENGINEERING IN
INFORMATION TECHNOLOGY

(HIGH-QUALITY PROGRAM)

BUILDING A FASHION E-COMMERCE APPLICATION WITH RECOMMENDATION AND IMAGE-BASED PRODUCT SEARCH

Student: Nguyen Thanh Phat
Student ID: B2005853
Class: DI20V7F1 (K46)
Advisor: Dr. Tran Cong An

Can Tho, December 2025

MINISTRY OF EDUCATION AND TRAINING
CAN THO UNIVERSITY
COLLEGE OF INFORMATION AND COMMUNICATION TECHNOLOGY
DEPARTMENT OF SOFTWARE ENGINEERING

GRADUATION THESIS
BACHELOR OF ENGINEERING IN INFORMATION TECHNOLOGY

(HIGH-QUALITY PROGRAM)

BUILDING A FASHION E-COMMERCE APPLICATION WITH RECOMMENDATION AND IMAGE-BASED PRODUCT SEARCH

Student: Nguyen Thanh Phat
Student ID: B2005853
Class: DI20V7F1 (K46)
Advisor: Dr. Tran Cong An

Can Tho, 01/2026

TRƯỜNG ĐẠI HỌC CẦN THƠ
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN

CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM
Độc lập – Tự do – Hạnh phúc

XÁC NHẬN CHỈNH SỬA LUẬN VĂN THEO YÊU CẦU CỦA HỘI ĐỒNG

Tên luận văn (tiếng Việt và tiếng Anh):

Building a Fashion Ecommerce Application with Recommendation and Image-based Product Search

Phát triển ứng dụng thương mại điện tử bán thời trang tích hợp gợi ý và tìm kiếm sản phẩm bằng hình ảnh

Họ tên sinh viên: Nguyễn Thanh Phát MASV: B2005853

Mã lớp: DI20V7F1

Đã báo cáo tại hội đồng ngành: Công nghệ Thông tin

Ngày báo cáo: 24/12/2025

Hội đồng báo cáo gồm:

1. TS. Phạm Thế Phi
2. TS. Thái Minh Tuấn
3. TS. Trần Công Án

Luận văn đã được chỉnh sửa theo góp ý của Hội đồng.

Cần Thơ, ngày ..... tháng ..... năm 2026

Giáo viên hướng dẫn
(Ký và ghi họ tên)

EVALUATION OF ADVISOR

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Advisor:

Dr. Tran Cong An

ACKNOWLEDGEMENTS

I wish to express my deepest gratitude and sincere thanks to my advisor, Dr. Tran Cong An, who dedicated immense time, patience, and effort to guide me through every challenge of this thesis. His invaluable support was the cornerstone of this work. I would also like to thank the lecturers at Can Tho University for their invaluable knowledge and guidance throughout my studies.

Most importantly, I am profoundly grateful to my family for being my constant companions, offering their unwavering love, care, and support throughout this journey. I am also thankful to my friends for their continuous encouragement and companionship.

Sincerely,

Can Tho, December 2025

Nguyen Thanh Phat

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS

Abbreviation	Description
API	Application Programming Interface
ANN	Approximate Nearest Neighbor
CBIR	Content-Based Image Retrieval
CNN	Convolutional Neural Network
CQRS	Command Query Responsibility Segregation
DDD	Domain-Driven Design
DSR	Design Science Research
EF Core	Entity Framework Core
HNSW	Hierarchical Navigable Small World
JWT	JSON Web Token
mAP	Mean Average Precision
P@K	Precision at K
R@K	Recall at K
REST	Representational State Transfer
SPA	Single Page Application
ViT	Vision Transformer
VSA	Vertical Slice Architecture

ABSTRACT

E-commerce platforms rely primarily on text-based search, yet fashion products are inherently visual. Patterns, silhouettes, and textures resist keyword description. This thesis presents a fashion e-commerce platform with integrated Content-Based Image Retrieval (CBIR) that enables customers to search for products by uploading images rather than typing keywords. The system implements a modular architecture with a .NET backend, Vue.js frontend, and a Python machine learning sidecar for embedding generation.

A systematic benchmark evaluates four pre-trained deep learning models across both retrieval accuracy and operational efficiency on a curated fashion product dataset. Fashion-CLIP, a CLIP variant fine-tuned on over 700,000 fashion images, achieved the highest mean Average Precision at 0.8788 (SD 0.0022), outperforming the generic CLIP (mAP 0.8341, SD 0.0043) by 5.4%, EfficientNet-B0 (mAP 0.8158, SD 0.0007) by 7.7%, and ResNet-50 (mAP 0.8120, SD 0.0052) by 8.2%. At the opposite end of the speed spectrum, EfficientNet-B0 delivered inference at 23.9 ms per image (SD 2.5) while maintaining competitive retrieval quality, representing a 3.8x speed advantage over Fashion-CLIP (92.0 ms, SD 5.8) for a 7.7% accuracy trade-off.

Results demonstrate that domain-specific pre-training provides measurable advantages for visual fashion retrieval. The pluggable model architecture, switchable via a single environment variable, allows production deployments to select the optimal accuracy-speed profile for their infrastructure. The thesis provides a reference implementation for systematic comparison of embedding models in the fashion domain.

Keywords: e-commerce, visual search, deep learning, modular architecture, computer vision, benchmarking

TÓM TẮT

Các sàn thương mại điện tử phụ thuộc nhiều vào tìm kiếm văn bản, một cơ chế kém hiệu quả trong lĩnh vực thời trang, nơi hoa văn, chất liệu và kiểu dáng khó diễn đạt chính xác qua từ khóa. Hệ thống được xây dựng theo hướng module, bao gồm ba thành phần chính: backend .NET xử lý logic nghiệp vụ, frontend Vue.js phục vụ giao diện người dùng, và dịch vụ Python chuyên biệt cho tác vụ trích xuất đặc trưng hình ảnh từ các mô hình học sâu tiền huấn luyện.

Thực nghiệm được thiết lập trên bộ dữ liệu ảnh sản phẩm thời trang với quy trình đánh giá chéo ba lần (3-fold cross-validation), so sánh bốn mô hình embedding trên hai chiều: chất lượng truy xuất (đo bằng mAP, Precision at K, Recall at K) và hiệu năng vận hành (đo bằng độ trễ suy luận, thông lượng, và dung lượng lưu trữ). Mô hình Fashion-CLIP, được tinh chỉnh trên tập dữ liệu hơn 700.000 ảnh thời trang, đạt mAP cao nhất 0,8788 với độ lệch chuẩn 0,0022. CLIP gốc đạt mAP 0,8341 (SD 0,0043) và EfficientNet-B0 đạt 0,8158 (SD 0,0007). Ở chiều ngược lại, mô hình EfficientNet-B0 cho tốc độ suy luận nhanh nhất, 23,9 ms mỗi ảnh (SD 2,5), nhanh gấp 3,8 lần Fashion-CLIP (92,0 ms, SD 5,8) trong khi chỉ đánh đổi 7,7% về chất lượng truy xuất.

Kết quả thực nghiệm khẳng định giá trị của huấn luyện chuyên biệt theo lĩnh vực trong bài toán truy xuất hình ảnh thời trang, đồng thời cung cấp dữ liệu định lượng cho việc lựa chọn mô hình dựa trên ràng buộc hạ tầng thực tế. Kiến trúc mô hình hoán đổi linh hoạt, được điều khiển qua biến môi trường, cho phép hệ thống thích ứng với nhiều cấu hình triển khai khác nhau mà không cần thay đổi mã nguồn.

Từ khóa: thương mại điện tử, tìm kiếm hình ảnh, học sâu, kiến trúc module, thị giác máy tính, đánh giá hiệu năng

PART 1: INTRODUCTION

I. CONTEXT AND MOTIVATION

Global fashion e-commerce revenue exceeded **770 billion USD** in 2024, with projections surpassing **one trillion by 2030** [1]. Yet keyword search fails where the domain succeeds: fashion products are defined by silhouette, drape, print density, and colour – attributes that resist textual description. Industry estimates place session abandonment after unsuccessful search at approximately **30 percent** [2].

Content-Based Image Retrieval (CBIR) addresses this gap by replacing textual intermediaries with direct visual comparison. Products are indexed not by human-authored labels but by **dense vector embeddings** computed from images, with similarity measured through mathematical distance functions. A query image of a dress with a particular neckline and print pattern retrieves visually similar products without any keyword translation step. Pre-trained convolutional neural networks [3], [4], vision transformers [5], and fashion-specific models [6] have substantially advanced this capability.

The contribution of this work is **architectural**, not algorithmic. It investigates how to embed existing CBIR capabilities into a practical e-commerce system built with conventional web technologies, and provides empirical data on which embedding models deliver the optimal balance of accuracy, latency, and resource efficiency. The work bridges two distinct software ecosystems, the **Python machine learning stack** and the **.NET enterprise web stack**, under real-time latency constraints appropriate for interactive search.

II. PROBLEM STATEMENT

Keyword-reliant fashion search suffers from four compounding inefficiencies.

Catalogue vocabulary mismatch. Varying vendor descriptors fragment result sets, silently excluding relevant products.

Visual inexpressibility. Attributes such as fabric drape, texture, silhouette proportion, and pattern rhythm elude text queries.

Cold-start invisibility. New products lack interaction history. Visual feature extraction enables discovery immediately from catalogue ingestion.

Polyglot integration cost. The Python deep learning ecosystem does not natively interoperate with .NET. Sub-second latency requires architectural isolation of the ML workload.

III. OBJECTIVES

This project builds a functional fashion e-commerce platform with integrated image-based search and evaluates pre-trained deep learning models within that system. The contribution is the **engineering demonstration** of embedding existing models into a conventional web application stack.

Technical Objectives

- **Model integration.** Integrate pre-trained vision models into a PostgreSQL and .NET e-commerce stack, establishing a reference pattern for teams with existing web infrastructure.
- **Polyglot architecture.** Architect a polyglot system in which a dedicated Python sidecar handles AI inference while the .NET backend manages transactional logic, business rules, and API routing.
- **Vector storage validation.** Validate **pgvector** (an open-source PostgreSQL extension) as the sole vector storage and retrieval layer, evaluating whether it meets real-time search latency requirements at catalogue scales representative of small-to-medium fashion retailers.
- **Empirical benchmarking.** Benchmark multiple embedding models spanning convolutional and transformer architectures on shared hardware, producing empirical guidance for model selection in resource-constrained deployments.

Research Questions

Three questions guide the investigation and are answered empirically in Chapter 3.

RQ1: Model comparison. How do fashion-specific embedding models compare with general-purpose CNN and ViT architectures on fashion product retrieval?

RQ2: Accuracy-speed trade-off. What trade-offs exist between retrieval accuracy and inference latency, and which model offers the best balance for real-time search?

RQ3: Architecture viability. Can a service-oriented architecture with a dedicated AI sidecar separate image inference from the main application while maintaining interactive response times?

Tasks Completed

- **Build AI service.** Python FastAPI service loading pre-trained embedding models for vector generation within interactive latency bounds.
- **Set up vector search.** PostgreSQL with pgvector for high-dimensional embedding storage and similarity queries.
- **Connect services.** .NET backend orchestrating image upload, embedding generation, vector database query, and result assembly.
- **Create user interface.** Vue.js storefront with drag-and-drop image upload and similarity-scored results grid.
- **Evaluate results.** Systematic benchmark measuring retrieval accuracy, inference speed, and operational trade-offs across models.

IV. SCOPE AND LIMITATIONS

In scope: visual search via image upload, embedding-based recommendations, core e-commerce (catalogue, cart, checkout), and multi-model comparison across CNN and transformer architectures. Out of scope: real payment processing, shipping and logistics, social login, mobile applications, and custom model training.

Known Limitations

Four limitations define the boundaries of this work.

- **Dataset.** 5,000 fashion product images [7]. Controlled benchmarking is feasible at this scale but results may not extrapolate to production catalogues containing millions of items.
- **Hardware.** Consumer-grade (Intel i7-1165G7, 16 GB RAM), all inference on CPU. Latency and throughput figures are relative to this profile; GPU acceleration would improve both metrics.
- **Evaluation.** Exclusively quantitative: accuracy, latency, throughput. No formal user study; relationship between measured metrics and user satisfaction remains open.
- **Model training.** All models used as published. Domain-specific fine-tuning, particularly for models pre-trained on generic corpora, might improve quality but was beyond scope.

V. RESEARCH METHODOLOGY

This section describes the methodology and tools used to implement and evaluate the system.

Development Methodology

The project follows **Design Science Research** (DSR) [8], [9] across four phases: Research and Planning (literature review, model and tool selection), Design

(technology stack, system architecture, schema design), Implementation (.NET backend with VSA, Python FastAPI sidecar, Vue 3 storefront), and Testing and Evaluation (mAP accuracy with cross-validation, inference latency, throughput across 11 models).

Technologies Used

The system is built using a modular stack designed for performance and scalability:

- **Backend:** .NET 10 with Carter, MediatR, FluentValidation.
- **AI Service:** Python 3.12 with FastAPI, PyTorch, Hugging Face Transformers.
- **Frontend:** Vue 3 with TypeScript, Vite, Pinia.
- **Database:** PostgreSQL with pgvector for relational and vector data in a single ACID database.

The system is evaluated using quantitative metrics: Mean Average Precision (mAP) with 3-fold cross-validation for retrieval accuracy, per-image inference latency and throughput (images/second) for efficiency, across four representative models and the Fashion Product Images Dataset [7] (5,000 images). Detailed results appear in Chapter 3.

VI. THESIS OUTLINE

The thesis is organized into five chapters across three parts.

Part I: Introduction. Chapter 1 establishes research context, the problem statement, objectives, research questions, scope, methodology, and outline.

Part II: Thesis Content contains three chapters:

- **Chapter 1: Background.** Surveys vector embeddings, neural architectures, vector databases, prior work in fashion image retrieval, and the technology stack.
- **Chapter 2: Design and Implementation.** Functional and non-functional requirements, system architecture (DDD, C4, database, API, security), and concrete implementation (.NET backend, Python ML sidecar, Vue storefront).
- **Chapter 3: Evaluation.** Systematic benchmark comparing retrieval accuracy and inference efficiency across embedding models using cross-validation on 5,000 fashion images.

Part III: Conclusion. Chapter 4 synthesizes findings, evaluates contributions and limitations, and proposes future work.

PART 2: THESIS CONTENT

1 BACKGROUND AND RELATED WORK

This chapter surveys the theoretical foundations and technical prerequisites for the project.

- **Fashion E-commerce.** Market context, semantic gap, business case.
- **Content-Based Image Retrieval.** Embeddings, cosine similarity, CBIR pipeline.
- **Machine Learning Models.** CNN, ViT, CLIP, Fashion-CLIP architectures.
- **Vector Databases.** ANN search, HNSW, IVFFlat, pgvector.
- **Platform Architecture and Technology Stack.** Modular monolith, vertical slice, .NET, Vue, PostgreSQL, Redis, Python sidecar, orchestration, auth, benchmarks.
- **Related Work and Research Gap.** Academic research, commercial systems, contribution differentiators.

1.1 FASHION E-COMMERCE

Global fashion e-commerce reached **770 billion USD** in 2024 and is projected to surpass **one trillion USD by 2030** [1]. However, traditional text search bars cause a **30 percent search abandonment rate** because keyword queries struggle to match visual intent [2].

Fashion is fundamentally visual. Attributes like silhouette, drape, color harmony, and pattern density do not translate easily into search terms. A customer can easily recognize a garment in an image but often fails to find it using keywords.

Major platforms like ASOS, Zalando, and Shopee invest in visual search because millions of catalog items rely on inconsistent vendor metadata. Identical patterns often carry different labels across suppliers, hiding relevant products from keyword search. Visual search solves this metadata problem, directly preventing lost revenue from search abandonment.

1.2 CONTENT-BASED IMAGE RETRIEVAL

Content-Based Image Retrieval replaces text queries with image queries. Rather than indexing products by human-authored labels, CBIR systems encode images into dense vector representations and measure similarity through mathematical

distance functions. This section introduces the core concepts and mathematical foundations of CBIR as applied to fashion product search.

1.2.1 Visual Search Concepts

Content-Based Image Retrieval (CBIR) replaces text queries with image queries. Rather than requiring users to describe what they want in words, CBIR lets them search by example.

The system encodes a query image into a **dense vector embedding**: a fixed-length sequence of numbers that captures shape, texture, colour, and pattern. It then retrieves catalog items whose embeddings are nearest in vector space. Visually similar products produce similar vectors; dissimilar products produce distant ones.

This approach bypasses the need for consistent textual labels. A photograph of a dress with a distinctive neckline retrieves visually similar products regardless of how the catalog describes them.

1.2.2 The Semantic Gap

The semantic gap, introduced in Section 1.1, is the discrepancy between the visual richness of a garment and a user's ability to express that richness in keywords. CBIR bridges this gap by operating directly on visual content rather than on human-authored metadata. The embedding serves as a universal descriptor that captures attributes such as fabric texture, silhouette proportion, and colour gradients automatically, without any keyword translation step.

1.2.3 Mathematical Foundations of Embeddings

An embedding model processes an input image and transforms its visual content into a fixed-length numerical array:

$$\boldsymbol{e} = [e_1, e_2, e_3, ..., e_d]^T \in \mathbb{R}^d$$

For a model with a 512-dimensional output, this vector $\boldsymbol{e}$ contains 512 continuous numbers. These arrays are called **vector embeddings**. Visually similar items yield nearby numerical values across these dimensions, converting visual comparison into a standard mathematical distance calculation.

1.2.3.1 The Latent Space

Embeddings act as coordinates within a high-dimensional continuous space known as the **latent space**. A 512-dimensional vector occupies a coordinate system with 512 orthogonal axes.

Within this space:

- Visually and semantically similar items sit close to one another.

- Dissimilar items lie further apart.
- The term "latent" indicates that individual axes rarely map directly to single human visual labels like "stripes" or "red." Instead, each dimension captures abstract, non-linear feature combinations learned by the model during training.

1.2.3.2 Measuring Similarity: Cosine Similarity

To evaluate how closely two images match, the system measures the directional angle between their corresponding embedding vectors $\boldsymbol{A}$ and $\boldsymbol{B}$ using **cosine similarity**:

$$\text{Cosine Similarity}(\boldsymbol{A}, \boldsymbol{B}) = \frac{\boldsymbol{A} \cdot \boldsymbol{B}}{\|\boldsymbol{A}\|_2 \times \|\boldsymbol{B}\|_2}$$

Where $\boldsymbol{A} \cdot \boldsymbol{B}$ represents the vector dot product, and $\|\boldsymbol{A}\|_2$ and $\|\boldsymbol{B}\|_2$ denote the Euclidean norms (L_2 norms) of each vector.

Cosine similarity produces values ranging from $+1.0$ (identical vector orientation) to 0.0 (orthogonal vectors) down to -1.0 (opposite orientations). For normalized fashion embeddings, scores above 0.70 generally correspond to strong visual similarity perceptible to human shoppers.

1.2.3.3 The CBIR Pipeline

A standard Content-Based Image Retrieval system operates through four core sequential stages:

1. **Image Input:** The user uploads or selects a target query photograph.
2. **Embedding Generation:** A deep learning model processes the image to output its feature vector.
3. **Vector Comparison:** The database compares the query vector against pre-indexed catalog vectors using cosine distance or cosine similarity.
4. **Ranking & Retrieval:** The system orders products by their similarity scores and renders the top nearest neighbors to the user.

1.3 MACHINE LEARNING MODELS

This section surveys the three families of architectures used for visual feature extraction: convolutional neural networks, vision transformers, and multimodal CLIP-based models. Each subsection explains how the architecture works, introduces the specific variants evaluated, and identifies their trade-offs for fashion retrieval. The section concludes with the model selection decision.

1.3.1 Convolutional Neural Networks

CNNs have dominated computer vision since AlexNet (2012). Variants evaluated: ResNet-50, ResNet-101, EfficientNet-B0, and EfficientNet-B4.

1.3.1.1 Hierarchical Feature Extraction

A CNN processes an image through a stack of learned filters. Each filter is a small window (typically 3 by 3 pixels) sliding across the image detecting local patterns [3], building increasingly complex representations:

Table 1: What different CNN layers learn to detect in fashion images

Layer	What It Detects
Early layers	Simple patterns: edges, colours, corners
Middle layers	Combinations: textures, shapes, parts (lapels, sleeves)
Late layers	High-level features: garment categories, styles

Local patterns cascade into global understanding: edges become textures in middle layers, garments in late layers. CNNs have a strong **inductive bias** toward local patterns: they excel at texture and colour but may miss relationships between distant image regions.

1.3.1.2 ResNet and Skip Connections

Deeper networks capture richer features but suffer from **vanishing gradients**: training signals decay as they propagate backward through many layers. ResNet solves this with **skip connections**: identity paths that bypass convolutional blocks and add the block input directly to its output [3]. This lets gradients flow unimpeded, enabling 50-, 101-, or 152-layer networks to train effectively.

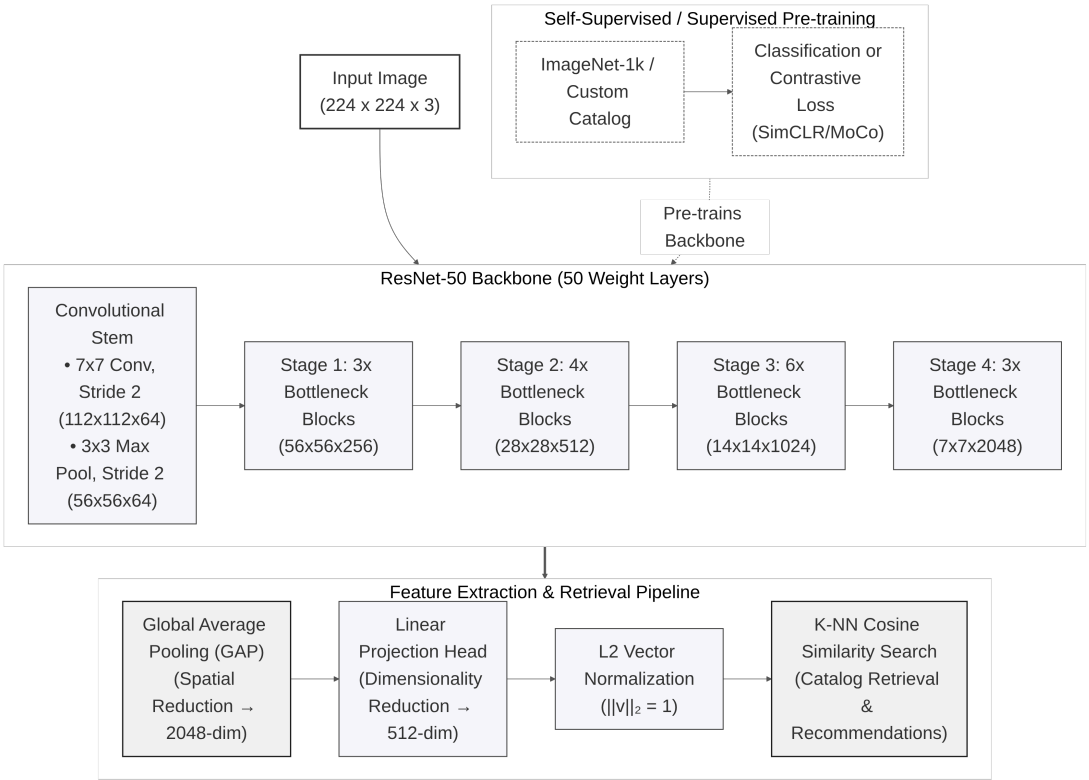

Figure 1: ResNet architecture with residual skip connections enabling gradient flow across very deep networks

ResNet-50 (25.6M parameters, 2,048-dim embeddings) remains a strong baseline for image retrieval. ResNet-101 (44.5M parameters) provides additional depth for comparison.

1.3.1.3 EfficientNet and Compound Scaling

Traditional scaling enlarges a network along one dimension: depth, width, or resolution. EfficientNet introduces **compound scaling**, balancing all three simultaneously using a learned coefficient [4]. This produces models (B0 through B7) with competitive accuracy and far fewer parameters.

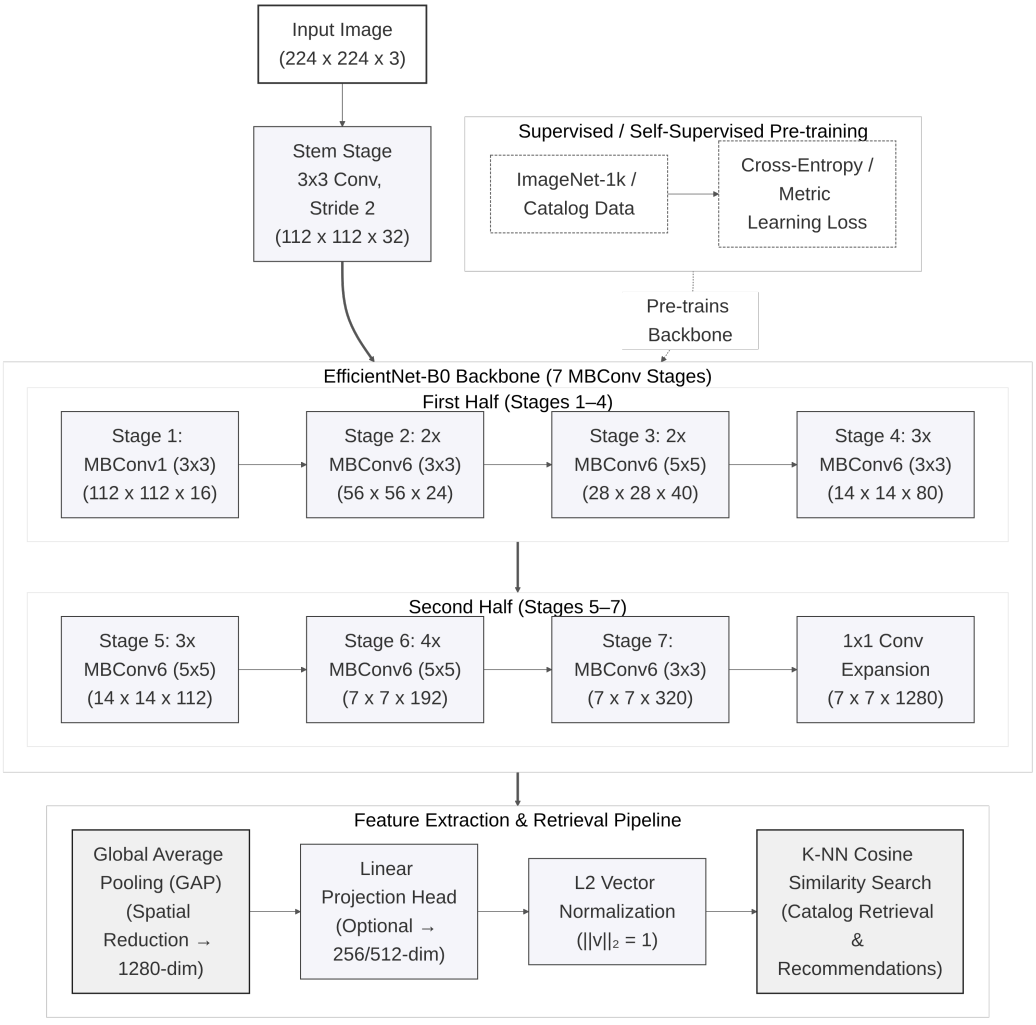

Figure 2: EfficientNet-B0 architecture showing the flow from input image to feature vector

EfficientNet-B0 uses 5.3M parameters and 1,280-dim embeddings, well suited to CPU-only deployments. EfficientNet-B4 (19.3M parameters, 1,792-dim embeddings) provides higher capacity.

1.3.1.4 Evaluated CNN Variants

Table 2: CNN-based models evaluated

Family	Model	Parameters	Embedding Dim	Training Data
CNN	ResNet-50	25.6M	2048	ImageNet (1.2M images)
CNN	ResNet-101	44.5M	2048	ImageNet (1.2M images)
CNN	EfficientNet-B0	5.3M	1280	ImageNet (1.2M images)
CNN	EfficientNet-B4	19.3M	1792	ImageNet (1.2M images)

1.3.2 Vision Transformers

Vision Transformers (ViTs) apply the transformer architecture from NLP to images, replacing convolutional filters with self-attention over image patches.

1.3.2.1 Patch Embedding and Tokenization

Transformers were originally developed for NLP tasks such as translation and text generation. Their key innovation is **self-attention**, allowing the model to consider relationships between all parts of the input simultaneously. In 2020, researchers showed this approach also works for images [10]:

- Split the image into a grid of fixed-size patches (e.g., 14 by 14 or 16 by 16 pixels).
- Flatten each patch into a vector and treat it as a token, analogous to a word in a sentence.
- Add position encodings to preserve spatial information.
- Pass the sequence through transformer layers with multi-head self-attention.

1.3.2.2 Global Context via Self-Attention

Unlike CNNs, which focus on local patterns, self-attention captures relationships across the entire image from the first layer. A ViT can directly compare any two patches, even if far apart. For fashion, this means understanding that collar and cuffs match across opposite sides of an image: valuable for garment retrieval where silhouette and drape matter as much as local texture.

1.3.2.3 DINOv2 and Self-Supervised Pre-Training

Where supervised learning requires expensive human labels, **self-supervised learning** learns from the images themselves. DINOv2 uses a student-teacher self-distillation framework [11]:

- Take an image and create two different views (different crops, slightly different colours).
- Pass them through student and teacher networks with identical architecture.
- Train the student to match the teacher's output.
- Update the teacher as an exponential moving average of the student weights.

- Repeat across 142 million uncurated images.

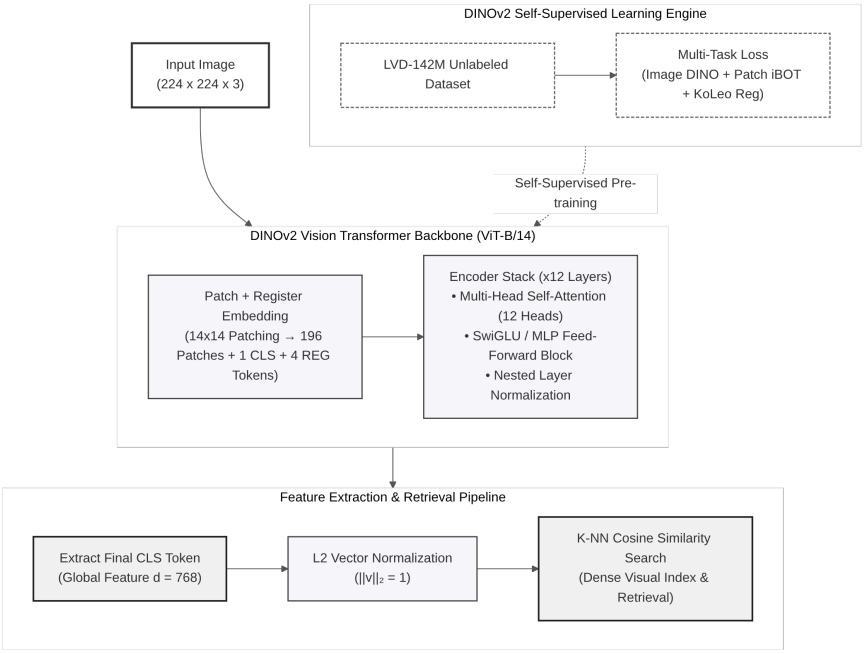

Figure 3: DINOv2 architecture: image patches pass through transformer layers with self-attention to produce a feature vector

DINOv2 produces features with strong object-level structure: silhouettes, part geometry, and garment boundaries without category labels, making it adaptable to fashion domains where curated labels are scarce.

1.3.2.4 Structural Fidelity for Fashion Retrieval

DINOv2 excels at **structural fidelity**: shapes, silhouettes, and proportions. It matches garments by cut (A-line, fitted, oversized), by proportion (cropped vs. full-length), and separates shape from colour; useful for finding a dress in a different colour with the same shape.

1.3.2.5 DINOv2 Model Specifications

Table 3: DINOv2 model specifications evaluated in this project

Property	**DINOv2 ViT-S/14**	**DINOv2 ViT-B/14**
Architecture	Vision Transformer (Small)	Vision Transformer (Base)
Parameters	21M	86M
Patch size	14 by 14 pixels	14 by 14 pixels
Embedding dimension	384	768
Training data	142M images	142M images
Training method	Self-supervised	Self-supervised

1.3.2.6 Trade-offs and Limitations

Vision Transformers trade off differently than CNNs:

Advantages:

- Better at understanding global structure and long-range relationships.
- Learned without human labels, so potentially more general.
- Strong structural fidelity: captures shapes and silhouettes.

Disadvantages:

- Slower than CNNs (more computation required).
- Require larger input images for best results.
- May be overkill for simple colour/pattern matching.

1.3.3 CLIP and Fashion-CLIP

CLIP (Contrastive Language-Image Pre-training) bridges vision and language, enabling search using both visual and textual queries. Fashion-CLIP, its domain-specialized variant, is the primary model for visual search.

1.3.3.1 Contrastive Language-Image Pre-Training

Traditional image models classify images into fixed categories ("cat," "dog," "dress"). CLIP learns to match images with text descriptions instead [5].

During training, CLIP processed 400 million image-text pairs from the public web (e.g., a floral dress with "colourful floral summer dress," sneakers with "white running shoes on grass"). From these pairs, the model learned to:

- Convert images into vectors (image encoder).
- Convert text into vectors (text encoder).
- Make matching image-text pairs produce similar vectors.

1.3.3.2 Dual-Tower Architecture

CLIP has two separate towers:

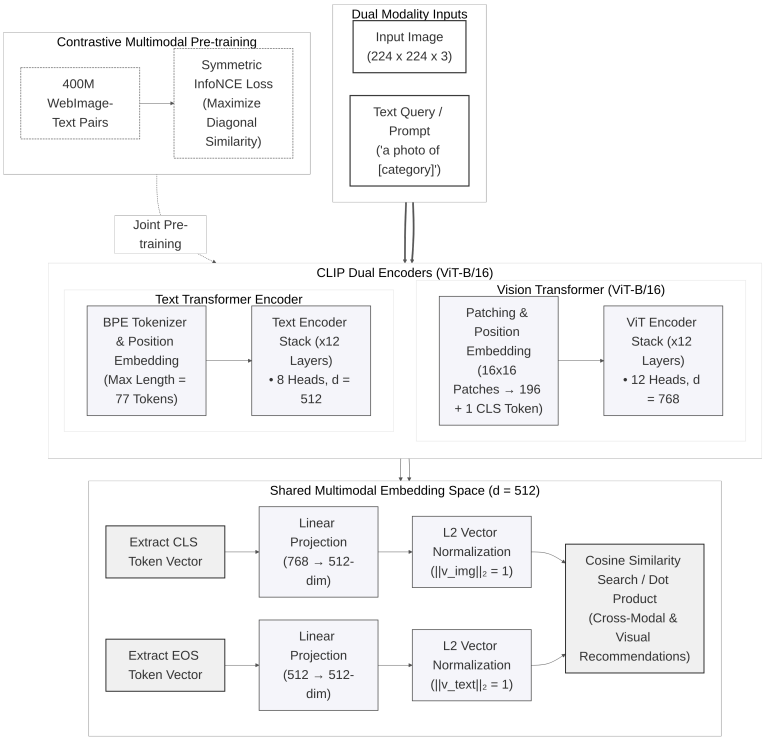

Figure 4: CLIP ViT-B/16 dual-tower architecture: images and text are converted to vectors in the same embedding space, allowing direct comparison

- **Image Tower.** Processes the image using a Vision Transformer (ViT-B/16, ViT-B/32, or ViT-L/14).
- **Text Tower.** Processes text using a transformer.

Both towers output vectors of the same size (512 dimensions for ViT-B variants, 768 for ViT-L), so images and text can be directly compared using cosine similarity.

1.3.3.3 Multimodal Embedding Space

CLIP's natural language understanding enables fashion concepts like "bohemian style" or "minimalist design," matching images to abstract descriptions ("something for a casual Friday"), and capturing semantic meaning beyond visual appearance. However, general CLIP was trained on diverse internet images, not specifically fashion; it may not distinguish "A-line dress" from "sheath dress" or "Bohemian" from "vintage."

1.3.3.4 Multimodal Query Capabilities

The dual-tower design enables query modalities unavailable in vision-only models such as DINOv2 or EfficientNet:

- **Text-to-image search.** A user types "red floral summer dress"; the text encoder maps the description into the same embedding space as catalog images.

- **Hybrid queries.** An uploaded photo combined with textual refinement ("like this, but in blue") by encoding both modalities and merging the results.

This flexibility makes CLIP-based models the primary choice for the visual search feature.

1.3.3.5 Fashion-CLIP and Domain-Specific Fine-Tuning

Fashion-CLIP further trains CLIP on over 700,000 fashion product images paired with detailed descriptions covering garment categories, fabric textures, style descriptors, and occasion labels [6]. This specialization helps Fashion-CLIP understand:

- Fashion-specific vocabulary ("A-line," "empire waist," "distressed denim").
- Style categories ("streetwear," "preppy," "athleisure").
- Occasion suitability ("office wear," "cocktail party," "beach vacation").

Fashion-CLIP inherits the ViT-B/16 architecture, producing 512-dimensional embeddings. The original paper reports a 15-to-20% improvement on fashion retrieval over general CLIP, confirmed in the benchmark evaluation presented in Chapter 3.

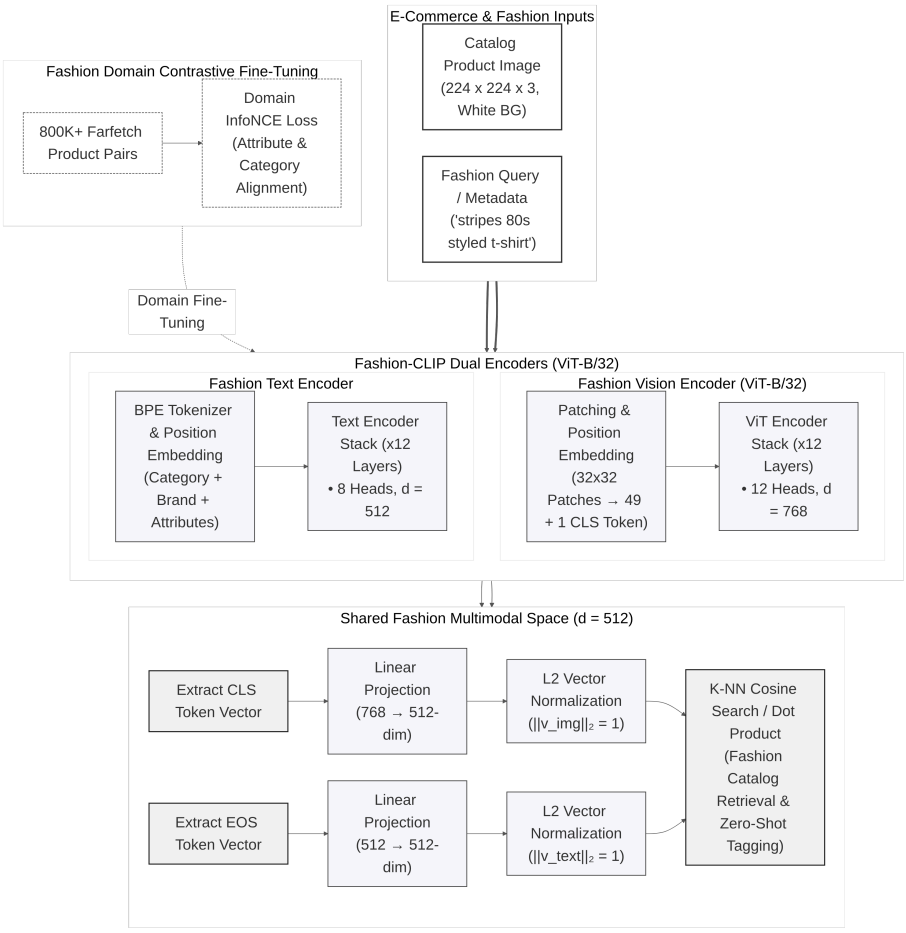

Figure 5: Fashion-CLIP dual-tower architecture: image and text towers independently encode their inputs into 512-dimensional embeddings converging in a shared latent space

1.3.3.6 Evaluated CLIP Variants

Table 4: CLIP variants evaluated

Model	Architecture	Training	Domain
CLIP ViT-B/32	Dual-tower (ViT + text transformer)	Contrastive (400M pairs)	General
CLIP ViT-B/16	Dual-tower (ViT + text transformer)	Contrastive (400M pairs)	General
CLIP ViT-L/14	Dual-tower (ViT + text transformer)	Contrastive (400M pairs)	General
Fashion-CLIP	Dual-tower (ViT + text transformer)	Contrastive, fine-tuned on 700K fashion images	Fashion-specific

1.3.4 Model Selection and Justification

The rationale for selecting Fashion-CLIP as the primary model for visual search is presented below.

1.3.4.1 Candidate Models

Eleven pre-trained models spanning three architectural families were evaluated:

Table 5: Candidate embedding models evaluated for fashion product retrieval. All models are pre-trained and publicly available.

Model	Architecture	Dim	Parameters	Training Method
ResNet-50	CNN	2048	25.6M	Supervised (ImageNet)
ResNet-101	CNN	2048	44.5M	Supervised (ImageNet)
EfficientNet-B0	CNN	1280	5.3M	Supervised (ImageNet)
EfficientNet-B4	CNN	1792	19.3M	Supervised (ImageNet)
DINOv2 ViT-S/14	ViT	384	21M	Self-supervised (142M images)
DINOv2 ViT-B/14	ViT	768	86M	Self-supervised (142M images)
CLIP ViT-B/32	ViT + Text	512	151M	Contrastive (400M pairs)
CLIP ViT-B/16	ViT + Text	512	150M	Contrastive (400M pairs)
CLIP ViT-L/14	ViT + Text	768	428M	Contrastive (400M pairs)
Fashion-CLIP	ViT + Text	512	150M	Fine-tuned (700K fashion)

1.3.4.2 Evaluation Methodology

All models were evaluated under identical conditions: 5,000 fashion product images from the Fashion Product Images dataset, split into training and query sets. Hardware was consumer-grade: Intel i7-1165G7 CPU with 16 GB RAM, all inference on CPU.

Metrics included Mean Average Precision (mAP@10) as the primary measure of overall retrieval quality, Precision at K (P@K) for top-ranked accuracy, Recall at K (R@K) for coverage of relevant items, and inference latency in milliseconds. A retrieved product was considered relevant if it belonged to the same category as the query image. The full evaluation protocol, benchmark results, and cross-validation methodology are presented in Chapter 3.

1.3.4.3 Weighted Selection Criteria

Model selection was based on four criteria: retrieval quality (mAP@10 and P@K scores), inference latency (sub-300 ms total response time), multimodal capability (search by image and text), and hardware compatibility within memory and compute constraints of commodity hardware.

1.3.4.4 Selection Decision

Fashion-CLIP was selected as the primary embedding model for the visual search feature. Three factors drove this decision.

First, retrieval quality: Fashion-CLIP achieved the highest mAP among the evaluated models, with a 15 to 20 percent improvement over general CLIP on fashion-specific queries, confirmed through the systematic benchmark in Chapter 3 [6].

Second, multimodal capability: Fashion-CLIP's dual-tower architecture enables search by image, by text description, and by hybrid image-plus-text queries, unavailable in vision-only models such as DINOv2 and EfficientNet.

Third, domain specialization: fine-tuning on 700,000 fashion images gives Fashion-CLIP an understanding of fashion-specific vocabulary, styles, and garment attributes that general-purpose models lack.

Fashion-CLIP provides the best overall balance of retrieval quality, search flexibility, and inference performance for the target deployment scenario, though EfficientNet-B0 offers faster CPU inference and DINOv2 excels at structural fidelity for silhouette-based matching.

1.3.4.5 Alternative Deployment Scenarios

For different deployment contexts, alternative models may be preferred. EfficientNet-B0 provides the fastest inference at 5.3 million parameters, trading off 3.4 percent lower mAP@10 with no text-to-image capability. DINOv2 excels at shape and silhouette matching but lacks multimodal capability. General CLIP

variants suit multi-category marketplaces with lower fashion accuracy. CLIP ViT-L/14 offers the largest capacity at 428 million parameters but requires substantial GPU VRAM.

The complete numerical comparison and error analysis across all models are presented in Chapter 3.

1.4 VECTOR DATABASES

Once images are converted to embeddings, those vectors must be stored and searched efficiently. This section explains the challenge of vector similarity search at scale, introduces two indexing algorithms, and describes pgvector, the PostgreSQL extension used in this project.

1.4.1 The Search Challenge

When a user uploads a query image, the system must compare its embedding vector against every product vector in the catalog. This is **nearest neighbour search**.

A naive brute-force approach scales linearly with catalog size:

- 10,000 products = 10,000 comparisons per search.
- 100,000 products = 100,000 comparisons.
- 1,000,000 products = 1,000,000 comparisons.

For real-time search (under one second), this is too slow.

1.4.2 Approximate Nearest Neighbour Search

The solution is **Approximate Nearest Neighbour** (ANN) search [12]. Instead of checking every vector, ANN algorithms build index structures (graphs or clusters) that let a query navigate directly to the neighbourhood of likely matches, skipping irrelevant vectors.

The key trade-off: we trade perfect accuracy for speed. If the true top match has similarity 0.95 and the returned result has 0.93, that is acceptable for product search. ANN algorithms typically achieve 97 to 99% recall of the true top matches while reducing search time by orders of magnitude [12]. Two algorithms are used in this project: **HNSW** for production search and **IVFFlat** for rapid evaluation [13].

1.4.3 HNSW: Hierarchical Navigable Small World

HNSW is the preferred index for production-scale vector search [12]. It builds a multi-layered graph where each vector is a node connected to its nearest neighbours.

- **Structure.** Multiple layers form a hierarchy. Top layers are sparse and connect distant regions for long-range jumps. Bottom layers are dense and refine the search locally.
- **Search.** Start at a top-layer node, greedily traverse toward the nearest neighbour, descend one layer, and repeat. Cost scales logarithmically with catalog size.
- **Recall.** Reported at over 95% with query latencies under 10 ms, sustained across catalog scales of up to 10 million vectors [12].
- **Build cost.** Graph construction is computationally expensive. At tens of thousands of vectors, build time is measured in minutes.

Configuration parameters:

- **M.** Connections per node. Default 16. Higher improves recall at cost of memory and build time.
- **ef_construction.** Candidates evaluated during index build. Higher produces better index at cost of build time.

HNSW's logarithmic query cost makes it suitable for interactive fashion retrieval at millions of catalog items [12], where sub-100 ms latency is required.

1.4.4 IVFFlat: Inverted File with Flat Compression

IVFFlat is a simpler ANN algorithm used during model evaluation. It partitions the embedding space into clusters via **k-means** and stores each vector in its nearest cluster [13].

- **Search.** Compute distance to all cluster centroids, select the nearest few clusters, and search exhaustively within only those clusters.
- **Recall.** Moderate: 65 to 72% at sub-10 ms for catalog-scale data, per pgvector documentation [13]. Recall degrades as the catalog grows because each cluster contains more candidates.
- **Build cost.** Low. Clustering completes in under one second for 5,000 vectors with minimal memory overhead.

Configuration parameters:

- **lists.** Number of clusters. More lists means smaller clusters and faster search, but risks missing relevant vectors.
- **probes.** Number of clusters searched per query. More probes improve recall at linear cost to query latency.

IVFFlat is used for the model comparison benchmarks in Chapter 3, where the objective is ranking embedding models rather than optimising index performance. Its fast build time and simple configuration suit rapid evaluation. For production, HNSW is the designated long-term index.

1.4.5 pgvector: Vector Search in PostgreSQL

pgvector is an open-source PostgreSQL extension adding vector operations to standard SQL [13], storing vectors alongside regular product data.

Key features:

- **Vector column.** `VECTOR(512)` stores embeddings, supporting varying dimensions.
- **Similarity operators.** `<=>` for cosine distance, `<->` for Euclidean.
- **Indexing.** HNSW and IVFFlat for fast approximate search.

The critical advantage is transactional consistency: vectors and product metadata share the same ACID boundary, eliminating dual-database drift. Combined queries can search for visually similar products filtered by category and price range in one query plan.

```
CREATE INDEX ON products USING hnsw (image_embedding vector_cosine_ops);
SELECT id, name, 1 - (image_embedding <=> query_vec) AS similarity
FROM products ORDER BY image_embedding <=> query_vec LIMIT 10;
```

1.4.6 Architectural Decision and Trade-offs

Specialised vector databases (Pinecone, Milvus, Weaviate) exist for large-scale search. pgvector was selected for simplicity and transactional integration.

Table 6: pgvector vs specialised vector databases

Feature	Specialised Vector DB	pgvector
Setup	Moderate to high	Low (extension only)
Consistency	Separate database	Same transaction as product data
Query language	Custom API	Standard SQL
Cost	Often paid service	Free, open source
Scale limit	Billions of vectors	Millions of vectors

For thousands to tens of thousands of products, pgvector's simplicity outweighs scaling advantages. Limitations: not designed for billion-vector deployments, fewer features than dedicated vector databases, no native multi-server distribution. For this project's 5,000-product scope, these are acceptable.

1.5 PLATFORM ARCHITECTURE AND TECHNOLOGY STACK

Building a production-capable e-commerce platform with integrated machine learning requires deliberate architectural and technology choices. This section surveys architectural patterns, describes each technology in the ReSys.Shop stack, and provides the rationale for each selection.

1.5.1 Architectural Patterns

An application's architecture determines code partitioning and component communication [14]. Table 7 compares four patterns governing system-level structure.

Table 7: Comparison of architectural patterns

Pattern	**Structure**	**Strengths**	**Limitations**	**Suited For**
Monolith [15]	Single deployable unit; one codebase, build, and deployment	No network overhead; simple development and debugging	Components entangle over time; full restart on any deployment	Small applications; one team; few business domains
Microservices [16]	Independent services each owning one business capability; network communication	Autonomous teams with different stacks; independent scaling and deployment	Network latency; service discovery; partial failure handling; distributed transaction coordination	Large organisations; independently evolving teams and domains
Modular Monolith [15]	Independent modules in a single process; in-process message bus; compile-time cross-module isolation	Module independence without networking cost; one deployment and one database; simple migration path to microservices	Single database may limit extreme-scale growth; all modules restart on any deployment	Single-team projects requiring logical separation without distributed infrastructure
VSA [17]	Self-contained feature folders with handler, validation, data access, and endpoint definition	All code for one feature co-located; features added, modified, or removed in isolation	Cross-cutting concerns (authentication, logging) require explicit extraction across slices	Numerous independent features evolving at different cadences

1.5.1.1 Architectural Decision

ReSys.Shop combines three patterns:

- **Modular monolith.** Nine business modules in a single .NET process communicate through MediatR CQRS [18]. Compile-time rules prevent direct cross-module references.
- **Vertical slice architecture** within each module [17]. Each feature is a self-contained folder with handler, endpoint, and validator.
- **Python sidecar.** Embedding generation runs in a dedicated FastAPI service over HTTP, isolated from the .NET runtime.

This combines monolith deployment simplicity, microservice-level code isolation, and ML capability without distributed infrastructure overhead [15].

1.5.2 .NET Backend

The backend is built on .NET 10, a high-performance runtime with ahead-of-time compilation and native asynchronous I/O [19]. Its architecture is organised around five core libraries:

- **Carter** extends ASP.NET minimal APIs with module-based endpoint registration. Each business module declares its own routes independently, keeping endpoint definitions co-located with their handlers rather than centralised in a startup file.
- **MediatR** implements CQRS, routing commands (writes) and queries (reads) to handlers through an in-process message bus [18]. Handlers are discovered at startup and dispatched by request type, with no direct coupling between modules.
- **Entity Framework Core** maps C# domain objects to PostgreSQL tables, including pgvector column types for embedding storage [20]. Migrations are version-controlled and applied at startup.
- **FluentValidation** enforces input rules at the application boundary. Each request type has an associated validator that runs before the handler executes, rejecting invalid data before it reaches business logic.
- **Vertical slice architecture** [17] (described in Section 1.5.1) organises each feature as a self-contained folder containing the handler, request, response, endpoint, and validator. This colocation keeps feature concerns together rather than spread across technical layers.

1.5.3 Vue.js Frontend

The frontend uses Vue 3 with TypeScript and the Vite build tool [21]. Two surfaces share a common component library and state management:

- **Storefront.** Product catalog browsing with category trees, faceted filters (price, size, colour), and paginated result grids. Visual search with image upload, similarity-ranked results displaying thumbnail, price, and similarity score. Cart management with session-based guest support and a multi-step checkout flow spanning address entry, delivery selection, payment confirmation, and order completion.
- **Administration interface.** Complete lifecycle management for products, variants, taxonomies, and option types. Order processing with fulfilment status tracking. User and role administration with permission assignment. Analytics overview summarising sales and inventory metrics.
- **Pinia**, a state management library, organises client-side state through typed stores. Each bounded context has its own store (catalog, cart, auth, orders), mirroring the backend module boundaries.

1.5.4 PostgreSQL and pgvector

PostgreSQL 17 hosts both relational business data and vector embeddings in a single database [13].

- **Relational schema.** Tables for products, variants, orders, users, and supporting entities are organised by bounded context. Foreign keys reference entities within the same context; cross-context references use identifier columns without database-level constraints, preserving logical isolation.
- **Performance.** Composite indexes on query-critical combinations (user status, session status, product slug) optimise frequent access patterns. Query plans combine vector similarity and relational filtering in a single execution.

The pgvector extension, HNSW indexing, and transactional consistency between product data and embeddings are described in detail in Section 1.4.

1.5.5 Redis Caching

Redis 7 operates alongside the .NET **HybridCache** abstraction in a two-tier arrangement [22].

- **L1: in-process.** Frequently accessed data (taxonomy trees, front-page product lists) is held in application memory with sub-millisecond read latency. Cache entries expire on a configurable sliding window, typically five minutes.
- **L2: Redis.** The shared tier synchronises cache across application instances. Redis also backs Hangfire job queues and guest session storage. On cache miss at L1, the value is retrieved from Redis and promoted to L1, ensuring subsequent hits stay in-process.

1.5.6 Python ML Sidecar

The machine learning capability runs as a dedicated Python 3.12 service, isolated from the .NET backend due to incompatible runtime dependencies (PyTorch requires Python, .NET requires the CLR) [23].

- **Framework.** FastAPI provides async HTTP endpoints with automatic OpenAPI schema generation. Uvicorn serves as the ASGI runtime.
- **Model management.** A singleton **ModelManager** lazy-loads models from the HuggingFace hub on first request. Once loaded, models persist in GPU memory (or CPU, if no GPU is available) for the lifetime of the service. The manager supports multiple architectures through a common embedding interface.
- **API surface.** `POST /embeddings` accepts raw image bytes (JPEG, PNG, WebP) and returns a JSON array of floating-point values. `GET /health` reports the currently loaded model, its embedding dimension, and the most recent inference latency.
- **Security.** Requests require an `X-API-Key` header validated at the middleware layer. The sidecar listens only on the internal Docker network, inaccessible from the public internet.

1.5.7 Hangfire Background Jobs

Hangfire processes operations that should not block HTTP requests [24]. Jobs are persisted in Redis for resilience across application restarts.

- **Cart expiry.** A recurring job runs daily, removing carts with no activity for seven days. This prevents abandoned carts from accumulating indefinitely.
- **Embedding queue.** When an admin uploads new product images, embedding generation tasks are enqueued for asynchronous processing. The upload endpoint returns immediately; the embedding appears in search results once the job completes.
- **Index maintenance.** A periodic job rebuilds the HNSW index on the embedding column, optimising search performance as the catalog grows.

1.5.8 Identity, Authentication, and Authorisation

- **ASP.NET Identity** provides user account management: password hashing with salted PBKDF2, email confirmation workflows, and optional two-factor authentication via TOTP [19].
- **JWT tokens.** Access tokens carry claims (user ID, roles, permissions) and expire after 15 minutes [25]. Refresh tokens enable silent renewal: the client exchanges a refresh token for a new access token, and each refresh token is

single-use. Reuse of an already-consumed refresh token triggers revocation of all tokens for that user, mitigating token theft.

- **Guest sessions.** Anonymous users receive a signed session cookie. This cookie links to a server-side session backed by Redis, enabling cart operations and product browsing without authentication. On registration or login, the guest session is transferred to the authenticated user context.
- **Permission model.** Authorisation uses granular claims in the format `domain:category:action` (e.g., `catalog:products:create`). Roles aggregate commonly used permission sets. Endpoint-level attributes evaluate claims at the middleware layer, so handler code contains no authorisation logic.

1.5.9 Benchmark Framework

The benchmark framework is a Python 3.12 pipeline for systematic evaluation of embedding models [23], operating separately from the application codebase. Experimental results are reported in Chapter 3.

- **Modes.** One-shot comparison, three-fold cross-validation with stratified category splits (default), and pgvector pipeline mode measuring end-to-end latency.
- **Models.** 11 architectures: CNN (ResNet-50, ResNet-101, ResNet-152, EfficientNet-B0, EfficientNet-B4), ViT (DINOv2 ViT-S/14, DINOv2 ViT-B/14), CLIP (ViT-B/32, ViT-B/16, ViT-L/14, Fashion-CLIP). Each has a thin adapter implementing `generate_embeddings`.
- **Caching.** Embeddings cached per model, fold, and split to avoid recomputation.
- **Metrics.** Retrieval accuracy: mAP, Precision at K, Recall at K, nDCG. Efficiency: inference latency, throughput, model load time, storage, RAM.
- **Outputs.** JSON, CSV, Markdown, and Typst table formats embed directly into the thesis without manual transcription.
- **Multi-label pipeline.** Enriched-dataset mode evaluates three label schemes (category; category+colour; category+colour+pattern).

1.5.10 Technology Stack Summary

The preceding sections introduced the principal technologies that compose the ReSys.Shop platform. Table 8 consolidates the complete stack.

Table 8: Technology stack of the ReSys.Shop platform

Layer	Technology	Role
Frontend	Vue 3, TypeScript, Vite	Customer storefront and administration interface

Layer	Technology	Role
Backend API	.NET 10, Carter, MediatR	REST endpoints via minimal APIs with CQRS across business modules
Database	PostgreSQL, pgvector	Relational data and vector embeddings with HNSW-indexed similarity search
Caching	Redis, HybridCache	Two-tier cache (L1 in-memory, L2 Redis); Hangfire job queue backing store
ML Sidecar	Python 3.12, FastAPI, PyTorch	Embedding generation with lazy model loading and GPU acceleration
Background Jobs	Hangfire	Persistent job processing for cart expiry, embedding queue, and maintenance
Auth and Identity	JWT, ASP.NET Identity	Access tokens, refresh rotation, permission-based authorisation
Benchmarking	Python 3.12, PyTorch	Systematic 11-model comparison across retrieval accuracy and efficiency

1.6 RELATED WORK AND RESEARCH GAP

This section positions the ReSys.Shop platform within the landscape of fashion image retrieval research and commercial visual search systems, and identifies the engineering gap that this thesis addresses.

1.6.1 Academic Research

The **DeepFashion** dataset, introduced by Liu et al., established the foundational benchmark for fashion recognition and retrieval with over 800,000 images annotated with attributes, landmarks, and in-shop-to-consumer photo pairs [26]. This dataset catalysed much of the subsequent work in fashion AI.

FashionIQ extended retrieval to the conversational setting, where users modify queries through natural language feedback (“like this dress but shorter”) [27]. While compelling, the interactive dialogue paradigm requires infrastructure beyond the scope of this project, which focuses on single-turn visual and text queries.

The **Fashion-CLIP** work demonstrated that domain-specific fine-tuning of CLIP on 700,000 fashion images improves retrieval by 15 to 20% over the general model [6]. This thesis follows that approach, using pre-trained models without custom training, and extends the evaluation to additional architectures (ResNet, EfficientNet, DINOv2) for systematic comparison.

1.6.2 Commercial Systems

Several platforms have deployed visual search at production scale.

Table 9: Comparison of commercial visual search products

Product	Key Strength	Limitation
Google Lens	Massive scale, general-domain coverage	Closed ecosystem, not customisable
Pinterest Lens	Over 600M monthly searches, style-aware	Proprietary, requires Pinterest integration
ASOS Style Match	Fashion-specific accuracy	Restricted to ASOS catalog only
ViSenze	API-based, good accuracy	Paid service with recurring per-query costs

These products share common limitations for independent projects: they are proprietary and cannot be studied or modified, API access incurs costs at query volume, and reliance on external services creates vendor lock-in. This thesis demonstrates that comparable functionality is achievable with open-source tools, providing both a reference implementation and a cost-effective alternative for smaller deployments.

1.6.3 Contribution Differentiators

This project distinguishes itself from prior work by addressing the **engineering gap** between model research and production systems. Four contributions define this gap:

1. Polyglot architecture. Python's machine learning ecosystem (PyTorch, HuggingFace) does not natively interoperate with the .NET stack common in enterprise e-commerce. This thesis presents a modular monolith with a dedicated AI sidecar, combining .NET's type safety and transactional integrity with Python's access to state-of-the-art vision models, without the operational overhead of a full microservices deployment.

2. Vector-native consistency. By using pgvector within PostgreSQL, embeddings and product metadata share the same transactional boundary. Product updates, image replacements, and index maintenance occur atomically, eliminating stale-index bugs that arise when a vector store and relational database have independent consistency guarantees.

3. Commodity hardware benchmarking. Commercial visual search runs on cloud TPU clusters. This thesis evaluates 11 models on consumer-grade hardware, establishing that production-quality visual search is achievable without specialised infrastructure, lowering the barrier for small to medium e-commerce platforms.

4. Applied model comparison. Rather than chasing leaderboard metrics, this thesis compares models within realistic deployment constraints (inference

latency budget, memory limits, storage cost). The resulting accuracy-efficiency trade-off data, presented in Chapter 3, provides a pragmatic guide for practitioners selecting embedding models.

2 DESIGN AND IMPLEMENTATION

This chapter translates requirements into a concrete system design and describes the implementation of the principal components.

- **Requirements Specification.** 88 functional requirements, non-functional requirements, feature classification.
- **System Modeling.** Actors, work breakdown structure, 26 use cases with traceability to requirements.
- **System Architecture and Design.** Domain-driven design, C4 diagrams, database schema, API design, security model.
- **Implementation.** Technology stack, vertical slice architecture, data persistence with pgvector, ML sidecar and CBIR search flow, frontend applications.

2.1 REQUIREMENTS SPECIFICATION

The platform delivers 88 functional requirements across nine **business modules**, each enforcing domain invariants expressed through entity validation rules and application-layer checks. Five non-functional quality dimensions define performance, security, modularity, observability, and reliability targets that shaped architectural decisions throughout design. Feature classification distinguishes three **core research** contributions, detailed in Sections 2.3 and 2.4, from four **supporting infrastructure** modules that provide the realistic evaluation context described in Section 3.2.

- **Functional Requirements.** Traceable per module: Catalog, Identity, Inventory, Ordering, Payment, Shipping, Profile, Location, and Dashboard.
- **Non-Functional Requirements.** Five quality dimensions with measurable, atomic constraints.
- **Feature Classification.** Core Research versus Supporting Infrastructure: scope of the thesis contribution.

2.1.1 Functional Requirements

Functional requirements are organized by business module with unique identifiers traceable throughout design, implementation, and evaluation chapters.

2.1.1.1 Catalog Module

Manages product lifecycle, classification, image handling, and CBIR infrastructure.

Table 10: Catalog module functional requirements.

ID	Requirement	Description	Priority
CAT-FR-01	Product Creation	Create products with name, description, URL slug, and SEO metadata.	High

ID	Requirement	Description	Priority
CAT-FR-02	Fashion Metadata	Assign fashion attributes: style code, season, material composition, care instructions, fit notes, department, and gender target.	Medium
CAT-FR-03	Product Variants	Define sellable variants combining option values with SKU, barcode, physical dimensions, weight, and independent pricing.	High
CAT-FR-21	Variant Option Values	Assign, synchronize, retrieve, and revoke option values to define attribute combinations for variants.	High
CAT-FR-22	Variant Pricing	Set, list, synchronize, and remove time-bound prices per variant with currency specification.	Medium
CAT-FR-04	Variant Images	Upload variant images with automatic thumbnail generation, supporting multiple images per variant with display ordering.	High
CAT-FR-14	Variant Image CRUD	Delete, download, list, and update image metadata (alt text, display order) for variant images.	Medium
CAT-FR-15	Embedding Regeneration	Regenerate vector embeddings for images when the active model configuration changes or corrupted embeddings are detected.	Medium
CAT-FR-05	Embedding Generation	Generate vector embeddings for uploaded variant images via the ML sidecar; store in pgvector [13] with model metadata.	High
CAT-FR-06	Image-Based Search	Enable CBIR: accept query images, extract embeddings, query pgvector via HNSW indexing [12] using cosine distance, and return ranked results.	High
CAT-FR-16	Product Availability	Expose real-time per-variant stock availability through storefront product detail endpoints.	High
CAT-FR-17	Similar Products	Retrieve visually similar products based on vector embedding similarity for a target product.	Medium
CAT-FR-07	Search Configuration	Configure minimum similarity thresholds and maximum result limits for CBIR queries.	Medium
CAT-FR-08	Pluggable Models	Switch embedding models via environment variables without code modifications; filter search results by active model.	High

ID	Requirement	Description	Priority
CAT-FR-09	Taxonomy	Organize products via hierarchical taxonomies with nested taxon trees.	Medium
CAT-FR-18	Taxon Rules	Define business rules on taxon nodes with update, synchronization, and retrieval capabilities.	Low
CAT-FR-19	Auto-Classification	Automatically classify products into taxons based on taxon rules when product attributes change.	Low
CAT-FR-10	Option Types	Define option types (e.g., Size, Color, Material) with ordered values reusable across products.	Medium
CAT-FR-20	Option Association	Assign, synchronize, retrieve, and revoke option types on a per-product basis.	Medium
CAT-FR-11	Slug Uniqueness	Enforce URL slug uniqueness across the catalog at the database constraint level.	High
CAT-FR-12	Status Lifecycle	Support product lifecycles (Draft, Active, Archived) with configurable availability dates.	Medium
CAT-FR-13	Master Variant	Designate one variant per product as the master representation for catalog displays.	High

2.1.1.2 Identity Module

Authentication, authorization, and user management with JWT access tokens and refresh token rotation.

Table 11: Identity module functional requirements.

ID	Requirement	Description	Priority
IDN-FR-01	Registration	Register customer accounts with email, password, and basic profile information.	High
IDN-FR-02	Password Login	Authenticate via email and password, issuing a 15-minute JWT access token and refresh token.	High
IDN-FR-03	OAuth Login	Authenticate via Google OAuth 2.0 as an alternative to password credentials.	Medium
IDN-FR-04	Token Rotation	Invalidate previous refresh tokens upon issuance, returning a new access/refresh pair.	High

ID	Requirement	Description	Priority
IDN-FR-05	Reuse Detection	Revoke all active refresh tokens for a user if a previously consumed token is presented.	High
IDN-FR-06	Guest Sessions	Support guest sessions via signed cookies, enabling anonymous cart management and browsing.	High
IDN-FR-07	Role-Based Access	Enforce RBAC with permission claims formatted as `domain:category:action` at middleware boundaries.	High
IDN-FR-11	Role Management	Create, update, delete, and list roles; manage permission assignments per role.	Medium
IDN-FR-12	User-Role Assignment	Assign and revoke roles per user, supporting direct permission overrides.	Medium
IDN-FR-13	User Status	Enable or disable user accounts without purging user record history.	Medium
IDN-FR-08	Password Reset	Reset passwords via single-use, time-limited email verification tokens.	Medium
IDN-FR-14	Password Change	Allow authenticated users to update passwords upon verifying current credentials.	Medium
IDN-FR-15	Email Lifecycle	Confirm email addresses via verification tokens and support email modification requests.	Medium
IDN-FR-16	Session Management	Retrieve current sessions, inspect refresh tokens, and terminate sessions via logout.	High
IDN-FR-09	User Governance	Manage user accounts, role bindings, and permission grants via administrative endpoints.	Medium
IDN-FR-10	Two-Factor Auth	Provide optional TOTP-based two-factor authentication.	Low

2.1.1.3 Inventory Module

Stock tracking, checkout reservations, and audit ledgers.

Table 12: Inventory module functional requirements.

ID	Requirement	Description	Priority
INV-FR-01	Stock Locations	Define physical stock locations with address metadata and active flags.	Medium
INV-FR-02	Stock Quantities	Track on-hand and reserved quantities per variant per location.	High
INV-FR-08	Stock Item CRUD	Create, update, delete, and list stock items; support bulk adjustments and CSV imports.	Medium
INV-FR-09	Low Stock Alerting	Identify inventory falling below configured thresholds for replenishment notifications.	Low
INV-FR-03	Checkout Reservation	Reserve inventory during checkout; release upon cart expiry, cancellation, or timeout.	High
INV-FR-11	Cart Reservations	Reserve stock at cart level, inspect status, and auto-release upon abandonment.	High
INV-FR-04	Overselling Protection	Reject order confirmation when requested quantities exceed available unreserved stock.	High
INV-FR-05	Stock Transfers	Record stock movements between locations with origin, destination, quantity, and timestamp.	Medium
INV-FR-10	Transfer Lifecycle	Manage transfer states (Created, In-Transit, Received, Cancelled) with paged listings.	Medium
INV-FR-06	Audit Logging	Maintain an immutable audit log for stock changes recording operator identity and reason codes.	Medium
INV-FR-12	Movement History	Provide detailed, paged audit trails for stock movements as first-class domain entities.	Medium
INV-FR-07	Reservation Expiry	Expire stale inventory holds after a configurable timeout (default: 15 minutes).	Medium

2.1.1.4 Ordering Module

Shopping cart through checkout state transitions to completed orders.

Table 13: Ordering module functional requirements.

ID	Requirement	Description	Priority
ORD-FR-01	Shopping Cart	Support guest and customer carts with variant item addition and quantity adjustments.	High
ORD-FR-10	Cart Management	Create, clear, and delete carts, as well as modify item quantities and line items.	High
ORD-FR-02	Cart Association	Merge guest cart contents into customer accounts upon authentication without data loss.	Medium
ORD-FR-03	Cart Expiry	Purge abandoned carts inactive for 7 days via scheduled background maintenance tasks.	Medium
ORD-FR-04	Checkout Pipeline	Enforce strict sequential checkout state transitions: Address → Delivery → Payment → Confirm → Complete.	High
ORD-FR-11	Checkout Validation	Validate inventory levels, address completeness, and shipping method selection prior to payment.	High
ORD-FR-12	Shipping Selection	Select applicable shipping rates within cart boundaries prior to finalizing payment.	Medium
ORD-FR-05	Order Totals	Calculate monetary balances independently: $\text{Item Total} + \text{Adjustments} + \text{Shipping} = \text{Grand Total}$.	High
ORD-FR-06	Dual State Tracking	Maintain parallel, decoupled state machine counters for payment state and fulfillment state.	Medium
ORD-FR-07	Order Cancellation	Cancel orders prior to fulfillment confirmation, releasing inventory holds and voiding payments.	Medium
ORD-FR-13	Admin Order Management	Approve, complete, resume, and modify pending order details via administrative surfaces.	Medium
ORD-FR-14	Customer History	Expose paged customer order histories with line item detail, payment status, and tracking info.	Medium
ORD-FR-08	Order Numbering	Generate unique, sequential order reference numbers within database isolation transactions.	Medium
ORD-FR-09	Order Immutability	Lock finalized orders as immutable to prevent modification post-completion.	High

2.1.1.5 Payment Module

Payment intent lifecycles across Stripe and internal test gateways.

Table 14: Payment module functional requirements.

ID	Requirement	Description	Priority
PAY-FR-01	Intent Creation	Generate payment intents specifying amount, currency, order reference, and gateway target.	High
PAY-FR-02	Intent Confirmation	Confirm payment intents to authorize fund capture during checkout finalization.	High
PAY-FR-08	Setup Intent	Create Stripe SetupIntents to securely save customer payment methods for future checkouts.	Low
PAY-FR-03	Capture and Refund	Execute capture, void, and refund operations using idempotency keys.	High
PAY-FR-04	Webhook Processing	Verify and process incoming gateway webhooks using HMAC cryptographic signatures.	High
PAY-FR-09	Payment Voiding	Void authorized uncaptured payments and release authorization holds on cancelled orders.	Medium
PAY-FR-10	Method Management	Create, update, toggle, and delete configured payment gateway methods via admin panels.	Medium
PAY-FR-05	Refund Validation	Enforce validation rules preventing refund amounts from exceeding total captured funds.	Medium
PAY-FR-06	Bogus Gateway	Provide a mock payment gateway simulating transaction lifecycles without external API calls.	Medium
PAY-FR-07	State Tracking	Maintain local payment state records parallel to external gateway records for offline query support.	Medium

2.1.1.6 Shipping Module

Delivery options, geographic zones, and rate calculation.

Table 15: Shipping module functional requirements.

ID	Requirement	Description	Priority
SHP-FR-01	Delivery Methods	Configure shipping methods with carrier details, rate rules, and geographic zones.	Medium

ID	Requirement	Description	Priority
SHP-FR-04	Method Management	Create, update, activate, deactivate, and delete shipping methods via administrative endpoints.	Medium
SHP-FR-05	Rate Lifecycle	Manage rate matrices mapped to shipping methods, geographic zones, and weight/value tiers.	Medium
SHP-FR-02	Rate Calculation	Calculate shipping costs dynamically based on address zone, method, cart weight, and total value.	Medium
SHP-FR-06	Storefront Evaluation	Evaluate and display eligible shipping methods and rates during checkout.	High
SHP-FR-03	Shipment Tracking	Assign carrier identifiers and tracking tracking numbers to active shipments.	Low

2.1.1.7 Profile Module

Customer preferences, addresses, and saved items.

Table 16: Profile module functional requirements.

ID	Requirement	Description	Priority
PRF-FR-01	Address Book	Manage shipping and billing addresses with default selections and custom labels.	Medium
PRF-FR-02	Wishlists	Create, update, and remove named wishlists containing targeted product variants and notes.	Low
PRF-FR-03	Notification Settings	Manage channel-specific notification preferences (email, SMS) with opt-in flags.	Low

2.1.1.8 Location Module

Reference data for countries and states used in address validation and shipping zones.

Table 17: Location module functional requirements.

ID	Requirement	Description	Priority
LOC-FR-01	Country Directory	Maintain country reference entries with ISO 3166-1 alpha-2 codes, names, and active flags.	Medium

ID	Requirement	Description	Priority
LOC-FR-02	State Directory	Maintain state/province entries with ISO 3166-2 codes linked to parent countries.	Medium
LOC-FR-03	Country Management	Create, update, delete, and retrieve country records by database ID or ISO code.	Medium
LOC-FR-04	State Management	Create, update, delete, and list state records filtered by parent country.	Medium

2.1.2 Non-Functional Requirements

Five quality dimensions define production-readiness constraints with atomic, measurable targets. Verification is evaluated in Chapter 3.

Table 18: Non-functional requirements across five quality dimensions.

ID	Category	Requirement Target
NFR-01a	Performance	**End-to-End Search Latency:** Complete CBIR search workflows within **1 second** per query.
NFR-01b	Performance	**API Response Latency:** Non-search REST endpoints within **200 milliseconds** under standard load.
NFR-01c	Performance	**Non-Blocking Processing:** Enforce asynchronous I/O across all data access and HTTP pipelines.
NFR-02a	Security	**Token Lifecycle:** JWT access tokens [25] expire after **15 minutes** with single-use refresh token rotation.
NFR-02b	Security	**Endpoint Authorization:** Enforce fine-grained claims (`domain:category:action`) at the API middleware boundary.
NFR-02c	Security	**Rate Limiting:** Authentication **5 attempts/min**; registration **3 attempts/hour** per client IP.
NFR-02d	Security	**Upload Hardening:** Validate files via magic-byte header inspection, enforce extension allowlists, cap at **10 MB**.
NFR-02e	Security	**Transport Security:** Inject security headers (`CSP`, `HSTS`, `X-Frame-Options`, `X-Content-Type-Options`).
NFR-03a	Modularity	**Compile-Time Isolation:** Zero direct cross-module references within a single .NET assembly via build policy checks.
NFR-03b	Modularity	**Decoupled Messaging:** Route inter-module communications exclusively through MediatR in-process dispatch.
NFR-03c	Modularity	**Module Testability:** Each module independently testable without initializing foreign contexts.

ID	Category	Requirement Target
NFR-04a	Observability	**Distributed Tracing:** Propagate OpenTelemetry trace contexts across .NET and Python ML sidecar boundaries.
NFR-04b	Observability	**Structured Logging:** Unique correlation identifier in every log entry across distributed execution paths.
NFR-04c	Observability	**Health Monitoring:** Dedicated endpoints verifying database, cache, and sidecar connectivity.
NFR-05a	Reliability	**Job Durability:** Background tasks via Hangfire [24] backed by Redis [22] to survive process restarts.
NFR-05b	Reliability	**Automated Maintenance:** Daily purge of carts inactive for **7 days** and release associated inventory reservations.
NFR-05c	Reliability	**Webhook Idempotency:** Idempotency key checks on Stripe webhooks to prevent duplicate state updates.
NFR-05d	Reliability	**Reservation Timeouts:** Unconfirmed checkout inventory holds expire after **15 minutes** of inactivity.

2.1.3 Feature Classification

Platform features are categorized into Core Research contributions and Supporting Infrastructure to define thesis scope. Core contributions are detailed in Sections 2.3 and 2.4; supporting modules provide a realistic e-commerce environment for evaluation (Section 3.2).

Table 19: Classification of system feature areas into Core Research contributions and Supporting Infrastructure components.

Feature Area	Classification	Rationale
Visual Search (CBIR)	Core Research	**Primary Contribution:** Integrated multi-model CBIR system [28] with a pluggable architecture for image-based product discovery across CNNs [3], ViTs [10], and CLIP-based architectures [5], [6].
ML Embedding Pipeline	Core Research	**Operational Backbone:** Automated pipeline processing product images, generating vector embeddings via a PyTorch sidecar [23], and managing pgvector HNSW indexing [12], [13].
Model Benchmark System	Core Research	**Secondary Contribution:** Systematic benchmarking of retrieval accuracy and latency across 11 embedding models, providing model selection guidelines for deployment.
Product Catalog	Supporting	**Evaluation Domain:** Structured fashion product data serving as the vector search target.

Feature Area	Classification	Rationale
Order System	Supporting	**Conversion Tracking:** Shopping workflows capturing cart additions and checkouts to validate visual search utility.
Inventory	Supporting	**Availability Constraints:** Filters search queries by real-time stock levels.
Authentication	Supporting	**Security Boundary:** Protects administrative surfaces and user session state.

2.2 SYSTEM MODELING

Three actors interact with the platform across 26 use cases, organised into a functional work breakdown structure (WBS) and a summary matrix with detailed scenario specifications. The use case specifications provide full traceability to the functional requirements defined in Section 2.1.

- **System Actors.** Customer, Administrator, and System: roles, responsibilities, interaction surfaces.
- **Functional Decomposition.** WBS: three functional areas with constituent modules and sub-functions.
- **Use Cases.** 26 specifications: summary matrix, detailed scenarios with traceability.

2.2.1 System Actors

Three categories of actors interact with the platform, distinguished by access level and interaction surface.

2.2.1.1 Customer

The **Customer** accesses the browser-based **storefront**.

- **Discovery.** Browse catalog with faceted filters, keyword search, and visual CBIR (Section 2.3.3). Guests browse and cart without registration; session promoted on login.
- **Purchase.** Persistent cart, multi-step checkout (address, delivery, payment, confirm).
- **Account.** Register or Google OAuth login; manage profile, addresses, wishlists, order history.

2.2.1.2 Administrator

The **Administrator** operates a separate administration interface.

- **Catalog.** CRUD products with fashion metadata; variants and pricing; image uploads triggering embedding pipeline; taxonomies, option types, product classification.
- **Operations.** Order review, payment capture/refund, shipment management; real-time stock monitoring per location with audit history.
- **Governance.** User CRUD; role assignment (Customer, Administrator); permission grants (domain:category:action).

2.2.1.3 System

The **System** actor runs automated background jobs via **Hangfire** [24] and **Redis** [22].

- **Embedding Generation.** Process uploaded images via ML sidecar.
- **Cart Expiry.** Daily job: delete carts inactive 7 days, release inventory.
- **Inventory Reservation.** Hold stock during checkout; expire after 15-minute inactivity.
- **Maintenance.** Periodic HNSW index rebuilds; async payment webhook validation.

2.2.2 Functional Decomposition

Figure 6 presents the work breakdown structure (WBS) of the platform across three functional areas:

- **Administration.** Seven modules: Catalog, Ordering, Payment, Inventory, Identity, Shipping, Location.
- **Storefront.** Three modules: Product Discovery (browse, search, visual search), Purchase Flow (cart, checkout, payment, order history), Account Management (authentication, session, profile).
- **Background Services.** Five processes: Embedding Generation, Cart Expiry, Inventory Reservation, Payment Webhook Processing, Index Optimisation.

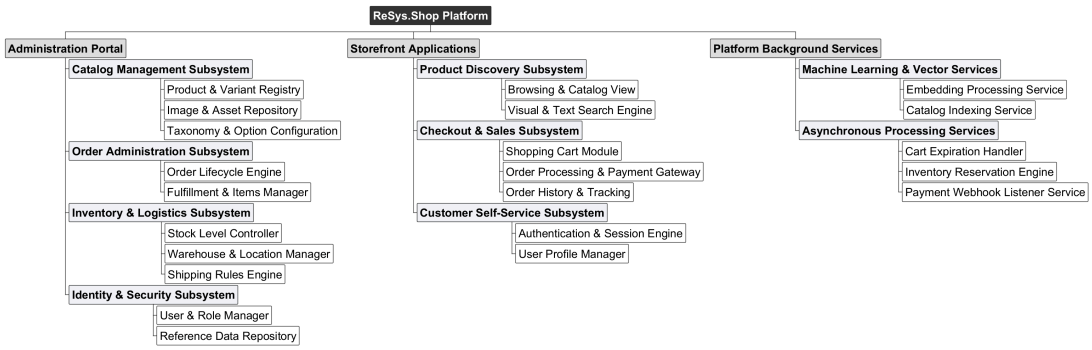

Figure 6: Functional decomposition of ReSys.Shop using a Work Breakdown Structure (WBS), showing the hierarchical breakdown into three functional areas and their constituent modules and sub-functions.

2.2.3 Administrator Use Cases

The Administrator actor manages data and operational workflows across all business modules through a dedicated administration interface. Each specification includes the actor's goal, trigger, preconditions, postconditions, a numbered main success scenario, alternative and exception flows, and related functional requirements.

2.2.3.1 Product Management

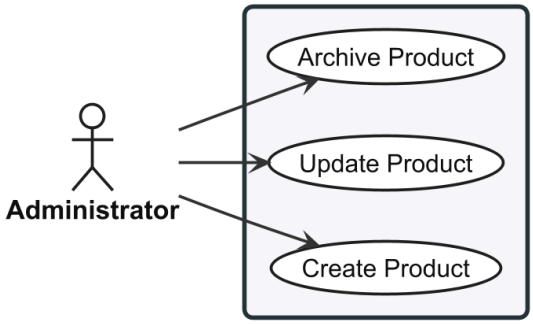

Figure 7: Use case diagram for Product Management (UC-ADM-PROD).

2.2.3.2 UC-ADM-PROD: Manage Products

Table 20: Manage Products.

Use Case	UC-ADM-PROD — Manage Products
Actor	Administrator
Goal	Create, update, and archive products in the catalog.
Pre/Post	Pre: authenticated with catalog management permissions. Post: product catalog reflects the performed operation; status transitions logged for audit.
Scenario	**Create Product** 1. Selects create option. 2. System presents form. 3. Enters product name, description, slug, and fashion metadata (style code, season, material, department, gender target). 4. Defines at least one variant with SKU and price as master variant. 5. Assigns product to relevant taxons. 6. Submits; system validates slug uniqueness, persists, confirms creation with Draft status. , **Update Product** 1. Selects product from catalog listing. 2. System displays edit form with current values. 3. Modifies fields (name, description, slug, fashion metadata, SEO attributes, or status). 4. Submits; system validates, persists, confirms update.

	, **Archive Product** 1. Selects product, chooses archive option. 2. System requests confirmation. 3. Confirms; product status changes to Archived, hidden from storefront. ,
Alternatives	1. A1. Slug not unique (Create/Update) → system rejects, prompts for different slug. 2. A2. No master variant (Create) → system rejects, instructs to designate one. 3. A3. Product has active orders (Archive) → system warns, requests explicit confirmation.
Exceptions	1. E1. System fails to persist → reports failure, retains form data for retry.
Requirements	CAT-FR-01, CAT-FR-02, CAT-FR-03, CAT-FR-11, CAT-FR-12, CAT-FR-13

2.2.3.3 Variant and Pricing

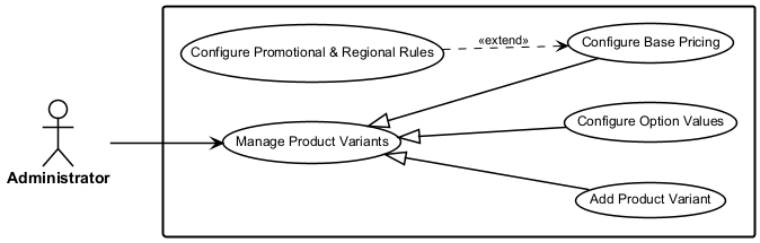

Figure 8: Use case diagram for Variant and Pricing (UC-ADM-VAR).

2.2.3.4 UC-ADM-VAR: Manage Variants

Table 21: Manage Variants.

Use Case	UC-ADM-VAR — Manage Variants
Actor	Administrator
Goal	Add and manage product variants including option-value configuration and pricing.
Pre/Post	Pre: authenticated with catalog permissions; parent product exists. Post: variant created or updated with option assignments and pricing.
Scenario	**Add Variant** 1. Navigates to product detail, selects add variant. 2. System presents variant creation form. 3. Enters variant details (SKU, barcode, dimensions, weight, position). 4. Assigns option values (e.g. Size M, Colour Red) from available option types.

	5. Submits; system validates SKU uniqueness and option combination, creates variant, confirms. , **Configure Options** 1. Selects variant from product detail page. 2. System displays variant detail with current option assignments. 3. Selects option type and chooses value, repeats for additional types. 4. Saves; system validates combination, persists, confirms change. , **Configure Pricing** 1. Navigates to pricing management for product. 2. System displays current prices for all variants grouped by currency. 3. Selects variants and specifies new price with currency and optional validity dates. 4. Submits; system validates non-negative prices and valid currency, persists, confirms. ,
Alternatives	1. A1. SKU not unique → system rejects, prompts for different SKU. 2. A2. Duplicate option combination → system rejects, highlights conflict. 3. A3. Zero price → system accepts but warns variant appears as free. 4. A4. Overlapping date ranges for same variant and currency → system rejects.
Exceptions	1. E1. Persistence failure → system reports, retains form data for retry.
Requirements	CAT-FR-03, CAT-FR-10, CAT-FR-21, CAT-FR-22

2.2.3.5 Image and Embedding Management

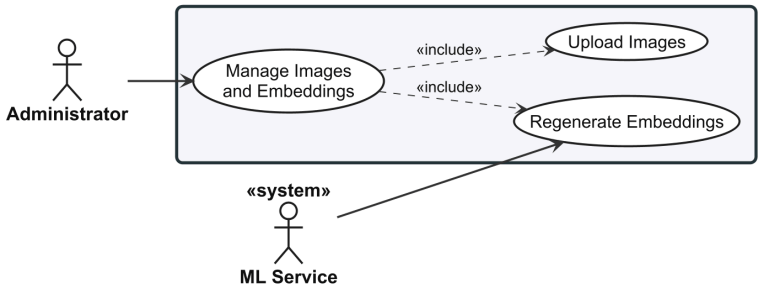

Figure 9: Use case diagram for Image and Embedding Management (UC-ADM-IMG).

2.2.3.6 UC-ADM-IMG: Manage Images and Embeddings

Table 22: Manage Images and Embeddings.

Use Case	UC-ADM-IMG — Manage Images and Embeddings

Actor	Administrator
Support	ML Service
Goal	Upload variant images and manage embedding generation.
Pre/Post	Pre: authenticated with image management permissions; variant exists. Post: image stored and associated with variant; embeddings available for visual search.
Scenario	**Upload Images** 1. Selects product variant, initiates image upload. 2. Selects image file from local file system. 3. System validates format (JPEG, PNG, WebP) and size (max 10 MB). 4. Provides alt text and display order position. 5. Confirms, submits; system stores image, generates thumbnails, schedules embedding generation, confirms upload. , **Regenerate Embeddings** 1. Navigates to image management, selects images. 2. Initiates regenerate embeddings action. 3. System displays confirmation with affected image count. 4. Confirms; system sends each image to ML service for embedding generation, stores embeddings with model metadata, reports completion with success count. ,
Alternatives	1. A1. Unsupported format → system rejects, lists accepted formats. 2. A2. Exceeds max size → system rejects, displays size constraint. 3. A3. No images selected for regeneration → system disables action, prompts to select.
Exceptions	1. E1. ML service unavailable → system reports failure, suggests retry when operational. 2. E2. Processing cannot be scheduled → system stores image, notifies search will exclude it until processing succeeds.
Requirements	CAT-FR-04, CAT-FR-05, CAT-FR-14, CAT-FR-15

2.2.3.7 Taxonomy and Classification

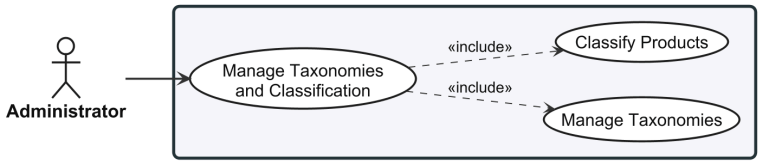

Figure 10: Use case diagram for Taxonomy and Classification (UC-ADM-TAX).

2.2.3.8 UC-ADM-TAX: Manage Taxonomies and Classification

Table 23: Manage Taxonomies and Classification.

Use Case	UC-ADM-TAX — Manage Taxonomies and Classification
Actor	Administrator
Goal	Create and modify taxonomy structures and assign product classifications.
Pre/Post	Pre: authenticated with taxonomy management permissions. Post: taxonomy structure and product classifications updated.
Scenario	**Manage Taxonomies** 1. Navigates to taxonomy management. 2. System displays taxonomy tree with existing taxons. 3. Creates new taxonomy root or selects existing taxonomy. 4. Adds, edits, reorders, or removes taxon nodes, optionally defines business rules. 5. Saves; system persists updated taxonomy, confirms. , **Classify Products** 1. Selects products from catalog listing. 2. Opens classification panel. 3. System displays taxonomy tree with current classifications. 4. Selects taxons to assign or deselects to remove. 5. Saves; system validates each taxon path, persists associations, confirms. ,
Alternatives	1. A1. Delete taxon with children → system prompts to cascade-delete or reassign. 2. A2. Delete taxon with products → system warns products lose classification. 3. A3. All taxons removed from product → system warns product will not appear in category browsing.
Exceptions	1. E1. Concurrent modification → system refreshes data, asks to retry.
Requirements	CAT-FR-09, CAT-FR-18, CAT-FR-19

2.2.3.9 Option Type Configuration

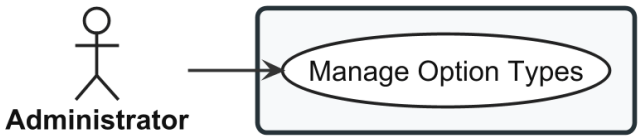

Figure 11: Use case diagram for Option Type Configuration (UC-ADM-OPT).

2.2.3.10 UC-ADM-OPT: Manage Option Types

Table 24: Manage Option Types.

Use Case	UC-ADM-OPT — Manage Option Types
Actor	Administrator

Goal	Create, modify, and remove option types and their associated values.
Pre/Post	Pre: authenticated with option type management permissions. Post: option types updated and available for product configuration.
Scenario	1. Navigates to option type management. 2. System displays all option types with their values. 3. Creates new option type with name and presentation style. 4. Adds ordered option values (e.g. S, M, L, XL for Size). 5. Optionally edits, reorders, or removes existing types and values. 6. Saves; system validates name uniqueness and non-empty values, persists, confirms.
Alternatives	1. A1. Delete type in use by products → system warns, asks for confirmation. 2. A2. Remove value in use by variants → system warns, asks for confirmation.
Exceptions	1. E1. Concurrent modification → system refreshes data, asks to retry.
Requirements	CAT-FR-10, CAT-FR-20

2.2.3.11 Order Lifecycle

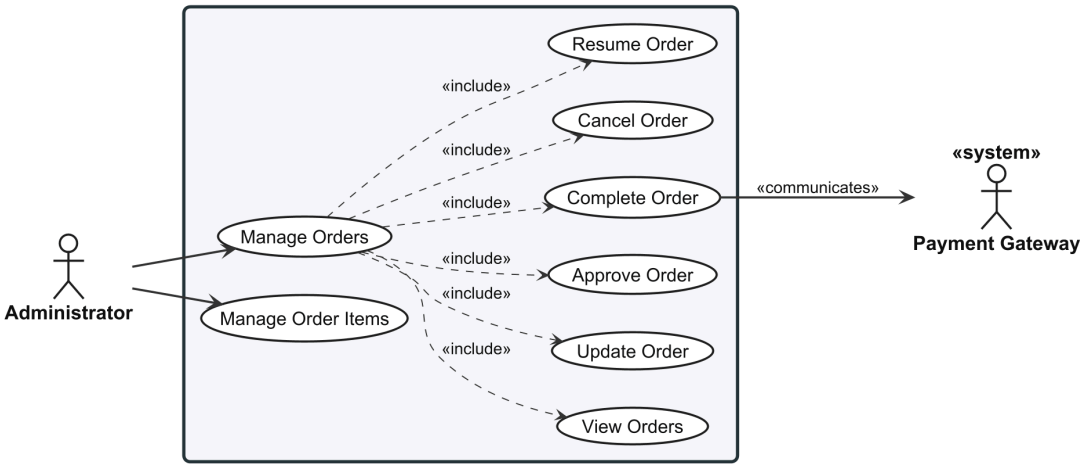

Figure 12: Use case diagram for Order Lifecycle (UC-ADM-ORD, UC-ADM-ORD-ITEMS).

2.2.3.12 UC-ADM-ORD: Manage Orders

Table 25: Manage Orders.

Use Case	UC-ADM-ORD — Manage Orders
Actor	Administrator
Support	Payment Gateway
Goal	View, modify, and manage the lifecycle of customer orders.

Pre/Post	Pre: authenticated with order management permissions. Post: order state transitions performed; status logged for audit.
Scenario	**View Orders** 1. Navigates to order management. 2. System displays order list sorted by most recent. 3. Applies optional filters (status, date range, customer email). 4. Selects order to view detail with line items, pricing, payment, shipment, addresses, timeline. , **Update Order** 1. Opens order, initiates edit. 2. System presents editable form with current values. 3. Modifies line items, addresses, or shipping method. 4. Submits; system validates modifications, recalculates totals, persists, confirms. , **Approve Order** 1. Opens pending order, verifies payment capture and inventory availability. 2. Selects approve; system validates, transitions order to approved, confirms. , **Complete Order** 1. Opens order in fulfilment state. 2. Verifies shipment dispatched. 3. Selects complete. 4. System displays confirmation. 5. Confirms; system decrements on-hand inventory, transitions to completed, logs, confirms. , **Cancel Order** 1. Opens order, selects cancel. 2. System displays confirmation with consequences summary. 3. Provides cancellation reason, confirms. 4. System releases reserved inventory, voids or refunds payment, transitions to cancelled, confirms. , **Resume Order** 1. Locates paused or stalled order. 2. Resolves underlying issue. 3. Selects resume; system validates prerequisites, transitions back to pending, confirms. ,
Alternatives	1. A1. No orders match → system displays empty message, suggests broadening filters. 2. A2. Payment not captured (Approve) → system prevents, suggests capturing first. 3. A3. Payment gateway unreachable (Cancel) → system cancels order, releases inventory, queues payment action. 4. A4. Prerequisites not met (Resume) → system prevents, displays issues to resolve. 5. A5. Concurrent state change → system refreshes, notifies.
Exceptions	1. E1. Order became immutable concurrently → system refreshes, notifies order is no longer editable.

	2. E2. Payment gateway state mismatch → system prevents, advises verifying with gateway. 3. E3. Inventory decrement data conflict → system reports, suggests verifying stock levels.
Requirements	ORD-FR-04, ORD-FR-05, ORD-FR-06, ORD-FR-07, ORD-FR-09, ORD-FR-13

2.2.3.13 UC-ADM-ORD-ITEMS: Manage Order Details

Table 26: Manage Order Details.

Use Case	UC-ADM-ORD-ITEMS — Manage Order Details
Actor	Administrator
Goal	Manage line items, shipping address, and billing address on existing orders.
Pre/Post	Pre: authenticated with order update permissions; order is in a mutable state. Post: order details updated with recalculated totals; change logged.
Scenario	1. Opens order from listing. 2. System displays order detail. 3. Initiates edit action. 4. System presents editable form with current values. 5. Modifies line items (add, update, remove), shipping address, or billing address. 6. Submits; system validates modifications (stock, address completeness), recalculates totals, persists, confirms.
Alternatives	1. A1. Quantity exceeds stock → system rejects, shows max available. 2. A2. All line items removed → system rejects, warns at least one is required. 3. A3. Address incompatible with shipping method → system warns, prompts method change.
Exceptions	1. E1. Order became immutable concurrently → system refreshes, notifies order is no longer editable.
Requirements	ORD-FR-13

2.2.3.14 Payment Processing

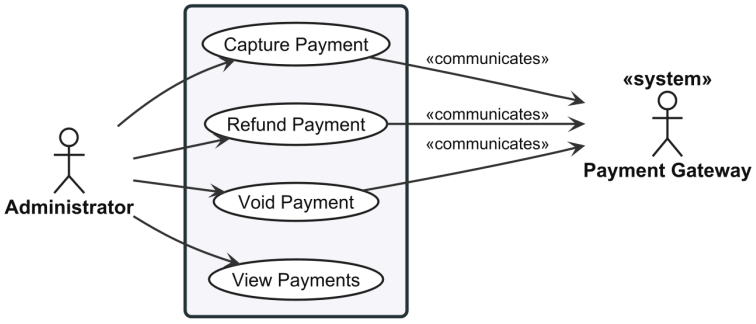

Figure 13: Use case diagram for Payment Processing (UC-ADM-PAY).

2.2.3.15 UC-ADM-PAY: Manage Payments

Table 27: Manage Payments.

Use Case	UC-ADM-PAY — Manage Payments
Actor	Administrator
Support	Payment Gateway
Goal	Capture, refund, void, and review payments.
Pre/Post	Pre: authenticated with payment management permissions. Post: payment state updated; gateway operations recorded.
Scenario	**Capture Payment** 1. Opens payment detail view for order. 2. System displays payment state, authorised amount, capture eligibility. 3. Optionally adjusts capture amount (partial) not exceeding authorised amount. 4. Confirms; system sends capture request to gateway with idempotency key, updates state to captured, confirms. , **Refund Payment** 1. Opens payment detail view for captured payment. 2. Enters refund amount (full or partial) and reason. 3. Confirms; system validates amount, sends refund to gateway, updates state, confirms. , **Void Payment** 1. Opens payment detail view for authorised but un-captured payment. 2. Selects void action, confirms. 3. System sends void request to gateway, updates state to voided, confirms. , **View Payments** 1. Navigates to payment management. 2. System displays payment list sorted by most recent. 3. Applies optional filters (state, date range, gateway, order number). 4. Selects payment to view full detail: amount, currency, gateway, state timeline, order reference, capture and refund history. ,
Alternatives	1. A1. Partial capture/refund → amount less than authorised/captured; remainder available. 2. A2. Already captured (Void) → system prevents, suggests refund instead. 3. A3. State mismatch (View) → system highlights discrepancy, suggests syncing.
Exceptions	1. E1. Gateway rejects → system reports rejection reason.

	2. E2. Gateway unreachable → system reports failure; idempotency key ensures safe retry.
Requirements	PAY-FR-03, PAY-FR-05, PAY-FR-07, PAY-FR-09

2.2.3.16 Payment Method Configuration

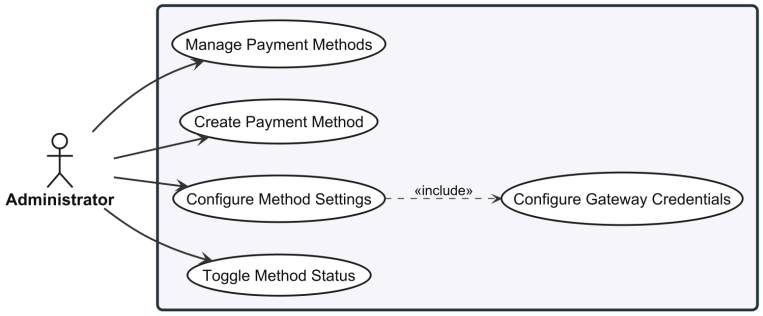

Figure 14: Use case diagram for Payment Method Configuration (UC-ADM-PAY-METHOD).

2.2.3.17 UC-ADM-PAY-METHOD: Manage Payment Methods

Table 28: Manage Payment Methods.

Use Case	UC-ADM-PAY-METHOD — Manage Payment Methods
Actor	Administrator
Goal	Create, update, activate, and deactivate payment methods.
Pre/Post	Pre: authenticated with payment method management permissions. Post: payment methods updated and available for storefront selection.
Scenario	1. Navigates to payment method configuration. 2. System displays configured payment methods with activation status. 3. Creates new method with name, description, gateway identifier, and parameters. 4. Optionally edits, activates, deactivates, or removes methods. 5. Saves; system validates name uniqueness and gateway support, persists, confirms.
Alternatives	1. A1. Deactivate method in use → system warns new orders cannot use it; existing unaffected. 2. A2. Remove method → system verifies no pending payments reference it. 3. A3. Invalid gateway parameters → system rejects, highlights invalid ones.
Exceptions	1. E1. Concurrent modification → system refreshes, asks to retry.
Requirements	PAY-FR-10

2.2.3.18 Stock Location Management

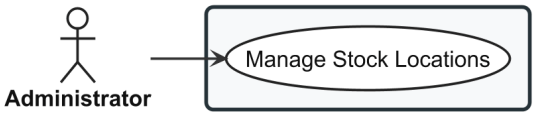

Figure 15: Use case diagram for Stock Location Management (UC-ADM-LOC).

2.2.3.19 UC-ADM-LOC: Manage Stock Locations

Table 29: Manage Stock Locations.

Use Case	UC-ADM-LOC — Manage Stock Locations
Actor	Administrator
Goal	Create, update, and remove stock locations.
Pre/Post	Pre: authenticated with location management permissions. Post: location configuration updated.
Scenario	1. Navigates to stock location management. 2. System displays existing stock locations with addresses and active status. 3. Creates new location with name, address, and active status flag. 4. Optionally designates new location as default for stock intake. 5. Optionally edits, deactivates, or removes existing locations. 6. Saves; system validates location name uniqueness, persists, confirms.
Alternatives	1. A1. Delete location with active stock → system prevents, requires transfer first. 2. A2. Deactivate location → system prevents new intake but allows existing movements. 3. A3. Delete last location → system prevents; at least one must remain.
Exceptions	1. E1. Concurrent modification → system refreshes, asks to retry.
Requirements	INV-FR-01

2.2.3.20 Stock Item Management

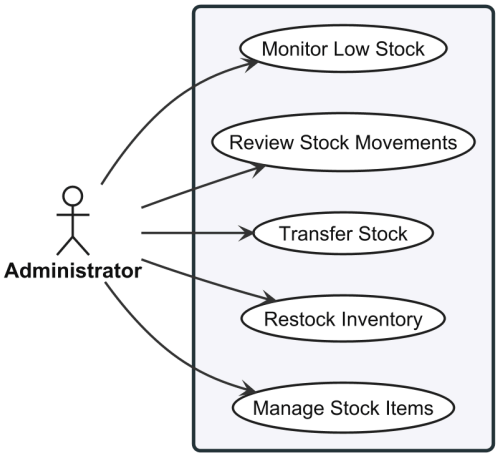

Figure 16: Use case diagram for Stock Item Management (UC-ADM-STK).

2.2.3.21 UC-ADM-STK: Manage Stock

Table 30: Manage Stock.

Use Case	UC-ADM-STK — Manage Stock
Actor	Administrator
Goal	Create, update, restock, transfer, and monitor stock levels.
Pre/Post	Pre: authenticated with stock management permissions. Post: stock quantities updated; changes logged for audit.
Scenario	**Manage Stock Items** 1. Navigates to stock item management. 2. System displays stock items with variant, location, on-hand, and reserved quantities. 3. Creates stock item by selecting variant and location, enters initial on-hand quantity. 4. Alternatively selects existing stock item and updates on-hand quantity. 5. Provides reason for adjustment. 6. Saves; system validates variant-location uniqueness, persists, records audit log, confirms. , **Restock Inventory** 1. Locates stock item in management interface. 2. Enters restock quantity, provides reference and notes. 3. Confirms; system increments on-hand quantity, records restock event, confirms updated quantities. , **Transfer Stock** 1. Navigates to stock transfer, initiates new transfer. 2. Selects source location, destination, variant, quantity. 3. System validates sufficient stock at source. 4. Submits; system creates transfer record pending, decrements source.

	5. Upon arrival, confirms receipt; system increments destination, transitions to completed, confirms. , **Review Stock Movements** 1. Navigates to stock movement audit interface. 2. System displays all stock movements in reverse chronological order with pagination. 3. Applies optional filters (date range, variant, location, movement type). 4. Selects movement to view full detail. , **Monitor Low Stock** 1. Navigates to low stock monitoring view. 2. System displays stock items below configured threshold with variant, location, on-hand, threshold, days since last restock. 3. Reviews list, identifies items needing replenishment. ,
Alternatives	1. A1. Bulk adjustment via file upload → system processes, validates, reports success/failure counts. 2. A2. Reduce below reserved quantity → system rejects, shows current reserved. 3. A3. Transfer exceeds available stock → system rejects, shows maximum. 4. A4. Cancel pending transfer → system returns deducted quantity to source, logs cancellation. 5. A5. No items below threshold (Low Stock) → system displays message that all stock levels are sufficient.
Exceptions	1. E1. Variant-location pair already exists → system rejects, suggests editing existing. 2. E2. Concurrent modification → system refreshes, asks to re-enter. 3. E3. Retrieval failure → system displays error, offers retry.
Requirements	INV-FR-02, INV-FR-05, INV-FR-06, INV-FR-08, INV-FR-09, INV-FR-10, INV-FR-12

2.2.3.22 User Management

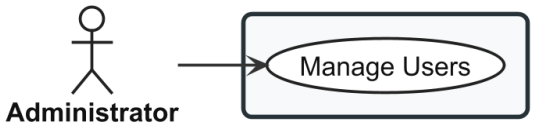

Figure 17: Use case diagram for User Management (UC-ADM-USR).

2.2.3.23 UC-ADM-USR: Manage Users

Table 31: Manage Users.

Use Case	UC-ADM-USR — Manage Users

Actor	Administrator
Goal	Create, update, enable, and disable user accounts.
Pre/Post	Pre: authenticated with user management permissions. Post: user account created or modified; status changes take effect on next authentication.
Scenario	1. Navigates to user management. 2. System displays user list with default sorting and pagination. 3. Applies optional filters (email, name, role, account status). 4. Creates new user account by entering email, name, assigning roles. 5. Alternatively selects existing user to view detail or edit. 6. Modifies profile fields, enables/disables account, or adjusts role assignments. 7. Saves; system validates email uniqueness, persists, confirms; if account was disabled, all active sessions are revoked.
Alternatives	1. A1. Disable account → system revokes all active sessions immediately. 2. A2. Re-enable disabled account → system restores active status. 3. A3. Email already registered → system rejects, prompts for different email.
Exceptions	1. E1. Persistence failure → system reports, retains form data for retry.
Requirements	IDN-FR-09, IDN-FR-13

2.2.3.24 Role and Permission Governance

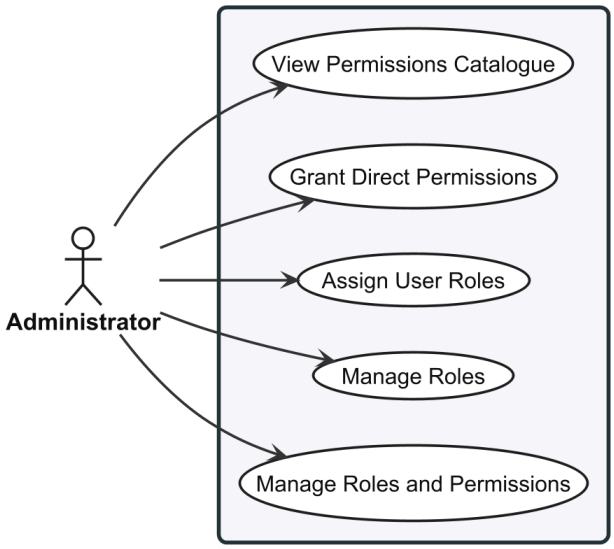

Figure 18: Use case diagram for Role and Permission Governance (UC-ADM-ROL).

2.2.3.25 UC-ADM-ROL: Manage Roles and Permissions

Table 32: Manage Roles and Permissions.

Use Case	UC-ADM-ROL — Manage Roles and Permissions
Actor	Administrator
Goal	Create and manage roles, assign permissions to roles, and grant roles to users.
Pre/Post	Pre: authenticated with role and permission management rights. Post: role and permission configuration updated; affected users receive updated permissions on next token.
Scenario	**Manage Roles** 1. Navigates to role management. 2. System displays list of roles with name, description, permission count. 3. Creates new role with name and description. 4. Assigns permissions from permissions catalogue. 5. Optionally edits existing role's name, description, or permission set. 6. Saves; system validates role name uniqueness, persists, confirms. , **Assign User Roles** 1. Opens user's detail page. 2. Opens role assignment panel. 3. System displays all available roles with checkboxes for current assignments. 4. Selects roles to assign and deselects to revoke. 5. Saves; system persists, displays updated effective permissions. , **Grant Direct Permissions** 1. Opens user's detail page. 2. Opens direct permission panel. 3. System displays permissions catalogue with role-inherited and direct-grant indicators. 4. Selects permissions to grant directly and deselects to revoke. 5. Saves; system persists, recalculates effective permissions. , **View Permissions Catalogue** 1. Navigates to permissions catalogue. 2. System displays all permission claims grouped by domain and module. 3. Applies optional filters (domain, category, keyword search). 4. Reviews permission matrix to plan role designs or audit assignments. ,
Alternatives	1. A1. Remove role assigned to users → system warns affected users lose permissions. 2. A2. Revoke permission from role → system warns all users with this role lose it on next token. 3. A3. Revoke last role from user → system warns user loses all role-derived permissions. 4. A4. Grant already inherited from role → system accepts; effective permission unchanged but direct grant recorded.
Exceptions	1. E1. Concurrent modification or role deleted → system refreshes, asks to retry.

Requirements	IDN-FR-11, IDN-FR-12

2.2.3.26 Shipping Method Configuration

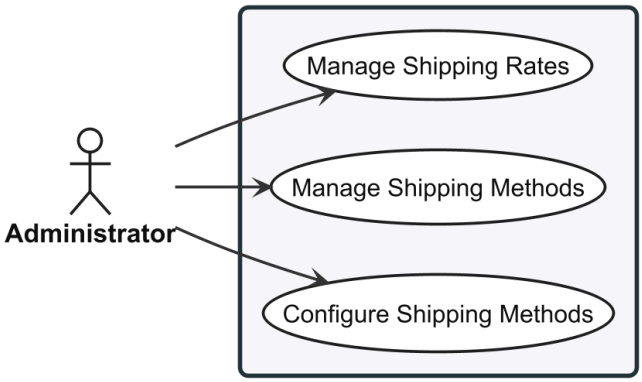

Figure 19: Use case diagram for Shipping Method Configuration (UC-ADM-SHP).

2.2.3.27 UC-ADM-SHP: Manage Shipping

Table 33: Manage Shipping.

Use Case	UC-ADM-SHP — Manage Shipping
Actor	Administrator
Goal	Configure delivery methods and their associated shipping rates.
Pre/Post	Pre: authenticated with shipping management permissions. Post: shipping methods and rates configured; active methods available for checkout if zone-applicable.
Scenario	**Manage Shipping Methods** 1. Navigates to shipping method management. 2. System displays configured shipping methods with activation status. 3. Creates new method with name, description, carrier identifier, and applicable zones. 4. Optionally edits, activates, deactivates, or removes existing methods. 5. Saves; system validates method name uniqueness, persists, confirms. , **Manage Shipping Rates** 1. Selects shipping method from method listing. 2. System displays current rate tiers for method. 3. Creates new rate tier: selects zone, defines weight and cart value ranges, enters rate amount. 4. Optionally edits or removes existing rate tiers. 5. Saves; system validates rate tier ranges do not overlap for same zone, persists, confirms. ,

Alternatives	1. A1. Deactivate method in active checkouts → system warns; existing selections unaffected. 2. A2. Remove method with active rates → system prompts to remove rates or reassign. 3. A3. No zones assigned → system warns method will not be selectable at checkout. 4. A4. Ranges overlap with existing tiers → system rejects, highlights conflicting tiers.
Exceptions	1. E1. Concurrent modification → system refreshes, asks to retry.
Requirements	SHP-FR-01, SHP-FR-04, SHP-FR-05

2.2.3.28 Reference Data Management

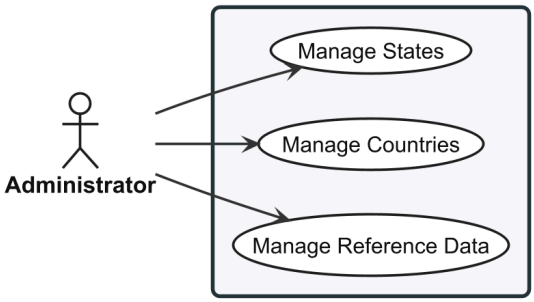

Figure 20: Use case diagram for Reference Data Management (UC-ADM-REF).

2.2.3.29 UC-ADM-REF: Manage Reference Data

Table 34: Manage Reference Data.

Use Case	UC-ADM-REF — Manage Reference Data
Actor	Administrator
Goal	Create and update country and state reference data.
Pre/Post	Pre: authenticated with reference data management permissions. Post: country and state data updated; active records available in address forms and shipping zones.
Scenario	**Manage Countries** 1. Navigates to country management. 2. System displays list of countries with name, ISO code, and active status. 3. Creates new country record with display name and ISO 3166-1 code. 4. Sets active status flag; optionally edits or removes existing country records. 5. Saves; system validates ISO code uniqueness and format, persists, confirms. , **Manage States** 1. Navigates to state management.

	2. System displays list of states with name, ISO code, parent country, and active status. 3. Creates new state record with display name, ISO 3166-2 code, selects parent country. 4. Sets active status flag; optionally edits or removes existing state records. 5. Saves; system validates ISO code uniqueness within parent country, persists, confirms. ,
Alternatives	1. A1. Deactivate country → system warns addresses retained but country not in new forms. 2. A2. Delete country with associated states → system warns states will be orphaned. 3. A3. Delete country in shipping zones → system warns, suggests deactivating instead. 4. A4. Deactivate parent country → system prompts whether to cascade-deactivate associated states.
Exceptions	1. E1. ISO code already exists → system rejects, prompts for different code.
Requirements	LOC-FR-01, LOC-FR-02, LOC-FR-03, LOC-FR-04

2.2.4 Customer Use Cases

The Customer actor interacts through the browser-based storefront for product discovery, purchase, and account management. Each specification follows the same structure as the Administrator use cases.

2.2.4.1 Catalog Browsing

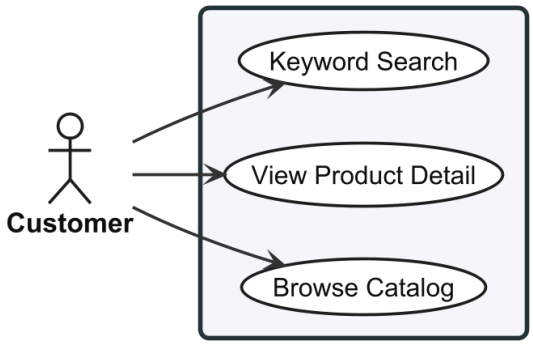

Figure 21: Use case diagram for Catalog Browsing (UC-STR-BRW).

2.2.4.2 UC-STR-BRW: Browse and Search Catalog

Table 35: Browse and Search Catalog.

Use Case	UC-STR-BRW — Browse and Search Catalog

Actor	Customer
Goal	Browse the product catalog, view product details, and search by keyword.
Pre/Post	Pre: catalog data is published and searchable. Post: product listing or detail displayed with pricing and availability.
Scenario	**Browse Catalog** 1. Visits storefront home page or category landing page. 2. System displays taxonomy tree with top-level categories. 3. Selects category or subcategory from taxonomy navigation. 4. System retrieves products associated with selected taxon and its descendants. 5. System displays paginated grid of product cards showing thumbnail, name, price, availability. 6. Applies optional facets to filter by option types (e.g. size, colour) and other attributes. 7. System refreshes listing with filtered results. , **View Product Detail** 1. Clicks product card from catalog listing, search result, or recommendation. 2. System displays product detail page with primary image, name, description, price range. 3. System shows fashion metadata (style code, season, material, department, gender target). 4. System displays taxonomy breadcrumb showing category path. 5. Browses product images in gallery. 6. Selects variant option values from available options. 7. System updates displayed price, availability, variant-specific images based on selection. , **Keyword Search** 1. Enters search keyword or phrase in storefront search bar. 2. System queries catalog for products matching name, description, or fashion attributes. 3. System ranks results by relevance, displays paginated grid of matching product cards. 4. Refines search with additional keywords or applies filters. ,
Alternatives	1. A1. No products in category → system displays empty state suggesting other categories. 2. A2. Selected variant out of stock → system shows out-of-stock status, disables add-to-cart. 3. A3. No products match keyword → system displays suggestions to check spelling or browse categories. 4. A4. Very short query (single character) → system prompts to enter at least two characters.
Exceptions	1. E1. Retrieval or search failure → system displays error, offers retry.
Requirements	CAT-FR-01, CAT-FR-02, CAT-FR-03, CAT-FR-09, CAT-FR-16, CAT-FR-22

2.2.4.3 Search

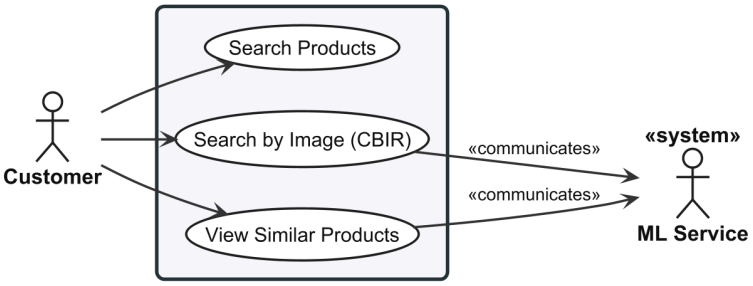

Figure 22: Use case diagram for Search (UC-STR-SRC).

2.2.4.4 UC-STR-SRC: Visual Search

Table 36: Visual Search.

Use Case	UC-STR-SRC — Visual Search
Actor	Customer
Support	ML Service
Goal	Search for products by uploading a reference image and discover visually similar items.
Pre/Post	Pre: catalog has products with embeddings; ML service is operational. Post: visually similar products displayed with similarity scores above configured threshold.
Scenario	**Search by Image (CBIR)** 1. Navigates to visual search interface. 2. Selects or drags image file from local file system. 3. System validates image format (JPEG, PNG, WebP) and size constraints. 4. System sends image to ML service for embedding generation. 5. System performs similarity search against image embeddings index. 6. System retrieves matching products ranked by similarity score above minimum threshold. 7. System displays results as grid of product thumbnails with scores, linking to detail pages. , **View Similar Products** 1. Opens product detail page with image embeddings available. 2. System retrieves product's primary image embedding. 3. System performs similarity search against other product embeddings. 4. System displays horizontal carousel or grid of visually similar product cards. 5. Clicks on similar product card to navigate to its detail page. ,
Alternatives	1. A1. Invalid image format → system rejects, lists accepted formats.

	2. A2. No products above threshold → system displays empty result suggesting different image or keyword search. 3. A3. No images or embeddings on detail page → system hides similar products section. 4. A4. ML service unavailable → system gracefully hides section without affecting detail page.
Exceptions	1. E1. ML service unavailable → system displays error, suggests retrying later. 2. E2. Embedding index empty → system reports visual search not yet available.
Requirements	CAT-FR-06, CAT-FR-07, CAT-FR-08, CAT-FR-17

2.2.4.5 Cart Management

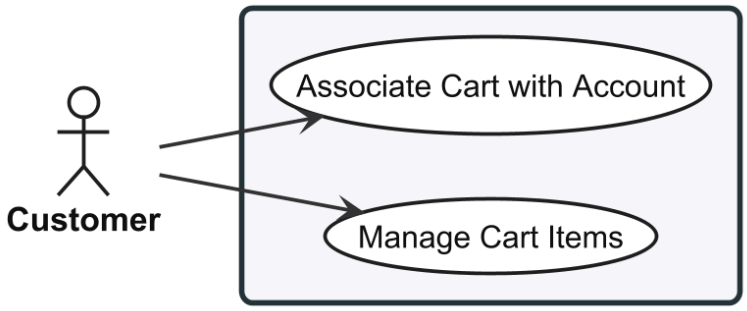

Figure 23: Use case diagram for Cart Management (UC-STR-CRT).

2.2.4.6 UC-STR-CRT: Manage Cart

Table 37: Manage Cart.

Use Case	UC-STR-CRT — Manage Cart
Actor	Customer
Goal	Add, update, and remove items in a shopping cart; associate guest cart with account.
Pre/Post	Pre: product variant exists and is available. Post: cart persisted across page navigation; guest carts survive browser sessions.
Scenario	**Manage Cart Items** 1. On product detail page, selects variant and specifies quantity. 2. Clicks Add to Cart; system validates variant and quantity. 3. System adds variant to customer's cart, displays confirmation. 4. Opens cart to view all items with quantities, prices, subtotal. 5. Updates quantity of item or removes item. 6. System recalculates cart subtotal, updates display. , **Associate Cart with Account** 1. Browses storefront as guest, adds items to cart. 2. Logs in or registers for account. 3. System detects guest cart exists, retrieves any existing user cart.

	4. System merges guest and user carts: matching variants increase quantity, unique variants are added. 5. System associates merged cart with user account, invalidates guest cart cookie. ,
Alternatives	1. A1. Quantity exceeds stock → system rejects, shows max available. 2. A2. Same variant already in cart → system increments existing quantity. 3. A3. Guest customer → system assigns session-based cart identifier in signed cookie. 4. A4. No existing user cart → system transfers guest cart to user account directly. 5. A5. Merge exceeds available stock → system caps at max available, notifies customer.
Exceptions	1. E1. Variant deactivated or archived → system rejects, suggests refreshing product page. 2. E2. Merge fails due to data conflict → system creates user cart with guest items, notifies to review.
Requirements	ORD-FR-01, ORD-FR-02, ORD-FR-10

2.2.4.7 Checkout Flow

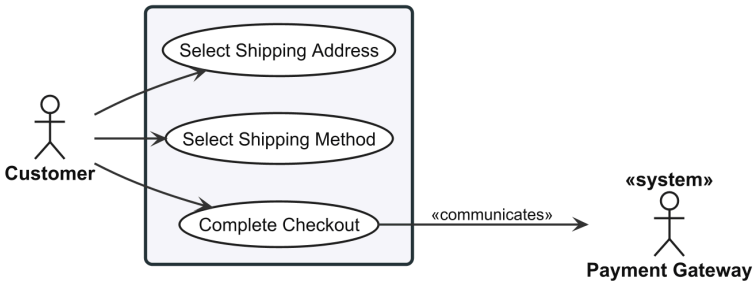

Figure 24: Use case diagram for Checkout Flow (UC-STR-CHK).

2.2.4.8 UC-STR-CHK: Checkout

Table 38: Checkout.

Use Case	UC-STR-CHK — Checkout
Actor	Customer
Support	Payment Gateway
Goal	Complete the multi-step checkout: address selection, shipping method, and order confirmation.
Pre/Post	Pre: customer is authenticated; cart is not empty; stock is available for all items. Post: order created with unique number; inventory reserved; payment linked; cart cleared.
Scenario	**Select Shipping Address**

	1. From cart, clicks Proceed to Checkout. 2. System transitions checkout to Address step. 3. System displays saved addresses with default pre-selected. 4. Selects existing shipping address or enters new one. 5. System determines shipping zone, proceeds to next step. , **Select Shipping Method** 1. System calculates available shipping methods and rates based on address zone, cart weight, cart value. 2. System displays list of available methods with rates and estimated delivery times. 3. Reviews options, selects preferred shipping method. 4. System applies selected method and rate, updates shipment total in order summary. 5. System presents next checkout step (payment). , **Complete Checkout** 1. System displays order summary with line items, shipping, tax, total. 2. Enters payment details or selects saved payment method. 3. System creates payment intent, reserves inventory for each line item. 4. Reviews final order summary, confirms purchase. 5. System captures payment, generates unique order number. 6. System transitions order to Confirmed state, clears cart, displays order confirmation. 7. System sends order confirmation notification. ,
Alternatives	1. A1. No saved addresses → system presents empty address step with prompt to create new. 2. A2. No methods available for zone → system displays message, prompts different address. 3. A3. Only one method available → system auto-selects, proceeds. 4. A4. Stock depleted during checkout → system notifies, removes affected items, returns to cart. 5. A5. Payment failure → system notifies with reason, allows retry; inventory not reserved.
Exceptions	1. E1. Address validation fails → system highlights missing fields, prevents progression. 2. E2. Rate calculation fails → system displays error, suggests contacting support. 3. E3. Payment captured but inventory reservation fails → system voids payment, notifies order not completed.
Requirements	ORD-FR-04, ORD-FR-05, ORD-FR-08, ORD-FR-11, ORD-FR-12

2.2.4.9 Order History

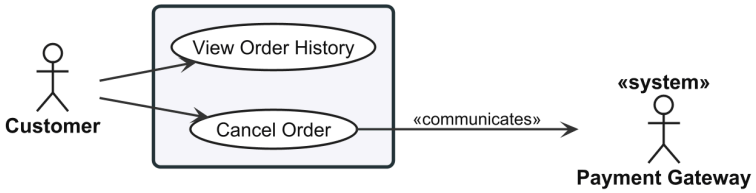

Figure 25: Use case diagram for Order History (UC-STR-OHI).

2.2.4.10 UC-STR-OHI: Order History

Table 39: Order History.

Use Case	UC-STR-OHI — Order History
Actor	Customer
Support	Payment Gateway
Goal	View past orders and cancel pending orders.
Pre/Post	Pre: customer is authenticated. Post: complete order history visible; cancelled orders release inventory and void payment.
Scenario	**View Order History** 1. Navigates to order history from account menu. 2. System displays past orders in reverse chronological order with pagination, showing order number, date, status, and total. 3. Applies optional date range or status filters. 4. Selects an order to view full detail: line items, shipping address, method, payment state, shipment state, and status timeline. , **Cancel Order** 1. Opens order detail from order history. 2. System displays order detail with current status and cancel action if cancellable. 3. Selects Cancel Order. 4. System displays confirmation explaining inventory release and payment void. 5. Confirms; system releases reserved inventory, voids payment, transitions to cancelled, and sends confirmation. ,
Alternatives	1. A1. No orders → system displays message with prompt to browse catalog. 2. A2. Order state changed since page loaded → system refreshes and informs cancellation unavailable.
Exceptions	1. E1. Payment gateway unreachable (Cancel) → system cancels order, releases inventory, queues void, notifies customer. 2. E2. Retrieval failure (View) → system displays error and offers retry.
Requirements	ORD-FR-07, ORD-FR-14

2.2.4.11 Payment Processing

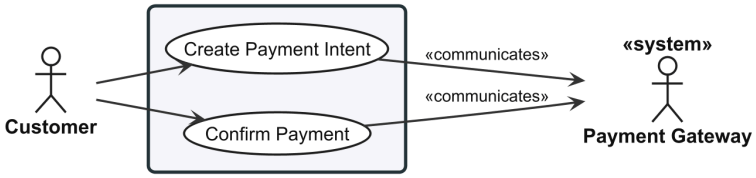

Figure 26: Use case diagram for Payment Processing (UC-STR-PAY).

2.2.4.12 UC-STR-PAY: Payment Processing

Table 40: Payment Processing.

Use Case	UC-STR-PAY — Payment Processing
Actor	Customer
Support	Payment Gateway
Goal	Create and confirm payment for an order.
Pre/Post	Pre: customer is authenticated; checkout has reached payment step; order has a calculated total. Post: payment authorised; order transitions to Confirmed state.
Scenario	**Create Payment Intent** 1. System displays payment step with order total and available payment methods. 2. Selects a payment method and enters payment details. 3. System submits payment details to gateway to create intent. 4. System receives confirmation intent is ready and displays review summary. , **Confirm Payment** 1. Reviews final order summary including line items, shipping, tax, and total. 2. Clicks Confirm Payment. 3. System submits payment intent confirmation to gateway. 4. System receives authorisation confirmation. 5. System updates payment state, transitions order to Confirmed, clears cart, and displays order confirmation. ,
Alternatives	1. A1. Invalid payment details → system returns gateway validation error, prompts correction. 2. A2. Authorisation declined → system displays reason and offers retry with different method. 3. A3. Additional authentication required → system redirects to authentication flow and resumes after.
Exceptions	1. E1. Payment gateway unreachable → system displays error; checkout progress saved. 2. E2. Payment confirms but order creation fails → system voids payment and notifies to retry.

	3. E3. Gateway timeout → system marks payment pending; advises checking order history; webhook updates state.
Requirements	PAY-FR-01, PAY-FR-02

2.2.4.13 Authentication

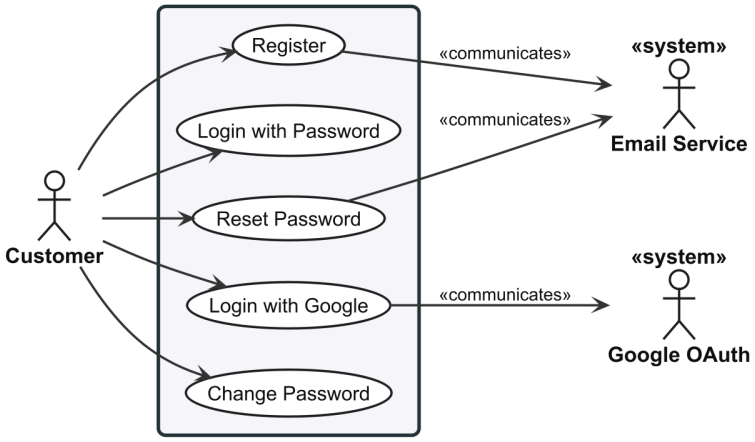

Figure 27: Use case diagram for Authentication (UC-STR-AUT).

2.2.4.14 UC-STR-AUT: Authentication

Table 41: Authentication.

Use Case	UC-STR-AUT — Authentication
Actor	Customer
Support	Email Service, Google OAuth
Goal	Register, log in, and manage account credentials.
Pre/Post	Pre: none (public access). Post: customer authenticated or credentials updated.
Scenario	**Register** 1. Navigates to registration page. 2. Enters email, password, and basic profile information. 3. Submits; system validates email not registered and password strength, creates account, sends verification; confirms. , **Login with Password** 1. Navigates to login page and enters email and password. 2. Submits; system validates credentials, issues access and refresh tokens, associates guest cart if exists; redirects to home page. , **Login with Google** 1. Clicks Login with Google on the storefront. 2. System redirects to Google's OAuth consent screen. 3. Grants consent; system exchanges code, looks up or creates account, issues tokens; redirects to home page.

	, **Reset Password** 1. Clicks Forgot Password and submits registered email. 2. System generates time-limited reset token and sends via email. 3. Opens email, clicks reset link, enters new password. 4. Submits; system validates token, updates password, revokes all sessions; confirms. , **Change Password** 1. Navigates to account security settings and selects Change Password. 2. Enters current password and new password. 3. Submits; system verifies current password, validates new password, updates, revokes all sessions except current; confirms. ,
Alternatives	1. A1. Email already registered → system rejects and suggests login with password reset link. 2. A2. Password does not meet strength requirements → system highlights requirements and prompts retry. 3. A3. Invalid credentials → system rejects with generic message. 4. A4. Account disabled → system rejects with message to contact support. 5. A5. Reset token expired → system displays message and prompts new reset request. 6. A6. Current password incorrect (Change) → system rejects and prompts retry.
Exceptions	1. E1. Verification or reset email fails to send → system creates account/request but logs failure internally. 2. E2. Token issuance fails → system reports failure and suggests retry.
Requirements	IDN-FR-01, IDN-FR-02, IDN-FR-03, IDN-FR-08, IDN-FR-14

2.2.4.15 Session Management

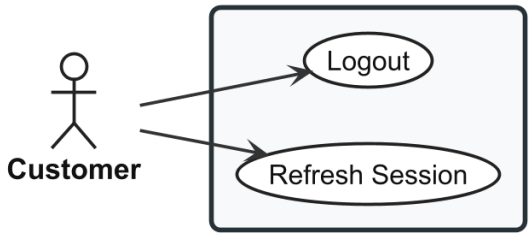

Figure 28: Use case diagram for Session Management (UC-STR-SES).

2.2.4.16 UC-STR-SES: Session Management

Table 42: Session Management.

Use Case	UC-STR-SES — Session Management
Actor	Customer

Goal	Maintain and terminate authenticated sessions.
Pre/Post	Pre: customer has an active session. Post: session extended or terminated.
Scenario	**Refresh Session** 1. Client detects access token will expire soon. 2. Client sends current refresh token to token endpoint. 3. System validates refresh token (not expired, not consumed). 4. System issues new access token with fresh expiry. 5. System issues new refresh token and invalidates previous (token rotation). 6. Client stores new token pair and continues session. , **Logout** 1. Clicks Logout in the storefront navigation. 2. System sends current refresh token for invalidation. 3. System invalidates the refresh token and removes the session cookie. 4. System redirects to the storefront home page. ,
Alternatives	1. A1. Refresh token expired → system rejects; client redirects to login. 2. A2. Refresh token consumed (reuse detected) → system revokes all tokens; client redirects to login with security notification. 3. A3. Logout request fails due to network → system clears local tokens and cookies; session expires naturally.
Exceptions	1. E1. Token issuance or invalidation fails → client retains existing pair if access token still valid; clears locally otherwise.
Requirements	IDN-FR-04, IDN-FR-05, IDN-FR-16

2.2.4.17 Profile and Preferences

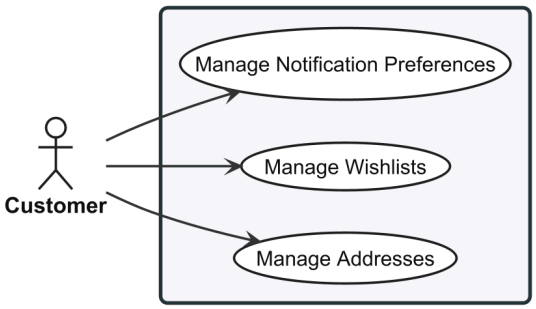

Figure 29: Use case diagram for Profile and Preferences (UC-STR-PRF).

2.2.4.18 UC-STR-PRF: Profile Management

Table 43: Profile Management.

Use Case	UC-STR-PRF — Profile Management

Actor	Customer
Goal	Manage shipping addresses, wishlists, and notification preferences.
Pre/Post	Pre: customer is authenticated. Post: profile data and preferences updated.
Scenario	**Manage Addresses** 1. Navigates to address book page. 2. System displays saved addresses with type labels and default indicators. 3. Creates a new address: enters name, street, city, postal code, country, and state. 4. Assigns address type (shipping, billing, or both) and optionally sets as default. 5. Optionally edits or removes existing addresses. 6. Saves; system validates required fields and address format, persists, and confirms. , **Manage Wishlists** 1. On a product detail page, clicks Add to Wishlist. 2. System displays dialog to select wishlist or create new. 3. Selects an existing wishlist or creates a new named wishlist; optionally adds a note. 4. System adds the product variant to the selected wishlist and confirms. 5. Navigates to wishlists section to view all with item counts and preview thumbnails. 6. Selects a wishlist to view items, remove items, rename, or delete the list. , **Manage Notification Preferences** 1. Navigates to notification preferences page. 2. System displays all notification categories with current per-channel opt-in/out status. 3. Toggles individual notification categories on or off per channel. 4. Saves; system persists and confirms. ,
Alternatives	1. A1. Same variant already in wishlist → system notifies and suggests adding note to existing entry. 2. A2. Remove address used in active orders → system warns it is referenced by past orders (retained); allows soft-delete. 3. A3. Opts out of all notifications → system warns important notifications also suppressed; asks confirmation. 4. A4. Deletes wishlist → system asks for confirmation; list and items permanently removed.
Exceptions	1. E1. Address validation fails for invalid country/state → system highlights mismatch and prevents save. 2. E2. Product variant archived since added to wishlist → system displays with Unavailable label and remove suggestion. 3. E3. Persistence failure → system reports failure and retains unsaved changes for retry.
Requirements	PRF-FR-01, PRF-FR-02, PRF-FR-03

2.2.5 System Use Cases

The System actor represents automated background processes that maintain data consistency, generate embeddings, and perform scheduled operations. Coordination relies on **Hangfire** [24] for job scheduling and **Redis** [22] for persistence.

2.2.5.1 Embedding Operations

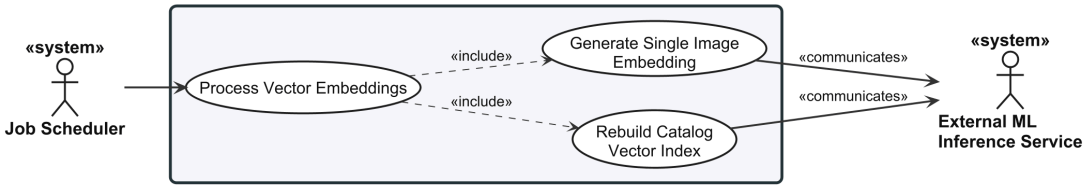

Figure 30: Use case diagram for Embedding Operations (UC-SYS-EMB).

2.2.5.2 UC-SYS-EMB: Embedding Operations

Table 44: Embedding Operations.

Use Case	UC-SYS-EMB — Embedding Operations
Actor	System
Support	ML Service
Goal	Generate and regenerate image embeddings for product search.
Pre/Post	Pre: images uploaded and stored; ML service is operational. Post: embeddings stored in the index for visual search.
Scenario	**Generate Image Embeddings** 1. System detects a new unprocessed image in the queue. 2. System retrieves the image file from storage. 3. System preprocesses the image (resize, normalise) for model requirements. 4. System sends preprocessed image to ML service with configured model identifier. 5. ML Service computes the visual embedding vector and returns it with metadata. 6. System stores embedding in the index with image reference and metadata. 7. System marks the image as processed. , **Regenerate All Embeddings** 1. System detects change to embedding model configuration. 2. System validates new model is available via ML service health check. 3. System retrieves list of all product images requiring regeneration. 4. System sends each image to ML service using the new model. 5. ML Service computes new embedding and returns it. 6. System stores new embedding replacing previous and updates model metadata. 7. System reports completion with success/failure count summary.

	,
Alternatives	1. A1. Image not accessible (deleted/corrupted) → system marks as failed and records reason. 2. A2. Multiple images queued → system processes sequentially, throttling to ML service capacity. 3. A3. Triggered during peak traffic (Regenerate) → system throttles to avoid impacting search performance. 4. A4. No model change detected → system logs event and skips processing.
Exceptions	1. E1. ML service returns error → system retries with exponential backoff; after max retries marks as failed. 2. E2. ML service unreachable → system requeues image and triggers alert. 3. E3. New model not available (Regenerate) → system aborts regeneration; existing embeddings remain in use.
Requirements	CAT-FR-05, CAT-FR-15

The embedding generation process described in the main success scenario follows the pipeline illustrated in Figure 31: an image is authenticated, pre-processed through standardised transforms, passed through the selected model, and the resulting feature vector is serialised alongside performance metadata.

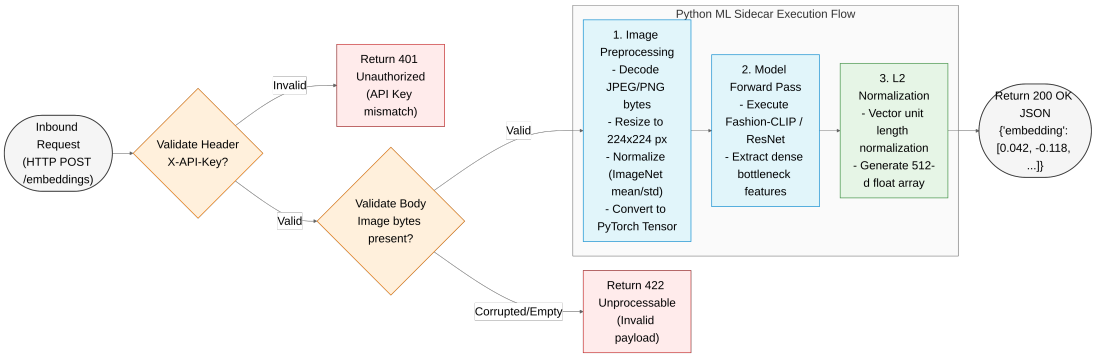

Figure 31: ML embedding pipeline: step-by-step flow from image upload to normalised vector response.

2.2.5.3 Background Maintenance

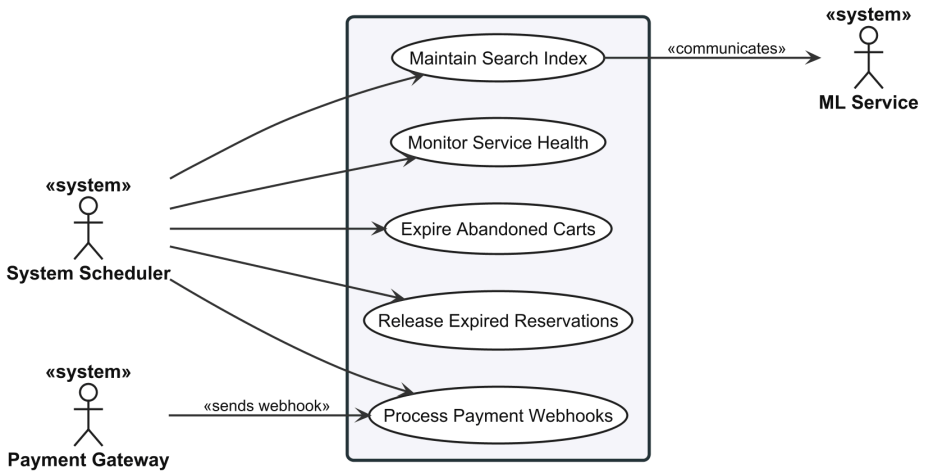

Figure 32: Use case diagram for Background Maintenance (UC-SYS-MNT).

2.2.5.4 UC-SYS-MNT: System Maintenance

Table 45: System Maintenance.

Use Case	UC-SYS-MNT — System Maintenance
Actor	System
Support	ML Service, Payment Gateway
Goal	Perform scheduled and event-driven system maintenance tasks.
Pre/Post	Pre: maintenance schedule is active; required services are operational. Post: system health, data consistency, and index performance maintained.
Scenario	**Monitor Service Health** 1. System invokes health check endpoint on ML service at configured interval. 2. ML Service responds with current health status: healthy, degraded, or unhealthy. 3. System records health status and response time. 4. System updates service availability flag; if unhealthy, routes search to degraded path. , **Expire Abandoned Carts** 1. System executes scheduled job at configured time. 2. System queries for carts with no modification in past seven days. 3. For each abandoned cart, releases reserved inventory back to availability. 4. System deletes or marks abandoned cart for cleanup and logs counts. , **Release Expired Reservations** 1. System executes scheduled job at configured interval. 2. System queries for reservations with hold time exceeding configured expiry window (default 15 min).

	3. For each expired reservation, releases reserved quantity back to available stock. 4. System marks reservation record as expired and logs counts. , **Process Payment Webhooks** 1. Payment Gateway sends signed webhook notification to system endpoint. 2. System validates cryptographic signature against configured gateway secret. 3. System extracts payment identifier, event type, and payload. 4. System verifies not a duplicate by checking idempotency key. 5. System updates local payment state and triggers any required order state transitions. 6. System returns success response to gateway. , **Maintain Search Index** 1. System executes scheduled job at configured interval. 2. System analyses current embedding index state: vector count, index size, recent query performance. 3. System determines whether maintenance is likely to improve performance. 4. System performs maintenance (e.g. index rebuild, dead tuple removal, statistics update). 5. System verifies maintenance completed and logs execution with before/after metrics. ,
Alternatives	1. A1. No carts/items meet criteria → system logs zero and completes. 2. A2. Duplicate webhook → system returns success without changes; duplicate logged. 3. A3. Out-of-order webhook event → system logs anomaly and applies state change. 4. A4. Index below maintenance threshold → system skips and logs decision with current metrics. 5. A5. Active search queries during index maintenance → system performs lighter, non-blocking operations.
Exceptions	1. E1. Health check endpoint does not respond → system marks unhealthy after timeout and triggers alert. 2. E2. Database unavailable → system records failure and retries on next execution. 3. E3. Webhook signature validation fails → system rejects with authentication error. 4. E4. Index maintenance fails → system logs error and triggers alert; search continues with current index. 5. E5. Index appears corrupted → system triggers full rebuild from stored embeddings.
Requirements	CAT-FR-06, CAT-FR-08, ORD-FR-03, INV-FR-07, PAY-FR-04

2.3 SYSTEM ARCHITECTURE & DESIGN

Six dimensions define the platform architecture, from service composition through domain modelling to database, API, and security layers. The design fol-

lows a service-oriented approach: a Vue 3 frontend, a .NET 10 modular monolith backend, and a Python FastAPI machine learning sidecar, each independently deployable.

- **System Overview.** Three services, eight bounded contexts, technology stack summary.
- **Domain-Driven Design.** Context map, aggregate roots with invariants, state machines.
- **C4 Architecture.** Context, container, and component-level structural views.
- **Database Design.** Per-context schemas, pgvector integration, core design decisions.
- **API Design.** Carter minimal APIs, MediatR CQRS, endpoint conventions.
- **Security Design.** JWT authentication, permission-based authorisation, defensive hardening.

2.3.1 System Overview

ReSys.Shop comprises three services – a Vue 3 frontend, a .NET 10 backend [19], and a Python **FastAPI** ML sidecar [23] – and eight **bounded contexts** using **Domain-Driven Design** with MediatR dispatch between modules.

Table 46: System services and their technology stacks. Each service communicates through well-defined HTTP contracts.

Service	Technology Stack	Responsibilities
Vue Frontend	Vue 3 + TypeScript + Vite	• Customer storefront and administrator dashboard • Image upload and visual search UI
.NET Backend	.NET 10 + ASP.NET Core + Carter + EF Core	• REST API via Carter minimal APIs and MediatR CQRS • PostgreSQL persistence with pgvector and Redis caching
Python ML	Python 3.12 + FastAPI + PyTorch	• Embedding model inference (Fashion-CLIP and others) • Multi-model support with lazy-loading strategy

Internally, the backend is partitioned into nine bounded contexts, each owning a dedicated database schema. Table 47 lists each context, its aggregate root, and key domain entities.

Table 47: Bounded contexts with aggregate roots and key domain entities. Each context owns its database schema and communicates through MediatR dispatch only.

Bounded Context	Aggregate Root	Key Domain Entities
Catalog	Product	Variant, VariantImage, OptionType, OptionValue, Classification, Taxonomy, Taxon
Ordering	Order	LineItem, Adjustment, Shipment
Payment	PaymentIntent	PaymentCapture
Inventory	StockItem	StockLocation, StockMovement, StockReservation
Identity	User	Role, UserRole, RefreshToken, UserLogin
Profile	UserProfile	Address, Wishlist
Shipping	ShippingMethod	ShippingRate, ShippingZone
Location	Country	State

2.3.2 Domain-Driven Design

The backend follows **Domain-Driven Design** (DDD) with eight bounded contexts, each managing explicit aggregate boundaries and domain invariants under a Conformist integration pattern via Published Language contracts.

2.3.2.1 Bounded Context Map

The platform is partitioned into eight **bounded contexts** along business capability boundaries. Each context independently owns its state model, business logic, and vocabulary. Integration follows a **Conformist** pattern with core abstractions from the Shared Kernel (`Result<T>`, `ICommand`, `IQuery`). Context-to-context communication relies exclusively on **MediatR** `ISender` in-process dispatch without direct compile-time project references.

Figure 33 illustrates the eight contexts and their inter-module communication pathways. Table 48 details each context's business capabilities and Published Language boundaries.

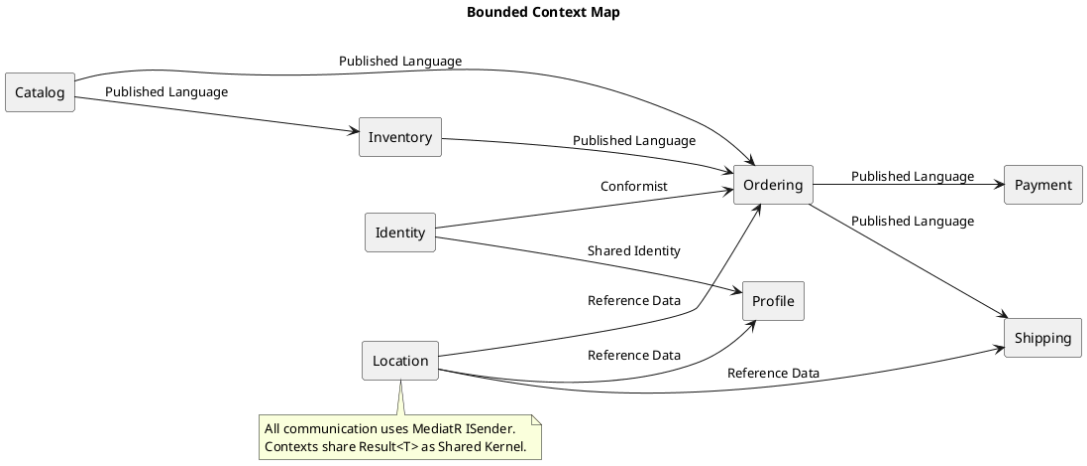

Figure 33: Bounded Context Map showing the eight business contexts with Published Language identifiers exchanged via in-process MediatR dispatch.

Table 48: Bounded context responsibilities and Published Language identifiers. The Published Language column lists the value types that other contexts may reference by identifier only, never by importing the source context's namespace.

Context	Business Responsibility	Published Language
Catalog	• Product lifecycle, variants with SKU and independent pricing • Image management with automated vector embedding generation • Hierarchical taxonomy classification	`ProductId`, `VariantId`, `Sku`, `Price`, `Slug`
Ordering	• Checkout workflow with forward-only state machine • Line item capture with locked price snapshots	`OrderId`, `OrderNumber`, `Total`, `Currency`, `CheckoutState`
Payment	• Payment intent lifecycle (authorize, capture, refund, void) • Local state mirroring for offline operation	`PaymentIntentId`, `PaymentState`, `Amount`
Inventory	• Stock tracking with reservations to prevent overselling • Append-only audit logging for stock adjustments	`StockItemId`, `QuantityOnHand`, `QuantityReserved`
Identity	• JWT authentication with refresh token rotation • RBAC with `domain:category:action` claims	`UserId`, `Email`, `PermissionClaim`
Profile	• Customer address book and wishlists	`ProfileId`, `AddressId`
Shipping	• Delivery method definitions, rate evaluation, and zone configuration	`ShippingMethodId`, `Rate`
Location	• ISO 3166 country and state reference data	`CountryId`, `StateId`, `IsoCode`

2.3.2.2 Aggregates and Invariants

An aggregate root defines a transactional consistency boundary, managing child entities and enforcing core business rules using a pragmatic DDD strategy without base class overhead or forced domain event dispatches.

- **Product (Catalog Root):** Manages product families, variants, media, options, and taxonomy mappings. Enforces slug uniqueness and a designated master variant. Variant images store 512-dimensional vector embeddings for visual search.

- **Order (Ordering Root):** Governs the purchasing pipeline with locked price snapshots at checkout initialization. Enforces the structural balance invariant

$$\text{Total} = \text{ItemTotal} + \text{AdjustmentTotal} + \text{ShipmentTotal}$$

  . Completed orders are immutable except through formal administrative cancellations.
- **PaymentIntent (Payment Root):** Models transactional authorization states (`Pending`, `RequiresAction`, `Processing`, `Succeeded`, `Canceled`, `Failed`). Enforces cumulative capture debit limits and mirrors gateway states locally.
- **StockItem (Inventory Root):** Tracks on-hand and reserved stock per warehouse location. Enforces non-negative balance constraints

$$\text{QuantityOnHand} \geq 0$$

  and writes all adjustments to an append-only ledger.

2.3.2.3 Ubiquitous Language Glossary

The ubiquitous language establishes a common domain vocabulary across software design and business rules. Table 49 defines key domain terms.

Table 49: Ubiquitous Language Glossary across domain contexts.

Term	Context	Definition
Product	Catalog	Core entity with shared metadata (description, slug, taxonomy). Prices and SKUs belong exclusively to Variants.
Variant	Catalog	Purchasable unit with SKU, barcode, price, dimensions, and stock flags.
Master Variant	Catalog	Primary variant for product listings. One required per product.
Option Type	Catalog	Reusable attribute categories (e.g., Color, Size) shared across products.
Taxonomy / Taxon	Catalog	Hierarchical classification with nested category nodes.
Cart	Ordering	Temporary purchase container. Purged after 7 days of inactivity.
Line Item	Ordering	Entry linking a variant to a quantity and locked price snapshot.
Checkout State	Ordering	Sequential stages (Address → Delivery → Payment → Confirm → Complete) with forward-only progression.
Payment Intent	Payment	Tracks authorization states across payment gateway interactions.
Payment Capture	Payment	Monetary debit executed against an authorized payment intent.

Term	Context	Definition
Stock Item	Inventory	Warehouse record tracking on-hand and reserved stock per location.
Stock Movement	Inventory	Immutable audit entry recording quantity deltas, operator, and reason.
Refresh Token	Identity	Single-use token for JWT access token generation. Reuse triggers session revocation.

2.3.2.4 State Machines

Explicit state machines inside domain entities govern checkout and payment workflows, validating transitions prior to database persistence.

2.3.2.4.1 Order Checkout State Machine

Checkout advances through five forward-only stages: Address, Delivery, Payment, Confirm, and Complete (Figure 34). Users can cancel at any pre-completion stage.

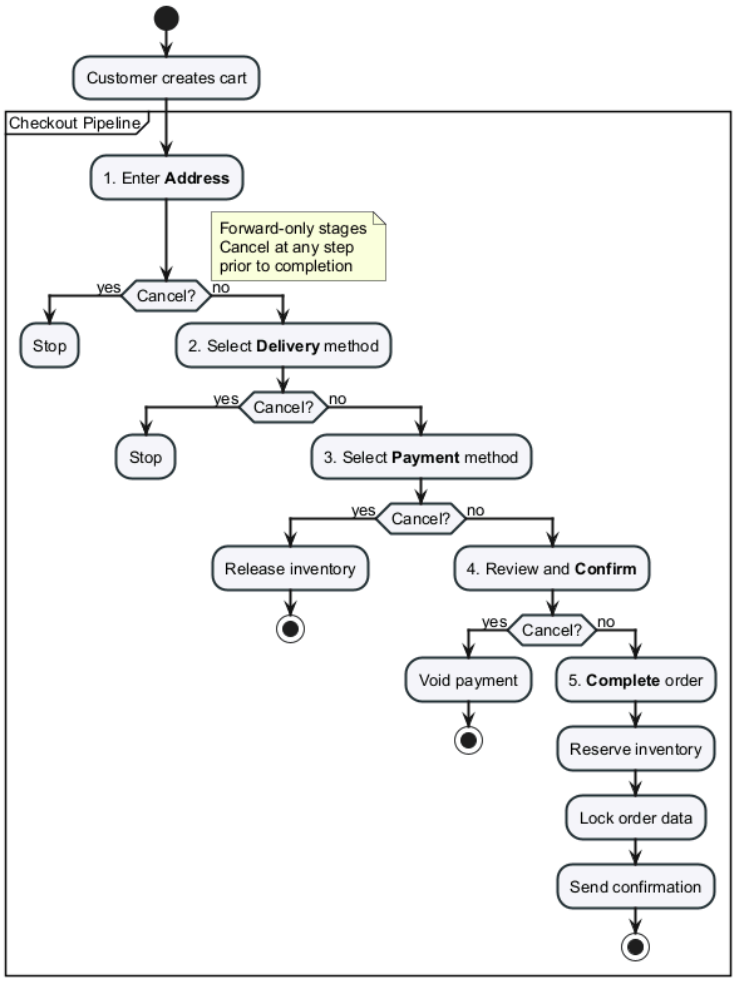

Figure 34: Order checkout state machine showing forward-only stages and cancellation paths.

2.3.2.4.2 Payment Intent State Machine

Payment intents follow a lifecycle from `Pending` through `Processing` to `Succeeded` or `Failed` (Figure 35). Authorized intents support capture and refund operations.

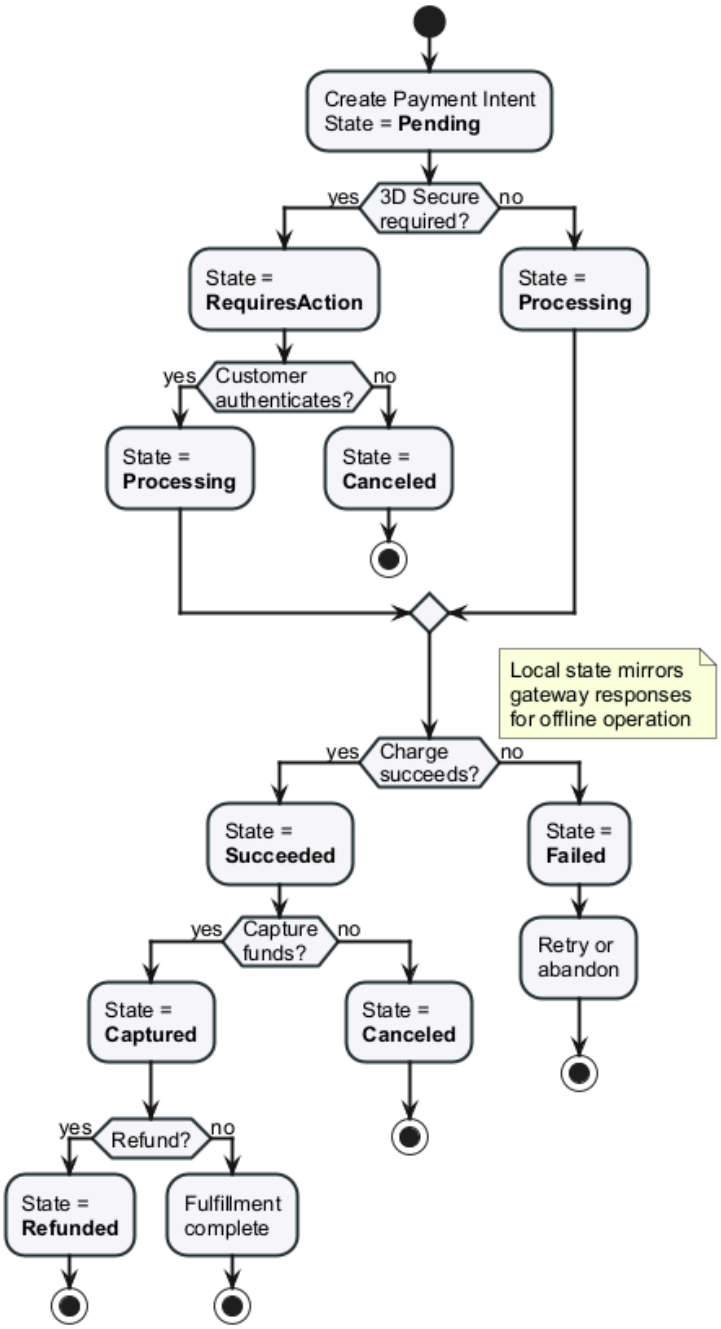

Figure 35: Payment intent lifecycle and terminal states.

2.3.3 C4 Architecture

The C4 model structures software architecture across four abstraction levels: system context, container, component, and code. This section presents the first three C4 levels, omitting code-level structures addressed in later implementation sections.

2.3.3.1 System Context

The system context positions ReSys.Shop within its operational environment, defining user roles and external dependencies (Figure 36).

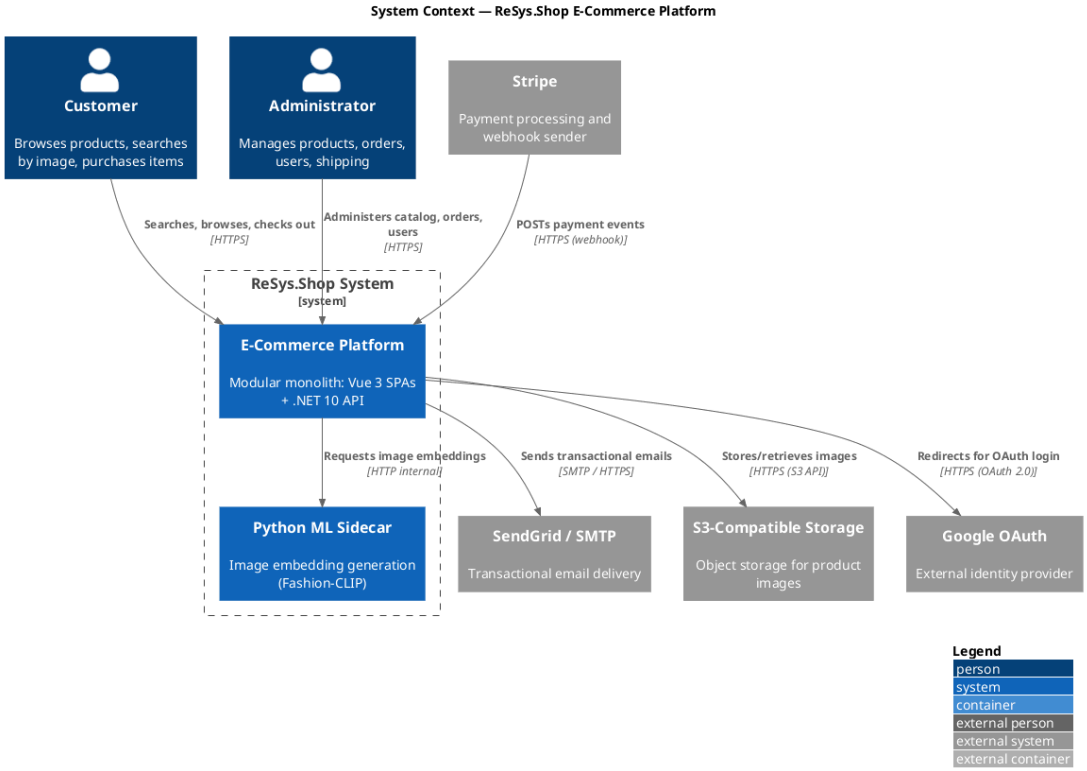

Figure 36: System Context diagram: ReSys.Shop system boundary showing user roles and external integration dependencies.

The platform interacts with two human user groups:

- **Customers:** Browse the catalog, run visual and keyword searches, manage carts, and complete checkouts.
- **Administrators:** Manage products, process orders, track inventory, and administer user accounts.

Five external integrations:

- **Stripe:** Payment intent lifecycles and webhook notifications via signature verification.
- **SendGrid:** Transactional emails (order confirmations, password resets, shipping updates).

- **S3-Compatible Storage:** Product asset persistence.
- **Google OAuth:** Customer single sign-on authentication.
- **Python ML Sidecar:** Image embedding generation within the container orchestration boundary.

2.3.3.2 Container

The container view decomposes ReSys.Shop into six standalone deployable processes and data stores (Figure 37).

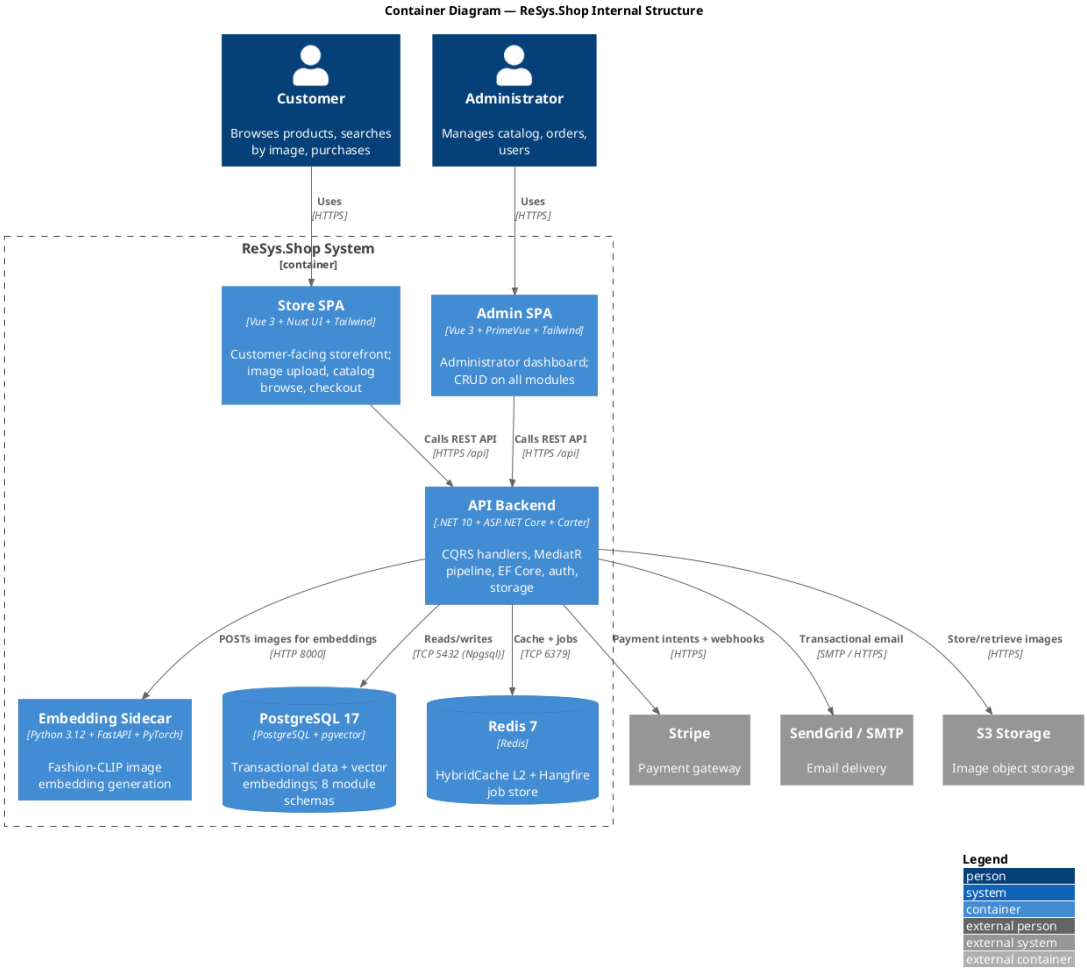

Figure 37: Container diagram showing Vue 3 SPAs, .NET 10 API backend, Python ML sidecar, PostgreSQL with pgvector, and Redis.

The deployable units:

- **Store & Admin SPAs:** Vue 3 single-page applications interacting with the backend over HTTPS REST endpoints.
- **API Backend:** .NET 10 application executing domain logic via Carter minimal APIs and MediatR CQRS pipelines.
- **Embedding Sidecar:** Python 3.12 FastAPI service loading ML models into GPU/CPU memory for on-demand embedding generation.

- **PostgreSQL 17 (with pgvector):** Relational domain schemas and high-dimensional vector embeddings.
- **Redis 7:** L2 distributed cache for `HybridCache` and persistent job store for Hangfire.

All communication routes through the API Backend; SPAs never query databases or external APIs directly.

2.3.3.3 Component

The component view details the internal structure of the API Backend container (Figure 38).

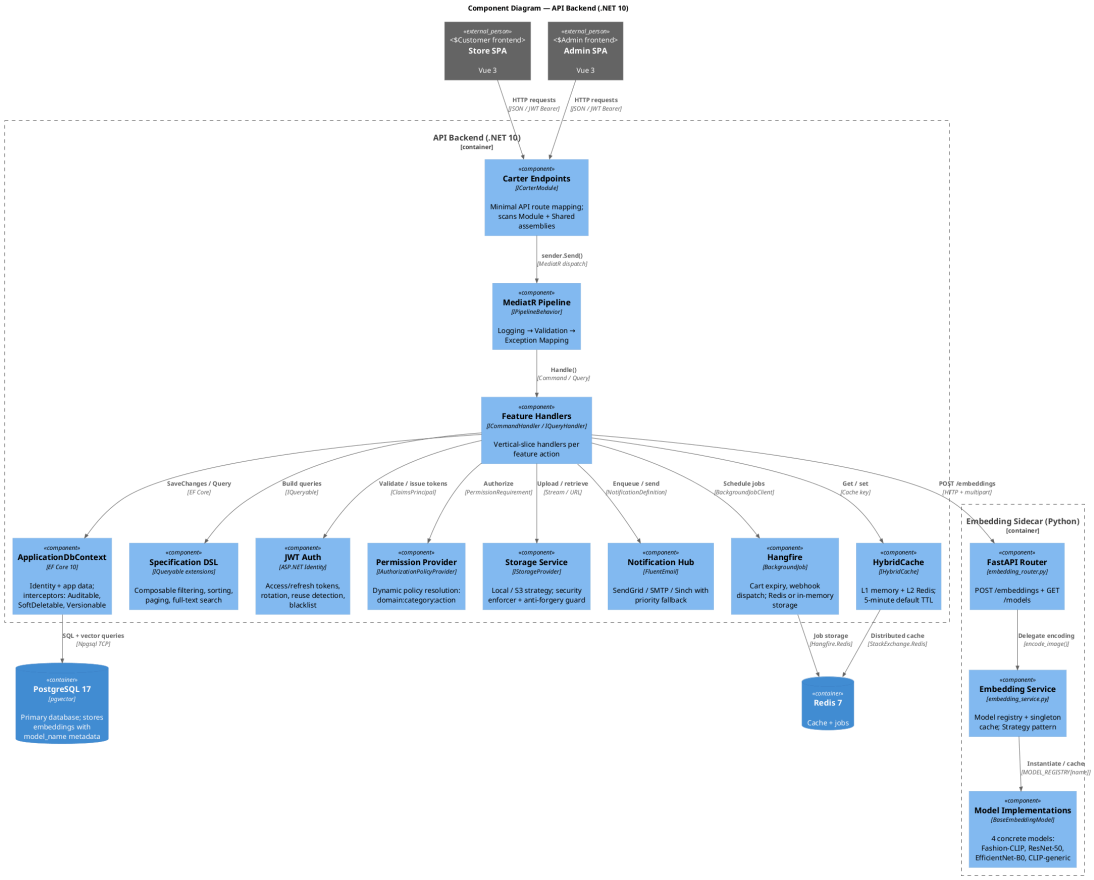

Figure 38: Component diagram detailing the API Backend architecture and the three-layer Python ML sidecar.

HTTP requests enter through Carter modules, which validate parameters and dispatch commands or queries via MediatR's `ISender` through logging, validation, and exception-handling pipeline behaviors.

Handlers delegate infrastructure tasks to eight internal components:

1. **ApplicationDbContext:** EF Core context with interceptors for auditing, soft-deletes, and concurrency.

2. **Specification DSL:** Composable `IQueryable` extensions for filtering, sorting, paging, and full-text search.
3. **Identity Provider:** ASP.NET Identity with JWT management.
4. **Dynamic Permission Provider:** Runtime resolution of `{domain}:{category}:{action}` policy claims.
5. **Storage Service:** Interchangeable local or S3-compatible file storage with upload validation.
6. **Notification Hub:** Email (SendGrid/SMTP) and SMS (Sinch) routing with fallback support.
7. **Hangfire Engine:** Background task execution (cart expiration, webhook dispatch, maintenance).
8. **HybridCache:** L1 in-memory and L2 Redis caching combined for cross-instance consistency.

The Python ML Sidecar uses a three-layer layout: a FastAPI router, a singleton model registry for lazy loading, and an interchangeable strategy interface supporting Fashion-CLIP, ResNet-50, EfficientNet-B0, and standard CLIP.

2.3.3.4 Deployment

The deployment diagram illustrates the production infrastructure topology (Figure 39).

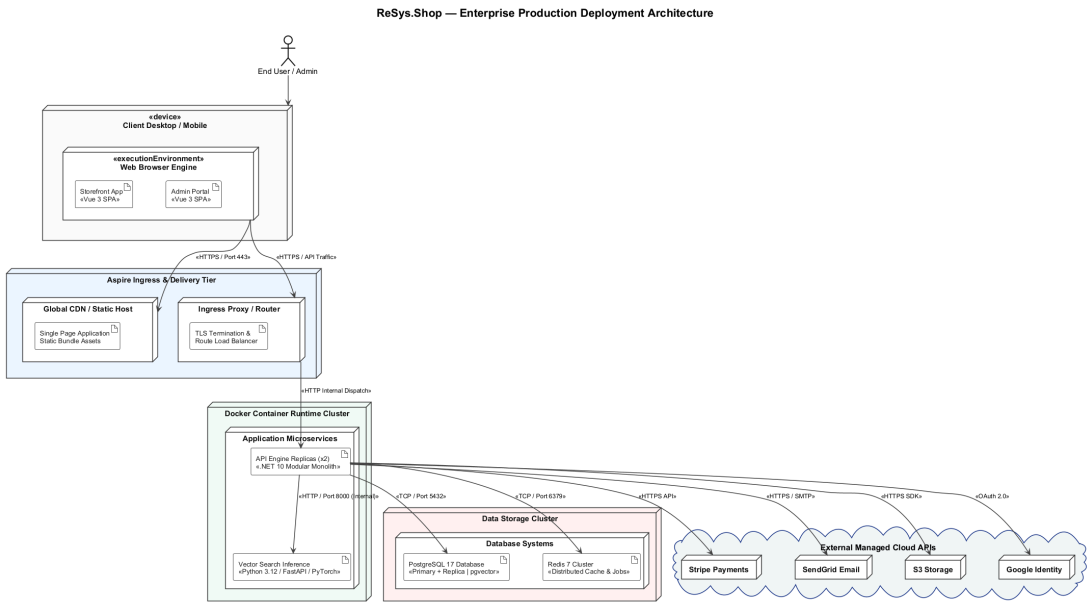

Figure 39: Deployment topology showing containerized orchestration with horizontal scaling.

All services run as Docker containers orchestrated via Docker Compose for service discovery and health monitoring. The API Backend scales horizontally across replicas sharing PostgreSQL and Redis. The stateless ML sidecar scales independently. PostgreSQL runs a primary instance for writes alongside read

replicas, with `pgvector` enabled across all nodes. Secrets and API keys are injected via environment configuration.

2.3.4 Database Design

The ReSys.Shop database consists of a single PostgreSQL 17 instance partitioned into per-context schemas. Each schema is owned by a bounded context and managed via Entity Framework Core migrations.

2.3.4.1 Schema Organisation

Each of the eight bounded contexts manages its own dedicated database schema:

- **Catalog:** `products`, `variants`, `variant_images`, `option_types`, `option_values`, `taxonomies`, and `taxons`.
- **Ordering:** `orders`, `line_items`, `adjustments`, `shipments`, and `inventory_units`.
- **Payment:** `payment_intents`, `payment_captures`, and `payment_logs`.
- **Inventory:** `stock_locations`, `stock_items`, `stock_movements`, and `stock_reservations`.
- **Identity:** `users`, `roles`, `user_roles`, `refresh_tokens`, and `external_logins` (built on ASP.NET Identity Core).
- **Profile:** `user_addresses`, `wishlists`, and `notification_preferences`.
- **Shipping:** `shipping_methods`, `shipping_rates`, and `shipping_zones`.
- **Location:** `countries` and `states` reference data.

Cross-context relationships use identifier references (UUIDs) without database-level foreign key constraints. An order references `UserId` and `VariantId` as loose attributes, maintaining logical module isolation while avoiding distributed transaction overhead.

2.3.4.2 Core Entity-Relationship Model

Figure 40 illustrates the entity-relationship model across all eight bounded contexts. Dotted lines indicate cross-context identifier references.

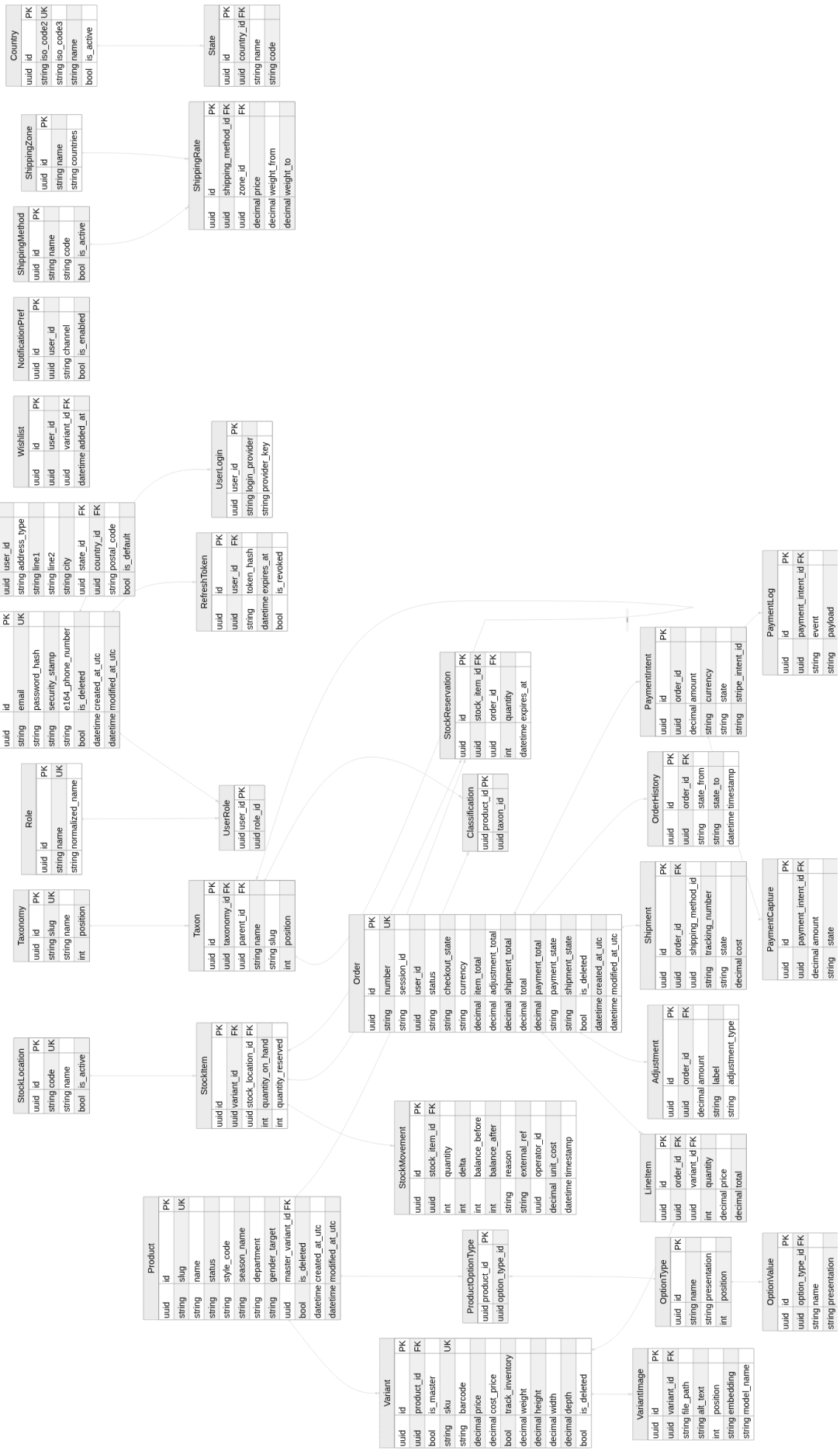

Figure 40: Core entity-relationship model across eight bounded contexts.

The **Catalog** domain centers on `Product` with one-to-many `Variant` relationships. `VariantImage` records hold image paths and an optional `vector(512)` embedding column. **Taxonomy** and **Taxon** trees manage hierarchical classification via self-referencing foreign keys.

The **Ordering** domain centers on `Order`, linking one-to-many with `LineItem`. Line items capture price snapshots at purchase time to decouple order history from catalog updates.

2.3.4.3 pgvector Integration

PostgreSQL's **pgvector** extension [13] executes vector similarity searches within the relational engine. The `variant_images` table stores feature vectors in an `embedding` column defined as `vector(512)`. The platform defaults to **HNSW** indexing [12] using cosine distance to meet the sub-second CBIR latency target (NFR-01a), with **IVFFlat** as a fallback for local environments (see Section 2.1.5 for index detail).

- **Cosine Distance:** Query operator (`<=>`) measures angular distance. Results rank by similarity score $1 - \text{cosine\_distance}$, filtered against a configurable threshold (0.70).
- **Model Isolation:** Every record includes a `model_name` string (e.g., `"Fashion-CLIP"`). Search queries filter by active model to enforce embedding alignment.

2.3.4.4 Key Design Decisions

The data layer adheres to five global design rules:

- **UUID Primary Keys:** Safe client-side key generation and no sequential enumeration leakage.
- **Soft Deletion:** `IsDeleted` flag filtered globally by EF Core, preserving referential integrity.
- **Audit Columns:** `CreatedAtUtc` and `ModifiedAtUtc` populated via EF Core save interceptors.
- **Composite Indexes:** Targeted indexes on high-frequency access paths (`(UserId, Status)`, `(SessionId, Status)` on `orders`).
- **Variable Vector Dimensions:** `pgvector` columns support per-model dimensionalities: 384 (DINOv2-S), 512 (Fashion-CLIP), 768 (DINOv2-B), 1280 (EfficientNet-B0), 2048 (ResNet-50).

2.3.4.5 Per-Context Schema Description

- **Identity Schema:** Argon2-hashed credentials, security stamps, single-use refresh tokens, and linked OAuth logins.
- **Catalog Schema:** `Product` stores fashion metadata; `Variant` manages SKU, pricing, and dimensions; `OptionType` and `OptionValue` implement an EAV pattern; `Taxon` uses a nested set model.
- **Ordering Schema:** Financial attributes use `decimal(18,2)`; `OrderHistory` provides an append-only state transition log.

- **Inventory Schema:** `StockMovement` serves as an immutable transaction ledger recording balance deltas, unit costs, and operator IDs.

2.3.5 API Design

The API exposes a RESTful interface via **Carter** modules and the **MediatR** CQRS pattern [18], registering approximately 262 endpoints across nine modules.

2.3.5.1 API Architecture

Each request follows a standard MediatR pipeline: Carter endpoint → `LoggingBehavior` → `ValidationBehavior` → `ExceptionMappingBehavior` → `Handler.Execute()` → `Result<T>.ToResult()` → HTTP Response (RFC 7807 Problem Details on error).

A representative endpoint pattern:

```
public void AddRoutes(IEndpointRouteBuilder app)
{
    app.MapPost(CatalogFeature.Admin.Products.Create.Route,
        async ([FromBody] Request r, ISender s, CancellationToken ct) =>
        {
            var result = await s.Send(new Command(r), ct);
            return result.ToResult();
        })
        .HasPermission(CatalogFeature.Admin.Products.Create.Permission);
}
```

Handlers return `Result<T>`; success maps to 200/201/204, domain errors to RFC 7807 Problem Details.

2.3.5.2 Endpoint Organisation

Endpoints follow the convention `/api/{module}/{surface}/{resource}`, where `surface` is `storefront` or `admin`. All `admin` routes enforce administrator policies via `.HasPermission()`. Eleven inter-module contract DTOs enable cross-module communication: `ReserveCartStock`, `ReleaseCartStockReservations`, `ConsumeCartStockReservations`, `CheckVariantAvailability` (Inventory); `GetCartForCheckout`, `GetCartForShipping`, `AdvanceCheckoutState` (Ordering); `GetPaymentForCheckout`, `MarkPaymentPaid` (Payment); `GetVariantDiscontinuedStatuses`, `GetVariantWeights` (Catalog).

2.3.5.3 API Endpoint Contract

Table 50 summarises the registered Carter endpoints.

Table 50: ReSys.Shop API endpoint contract: approximately 262 Carter endpoints across nine business modules.

Module	Admin Routes	Storefront Routes	N
Catalog	Products, variants, images, option types/values, taxonomies, pricing, dashboard	Product listing/search/detail, availability, similar products, CBIR search, taxonomy, taxon browsing	80
Identity	Users, roles, permissions catalogue	Login (password/Google), register, logout, session refresh, email, password reset	37
Ordering	Orders, line items, status transitions, shipping/billing address, dashboard	Cart CRUD, cart items, checkout, shipping rate, customer orders	35
Inventory	Stock locations, stock items, bulk adjust, low stock, reservations, transfers, dashboard	Variant availability, cart reserve/release	32
Profile	Profiles, addresses CRUD	Profiles, addresses, notification preferences, wishlists	27
Location	Countries, states CRUD	Countries, states browse	18
Payment	Payment methods, payments list/detail, capture/void/refund	Payment intent create/confirm, methods, setup intent, Stripe webhook	17
Shipping	Shipping methods, shipping rates CRUD	Available methods, calculate cost, rates	15
Dashboard	Aggregated metrics: sales, inventory, catalog, activity	–	1

2.3.5.4 Error Handling

All API endpoints conform to RFC 7807 Problem Details: `400` for FluentValidation failures, `401` for missing/expired JWT, `403` for insufficient permissions, `404` for entity/route resolution failure, `409` for concurrency conflicts (PostgreSQL `xmin`), and `500` via global exception middleware preventing stack trace leakage.

2.3.6 Security Design

The security framework operates across three layers: authentication, authorization, and defense-in-depth infrastructure.

2.3.6.1 Authentication and Session Management

JWT authentication is configured via `JwtSettings` with HS256 algorithm, 15-minute access token expiration, and 30-day maximum token age. Single-use refresh token rotation is enforced: exchanging an expired access token consumes the current refresh token and issues a new pair. Re-submitting a previously consumed refresh token triggers breach detection, immediately revoking all active refresh tokens for that user and forcing full re-authentication. Unauthenticated shoppers receive an HTTP-only cookie tracking an anonymous session ID; on login, the guest cart automatically merges with the user's persistent cart.

```
// JWT configuration (JwtSettings)
public sealed class JwtSettings
{
    public string Secret { get; init; }
    public string Issuer { get; init; }
    public string Audience { get; init; }
    public int AccessTokenExpirationInMinutes { get; init; } = 15;
    public int RefreshTokenExpirationInDays { get; init; }
    public string Algorithm { get; init; } = "HS256";
    // Token security
    public bool RotationEnabled { get; init; } = true;
    public bool ReuseDetectionEnabled { get; init; } = true;
    public bool SlidingExpirationEnabled { get; init; } = true;
    public int MaxTokenAgeDays { get; init; } = 30;
}
```

2.3.6.2 Dynamic Authorization

Role-Based Access Control separates administrative and storefront surfaces. Unprivileged accounts accessing `/api/*/admin/*` endpoints receive an immediate `403` prior to command dispatch. The built-in Administrator role bypasses all permission checks.

A three-layer permission architecture resolves claims at runtime:

```
// Permission resolution pipeline
// L1: IPermissionCache      - in-memory cache of resolved permissions per
user
// L2: IPermissionService    - merges role-derived + direct-grant
permissions
// L3: IPermissionStore      - EF Core persistence to Identity claims
tables

// Custom policy provider resolves permission names dynamically
public sealed class PermissionPolicyProvider :
IAuthorizationPolicyProvider
{
    public Task<AuthorizationPolicy> GetPolicyAsync(string policyName)
    {
        if (PermissionContext.IsKnown(policyName))
            return Task.FromResult(new AuthorizationPolicyBuilder()
                .AddRequirements(new PermissionRequirement(policyName))
                .Build());
        return FallbackProvider.GetPolicyAsync(policyName);
    }
}
```

Operations enforce granular resource-action claims. Ten `FeatureMetadata` files define the permission registry across modules, each mapping to the `PermissionContext` static catalogue. Permissions use the format `Domain.Category.Resource.Action`:

```
catalog.products.create
catalog.products.update
catalog.variants.delete
identity.roles.manage
ordering.orders.approve
payment.intents.capture
inventory.stock.transfer
```

A custom `IAuthorizationPolicyProvider` resolves claim strings to policies dynamically at runtime, allowing permission modifications in the database without application redeployments.

2.3.6.3 System Hardening

Rate limiting restricts authentication to 5 requests per minute, registration to 3 per hour, and payment processing to 30 per minute. Security middleware injects `Content-Security-Policy`, `Strict-Transport-Security`, `X-Frame-Options`, and `X-Content-Type-Options` headers. Visual search uploads enforce a 10 MB limit and validate magic bytes to verify valid JPEG, PNG, or WebP formats. Stripe payment webhooks validate the `Stripe-Signature` header using HMAC signature verification before executing state transitions.

The storage service includes a pre-upload virus scanning layer via ClamAV (nClam integration) and image format validation through SkiaSharp, preventing malicious payloads from reaching persistent storage.

2.4 IMPLEMENTATION

This section describes the concrete realization of the ReSys.Shop system, showing how the architecture and design decisions from Sections 2.2 and 2.3 were translated into working software. The presentation follows the system's actual structure: the technology stack that underpins development, the vertical slice pattern that organizes the .NET codebase, the persistence layer that stores relational and vector data in a single database, the machine learning sidecar that constitutes the core research contribution, and the frontend applications that deliver the user-facing experience.

- **Technology Stack.** Framework versions and containerization strategy.
- **Vertical Slice Core.** Feature co-location, the Carter–MediatR request pipeline, and the functional Result pattern.
- **Data Persistence.** Multi-schema EF Core, pgvector integration with HNSW indexing, and concurrency control.
- **ML Sidecar.** Model management, the embedding generation pipeline, and the end-to-end CBIR search flow.
- **Frontend Applications.** Dual-SPA architecture, visual search interface, and key administration workflows.

2.4.1 Technology Stack

The platform uses pinned versions across three ecosystems: .NET 10 for transactional semantics and API abstractions, Python 3.12 for deep learning (PyTorch, Hugging Face), and Vue 3 for reactive UIs. Centralized package management enforces reproducibility via `Directory.Packages.props` (NuGet), `uv.lock` (Python), and `pnpm-lock.yaml` (JavaScript).

Table 51 details the core technologies grouped by architectural role.

Table 51: Principal technologies grouped by architectural role with pinned version specifications.

Architectural Role	Technology and Version
Backend Runtime	.NET 10 / ASP.NET Core 10.0.9 (C# 13)
API Framework	Carter 10.0.0, MediatR 14.1.0, FluentValidation 12.1.1
Object Mapping	Mapster 10.0.9
ORM and Database	EF Core 10.0.9, Npgsql 10.0.2, pgvector 0.3.2
Caching Layer	HybridCache 10.6.0, StackExchange.Redis 10.0.9
Background Jobs	Hangfire 1.8.23 (Redis-backed)
Observability	OpenTelemetry 1.16.0
ML Runtime	Python 3.12, PyTorch >= 2.0.0, TorchVision >= 0.15.0
ML Framework	FastAPI >= 0.115.0, Uvicorn, Hugging Face Transformers
ONNX Runtime	onnxruntime >= 1.17.0
Storefront UI	Vue 3.5, TypeScript 6.0, Vite 8, PrimeVue 5 (Aura)
Admin UI	Vue 3.5, TypeScript 6.0, Vite 8, PrimeVue 5 (Sakai), Chart.js 4
State and HTTP	Pinia, Axios 1.x, Zod
Data Storage	PostgreSQL 17 (pgvector/pgvector:pg17-trixie)
Cache Storage	Redis 7 (Alpine)
Test Suites	xUnit v3 3.2.2, pytest >= 8.0, Vitest 4, Playwright

2.4.1.1 Service Containerization

The platform defines six containerized resources with startup dependencies: PostgreSQL and Redis initialize first, followed by the Python ML sidecar with `/health` readiness probe, then the .NET API. Vue SPAs run as Vite dev servers reverse-proxying `/api/` endpoints. Multi-stage Docker builds isolate runtime dependencies with non-root execution via `tini`.

2.4.2 Vertical Slice Architecture Core

The .NET backend implements Vertical Slice Architecture (VSA) combined with Command Query Responsibility Segregation (CQRS), organizing code by business capability rather than technical layer. Each feature co-locates its request DTOs, domain logic, validation, Carter endpoint, and response models within a single directory using `static partial class` definitions.

2.4.2.1 Feature Co-Location and Execution Mechanics

Every feature is structured across five files: `{Feature}.cs`, `{Feature}.Request.cs`, `{Feature}.Response.cs`, `{Feature}.Endpoint.cs`,

and `{Feature}.Validator.cs`. The handler pipeline (Validate → Build → Complete):

Table 52: Sequential execution flow within the CreateProduct command handler.

Step	Action
1. Validate	Checks slug uniqueness; returns `DuplicateSlug` on collision.
2. Build	Constructs `Product` via domain factory, persists to database, dispatches cross-module `AddVariant` via `ISender`.
3. Complete	Assigns master variant ID, returns `Result<Response>.Created(...)`.

Inter-module communication relies exclusively on `ISender.Send()` messages. Bounded contexts never import foreign context namespaces, enforcing isolation within a single assembly via static analysis and build policies.

```
public sealed record Command(Request Request) : ICommand<Response>;

public sealed class CommandHandler(IApplicationDbContext db, ISender
sender)
    : ICommandHandler<Command, Response>
{
    public async Task<Result<Response>> Handle(Command c,
CancellationToken ct)
    {
        if (await db.Set<Product>().AnyAsync(x => x.Slug ==
c.Request.Slug, ct))
            return ProductResult.Errors.DuplicateSlug;

        var product = c.Request.MapToDomain().Value;
        db.Set<Product>().Add(product);
        await db.SaveChangesAsync(ct);

        var v = await sender.Send(new AddVariant.Command(product.Id, vr),
ct);
        product.MasterVariantId = v.Value.Id;
        await db.SaveChangesAsync(ct);

        return Result<Response>.Created(product.MapToDetail<Response>(),
            ProductResult.Success.Created(product.Id));
    }
}
```

2.4.2.2 Request Pipeline Architecture

Each feature implements `ICarterModule` with thin endpoints that parse requests, construct MediatR commands, dispatch via `ISender`, and convert `Result<T>` to HTTP responses. Routes follow `/api/{module}/{surface}/{resource}` with Carter auto-discovery at startup.

Three MediatR pipeline behaviors wrap every command: `LoggingBehavior` (outermost, captures request metadata and duration), `ValidationBehavior` (runs FluentValidation, short-circuits on failure), and `ExceptionMappingBehavior` (innermost, wraps unhandled exceptions).

2.4.2.3 Functional Result Pattern

The application uses `Result<T>` (`readonly record struct`) instead of exception-driven control flow. Domain methods return explicitly typed results; `Error` structs carry machine-readable codes, human-readable messages, and optional metadata. The `ToResult()` extension maps domain results to HTTP status codes: 200 OK or 201 Created for success, 400 for validation failures, 404 for missing entities, and 409 for state conflicts.

2.4.3 Data Persistence Architecture

The persistence layer maps eight bounded contexts to dedicated PostgreSQL schemas, co-locating vector embeddings with relational product data in a single PostgreSQL 17 instance.

2.4.3.1 Schema Organisation and Vector Storage

Each bounded context owns an isolated schema. Table 53 outlines the distribution:

Table 53: Schema-per-context mapping. Cross-schema references use UUID attributes without database-level foreign key constraints.

Schema	Principal Tables and Aggregate Entities
`catalog`	`products`, `variants`, `variant_images`, `product_image_embeddings`, `taxonomies`, `taxons`
`ordering`	`orders`, `line_items`, `order_adjustments`, `shipments`
`payment`	`payment_intents`, `payment_captures`, `payment_methods`
`inventory`	`stock_locations`, `stock_items`, `stock_movements`, `stock_reservations`
`identity`	`users`, `roles`, `user_roles`, `refresh_tokens`
`profile`	`user_profiles`, `addresses`, `wishlists`
`shipping`	`shipping_methods`, `shipping_rates`, `shipping_zones`
`location`	`countries`, `states`

Cross-schema references use unconstrained UUIDs, enforcing module independence. Product embeddings reside in `catalog.product_image_embeddings` alongside product metadata, participating in the same ACID transactions.

2.4.3.2 pgvector Indexing Strategy

Embeddings are stored in a `vector(512)` column with model-aware discriminators (e.g., `Fashion-CLIP`). The production HNSW index uses cosine distance for sub-second CBIR queries (see Section 2.3.4 for index detail and Section 2.1.5 for ANN algorithm comparison).

2.4.3.3 Model-Aware Vector Search

Each embedding carries a `model_name` discriminator. Cosine similarity search proceeds through four stages: candidate retrieval via `<=>` filtered by `model_name`, distance-to-similarity transformation, threshold filtering (≥ 0.70), and deduplication by parent product.

2.4.3.4 Concurrency and Data Integrity

Three complementary strategies maintain integrity under concurrent throughput:

Table 54: Concurrency control mechanisms across system domains.

Pattern	Implementation	Applied To
Optimistic Locking	PostgreSQL `xmin` column via EF Core `IsRowVersion()`	General entity updates (HTTP 409 on stale write)
Pessimistic Locking	`SELECT FOR UPDATE` row locks	Stock allocation during checkout (serializes low-quantity purchases)
Repeatable Read	`IsolationLevel.RepeatableRead` transaction scope	Sequential order code generation (prevents phantom reads)

Audit tracking uses EF Core interceptors: `AuditInterceptor` auto-populates `CreatedAtUtc`, `CreatedBy`, `UpdatedAtUtc`, and `UpdatedBy` during `SaveChangesAsync`. Stock modifications write append-only entries to `inventory.stock_movements`.

2.4.4 ML Sidecar and CBIR Search

The Python ML sidecar generates vector embeddings from product images over HTTP, managing multiple pre-trained models through a strategy pattern switchable at runtime via `EMBEDDING_MODEL`.

2.4.4.1 Model Management

Six models span four architectures, selected from a decorator-based registry on first inference:

Table 55: Registered embedding models with output dimensionality and architectural family.

Model ID	Dim	Architecture	Source
fashion_clip	512	ViT-B/32 + CLIP	patrickjohncyh/fashion-clip (HuggingFace)
clip_vit_b16	512	ViT-B/16 + CLIP	OpenAI CLIP (torchvision)
openclip-vit-b-32	512	ViT-B/32 + CLIP	OpenCLIP (HuggingFace)
efficientnet_b0	1280	EfficientNet-B0	torchvision ImageNet1K_V1

Model ID	Dim	Architecture	Source
resnet50	2048	ResNet-50	torchvision ResNet50_Weights.DEFAULT
dinov2_vits14	384	ViT-S/14	facebookresearch/dinov2 (torch.hub)

All models conform to `BaseEmbedder` via the Template Method pattern. The base class orchestrates loading, forwarding, and L2-normalisation; subclasses provide only the forward pass. Models load lazily on first request within `torch.no_grad()`. The `device` property resolves CUDA, MPS, or CPU at runtime.

```
class BaseEmbedder(ABC):
    @abstractmethod
    def _forward(self, image: torch.Tensor) -> torch.Tensor:
        """Model-specific forward pass."""

    async def extract(self, image_input) -> list[float]:
        image = self._load_image(image_input)
        features = self._forward(image)
        return self._normalize(features)
```

`_load_image` handles URLs, paths, raw bytes, and PIL Images, converting to RGB tensors. `_normalize` applies L2-normalisation and handles CLIP `image_embeds`, ViT `pooler_output`, and CNN feature maps via global average pooling.

2.4.4.2 Embedding Generation Pipeline

Six pipeline stages:

Table 56: Embedding generation pipeline stages.

Stage	Component	Description
1	Authentication	Validates `X-API-Key` header; rejects with 401.
2	Model Selection	Retrieves active model from registry; initialises from disk if unallocated.
3	Preprocessing	Resizes to 224 × 224 px, converts to tensor, applies ImageNet normalisation.
4	Forward Pass	Propagates through convolution or self-attention within `torch.no_grad()`.
5	Pooling	Global average pooling to fixed-length vector (512-dim for CLIP, up to 2048 for ResNet-50).
6	Serialization	L2-normalisation, JSON packaging with model metadata and inference latency.

API endpoints:

Table 57: ML sidecar API endpoints. All inference endpoints require `X-API-Key` authentication.

Method	Path	Purpose
POST	`/embeddings/bytes`	Multipart image upload with optional `model` query parameter.
POST	`/embeddings`	Image URL for batch embedding during catalogue indexing.
GET	`/health`	Readiness probe: validates CUDA/MPS, active model, and synthetic forward pass.

Response shape:

```
{
  "value": {
    "vector": [0.023, -0.154, ..., 0.042],
    "model": "fashion_clip",
    "dimensions": 512,
    "inference_ms": 92.0
  },
  "isSuccess": true,
  "statusCode": 200
}
```

2.4.4.3 CBIR Search Flow

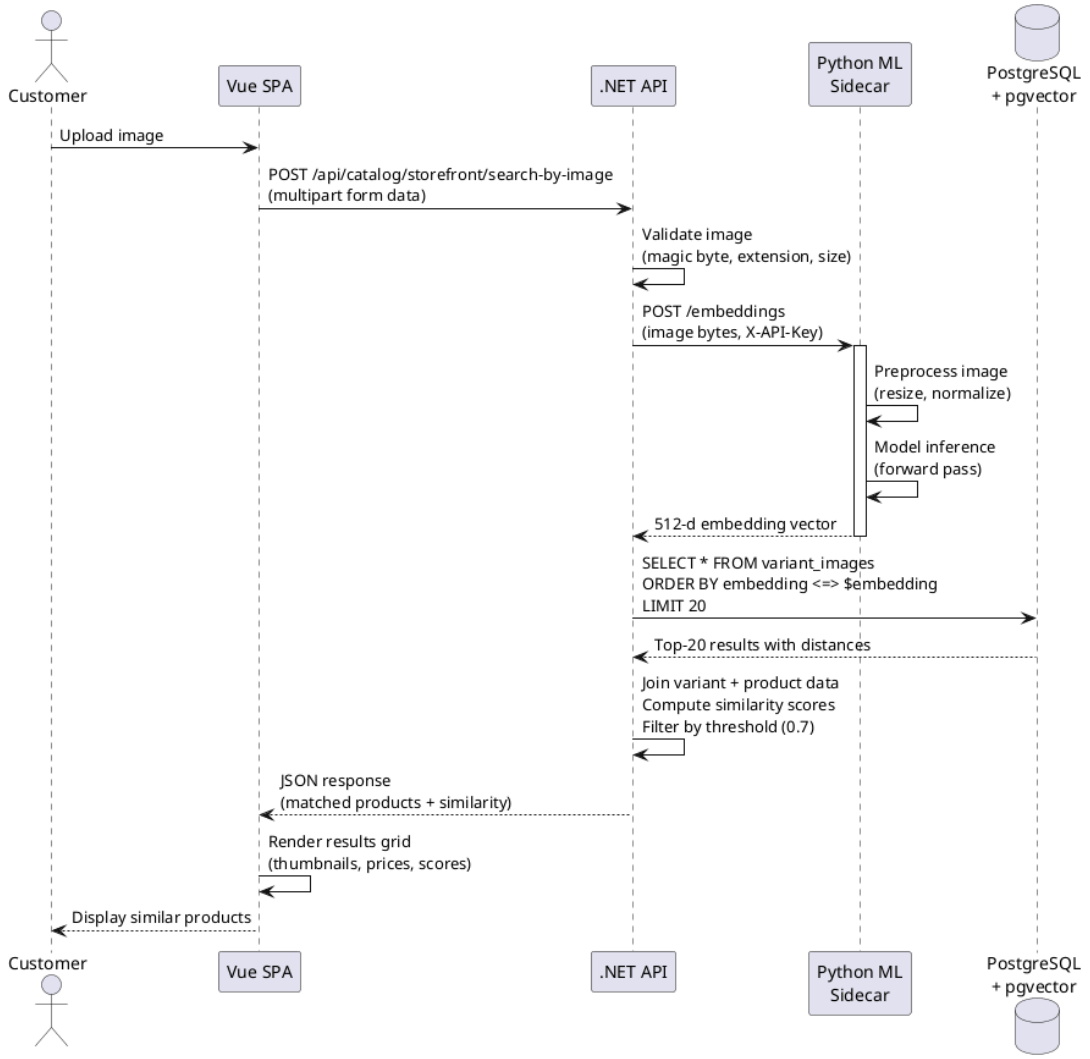

Figure 41: CBIR search sequence: end-to-end flow spanning customer upload, embedding extraction, pgvector search, and ranked results rendering.

Six stages across four architectural layers:

1. **Client Validation (Vue 3).** Validates format (JPEG, PNG, WebP) and size ($\leq$ 10 MB). Dispatches multipart form to `POST /api/catalog/storefront/search-by-image`.
2. **Server Validation (.NET API).** Verifies magic bytes, reapplies payload ceiling.
3. **Vector Extraction (ML Sidecar).** Forwards image bytes to `/embeddings`; sidecar executes preprocessing and inference.
4. **Vector Search (pgvector).** Queries via cosine distance `<=>` filtered by `model_name`; HNSW enables sub-10 ms lookup.
5. **Post-Processing.** Converts distance to similarity (1 − distance), filters below 0.7, deduplicates by product.

6. **UI Rendering (Vue 3).** Renders product grid with thumbnails, prices, and similarity scores within the sub-second budget.

The pluggable architecture supports automated benchmarking (update `EMBEDDING_MODEL`, restart, sequential evaluation of all models without code changes) and production A/B testing via dual sidecar instances with distinct models.

2.4.5 Frontend Applications

The user-facing components consist of two Vue 3 single-page applications: a customer storefront (PrimeVue Aura theme) and an administration dashboard (PrimeVue Sakai theme with Chart.js 4). This section presents the implemented interfaces organized by the 26 use cases defined in Section 2.2.2.

2.4.5.1 Frontend Architecture

Both applications share Vue 3.5, TypeScript 6.0, Vite 8, Pinia, and Axios. All backend communication follows a typed repository pattern mirroring the .NET `Result<T>` convention with `isSuccess`, `statusCode`, `data`, and `errors[]` fields. The `BaseRepository` wraps Axios with typed CRUD:

```
export interface Result<T> {
  isSuccess: boolean
  isFailure: boolean
  statusCode: number
  data?: T
  errors?: Array<{ code: string; description: string; field?: string }>
}
```

For visual search, the repository extends the base with a multipart upload method: `async searchByImage(file: File): Promise<Result<Product[]>>` dispatching `POST /api/storefront/search-by-image` with `Content-Type: multipart/form-data`.

2.4.5.2 Storefront Interfaces

The customer storefront implements eight use cases covering product discovery, purchasing, and account management.

2.4.5.2.1 Visual Search: UC-STR-SRC

The visual search interface implements a four-state UI model:

Table 58: Visual search UI state model.

State	Display	Trigger
Empty	Upload prompt with dashed-border drop zone, cloud-upload icon, and format guidance text	Page load or after clearing previous search

State	Display	Trigger
Upload	Selected image preview with a "Search Similar Products" button below	User drops or selects an image file
Loading	Skeleton grid of animated placeholder cards (4 columns desktop, 2 mobile)	Search API request in flight
Results	Product card grid with thumbnails, names, prices, and colour-coded similarity badges	API returns non-empty results array

Two input methods are supported: drag-and-drop onto the drop zone and file browser selection. Client-side validation rejects non-image MIME types and files exceeding 10 MB:

```
{
  "results": [{
    "productId": "a1b2c3d4-...",
    "title": "Floral Summer Midi Dress",
    "price": 750000, "currency": "VND",
    "thumbnailUrl": "/images/products/a1b2c3d4_thumb.webp",
    "similarityScore": 0.9328
  }],
  "searchDurationMs": 287,
  "model": "Fashion-CLIP"
}
```

Each product card renders a thumbnail, name, formatted price, and colour-coded similarity badge (green for $\geq 90\%$, amber for $\geq 80\%$, grey below). The query image persists in a sidebar panel while scrolling results.

The five visual search states are illustrated below.

2.4.5.2.2 Catalogue Browsing: UC-STR-BRW

The catalogue page presents products in a paginated, faceted grid with left-sidebar category tree navigation built from hierarchical taxonomy data. Selecting a category filters the grid; a keyword search bar sends full-text queries. Product cards show thumbnails, names, prices, and a hover overlay with quick-add-to-cart (see screenshot below).

The product detail page displays complete product information with a variant image gallery, size and colour pickers with real-time stock indicators, current price with strikethrough original price, quantity selector, and Add to Cart button. Expandable sections provide description, material, care instructions, and size guide. A Similar Products carousel appears at bottom (see screenshot below).

2.4.5.2.3 Shopping Cart: UC-STR-CRT

The cart page lists line items with thumbnail, title, variant details, unit price, quantity controls, per-line subtotal, and remove button. A sticky summary panel shows item count, subtotal, and Proceed to Checkout button. Cart data persists via backend API and synchronises across tabs through Pinia state. Guest carts merge into authenticated accounts on login (see screenshot below).

2.4.5.2.4 Checkout: UC-STR-CHK

Checkout progresses through the five-state pipeline (Address, Delivery, Payment, Confirm, Complete) with a visual progress indicator. Each step renders as a single-page form; each transition is validated server-side.

- **Address:** Saved addresses with radio-button selection and inline "Add new address" form (screenshot below).
- **Delivery:** Available shipping methods with carrier names, delivery estimates, and calculated rates (screenshot below).
- **Payment:** Payment methods with provider icons, selected method confirmed via `POST /api/storefront/payment/create-intent` (screenshot below).
- **Confirm:** Read-only order summary with line items, addresses, method, and totals; "Place Order" button finalises (screenshot below).
- **Complete:** Order confirmation with generated order number, summary, and "Continue Shopping" link (screenshot below).

2.4.5.2.5 Order History: UC-STR-OHI

Past orders display in vertically stacked cards with order number, date, colour-coded status badge, item count, and total. Expanding a card reveals line items with thumbnails, quantities, and prices. Active orders show a Cancel button when cancellable. A detail page displays the full state transition timeline (see screenshots below).

2.4.5.2.6 Authentication: UC-STR-AUT, UC-STR-SES

Three authentication pathways: email/password login, Google OAuth 2.0, and guest sessions. Registration includes email, password with strength indicator, and full name. Password reset uses a two-step email-token flow. The session page lists active sessions with device, IP, and last-activity; "Logout All Devices" terminates all sessions (see screenshots below).

2.4.5.2.7 Payment Processing: UC-STR-PAY

During checkout, the payment step presents methods from `GET /api/storefront/payment/methods`. Selecting a method triggers payment intent creation. For Stripe, the Stripe Elements embedded UI collects card details within an iframe, isolating sensitive data from the storefront's JavaScript context (see screenshots below).

2.4.5.2.8 Profile Management: UC-STR-PRF

Three management areas: address book with type labels and default-address toggle, named wishlists with product thumbnails and privacy settings, and notification preferences with per-channel toggles for order updates, promotions, and stock alerts (see screenshots below).

2.4.5.3 Administration Interfaces

The administration dashboard implements fifteen administrative use cases using PrimeVue data-table components with server-side pagination, sorting, filtering, and form dialogs with inline validation.

2.4.5.3.1 Product Management: UC-ADM-PROD, UC-ADM-VAR, UC-ADM-IMG, UC-ADM-TAX, UC-ADM-OPT

The product management module is the most data-intensive admin area, organized into five interlinked management surfaces.

Product list (UC-ADM-PROD). Paginated data table with thumbnail, name, SKU, category, status badge (Draft/Active/Archived), and price. Toolbar: keyword search, category/status filters, New Product button. Bulk actions support status and category changes (see screenshot below).

Product form (UC-ADM-PROD). Multi-section page: Basic Info (name, auto-generated slug, description, status), Fashion Metadata (style code, season, material, department, gender), SEO (meta title, description). Inline validation below each field (see screenshot below).

Variant management (UC-ADM-VAR). Embedded table: SKU, barcode, master-variant radio, option combination, inventory toggle, price, dimensions. Add Variant modal with option-value selectors, SKU auto-generation, and time-bound pricing (see screenshots below).

Image and embedding management (UC-ADM-IMG). Sortable image grid with drag handles, thumbnails, alt-text input, and delete action. Each image shows embedding status: green checkmark with model name for processed, yellow spinner for pending, red exclamation with Regenerate button for failed. “Regenerate All Embeddings” triggers re-embedding with active model (see screenshot below).

Taxonomy and option types (UC-ADM-TAX, UC-ADM-OPT). Tree editor with left panel showing expandable taxonomy, drag-and-drop reordering, context-menu actions. Right panel shows selected taxon details. Option types table lists Size/Colour/Material with ordered values and drag handles (see screenshots below).

2.4.5.3.2 Order Management: UC-ADM-ORD, UC-ADM-ORD-ITEMS

Filterable, paginated order grid with order number, customer name (linked), checkout state (colour-coded badge), payment status, shipment status, total, creation date. Filters: status multi-select, date range, payment method, keyword search (see screenshots below). Order detail page shows state transition timeline, line items table with thumbnails and variant details, address blocks, payment section with transaction ID and capture/refund/void buttons, shipment tracking. State transition buttons appear conditionally per checkout state.

2.4.5.3.3 Payment Management: UC-ADM-PAY, UC-ADM-PAY-METHOD

Payment detail panel shows payment intent ID, gateway provider, transaction ID, amount, currency, state badge with transition timeline, and capture/refund/void action bar (conditionally enabled). Payment log records gateway interactions; webhook event log records Stripe-triggered changes (see screenshots below). Payment method management table lists configured gateways with active toggle and supported currencies.

2.4.5.3.4 Inventory Management: UC-ADM-STK, UC-ADM-LOC

Stock list. Table grouped by product showing each variant with SKU, size/colour, on-hand, reserved, available (computed), and low-stock indicator (red badge when below threshold). Filterable by location, category, and low-stock-only (see screenshot below).

Stock movements. Append-only audit log: timestamp, product/variant, movement type (Receiving, Selling, Returning, Transferring), quantity delta, before/after quantities, reason, operator. Paginated and filterable (see screenshot below).

Stock locations. Management table with location name, address, active toggle, item count. Restock form: product/variant selection, quantity, unit cost, reason. Transfer form: source/destination locations, variant, quantity; lifecycle progresses Created to In-Transit to Received (see screenshots below).

2.4.5.3.5 User and Role Administration: UC-ADM-USR, UC-ADM-ROL

User management. Paginated table: avatar, name, email, registration date, enabled toggle, roles (compact badges), last login. Filters: status, role, keyword search. Create/edit form: full name, email, password (create only), enabled toggle, role multi-select (see screenshots below).

Role management. Role table: name, description, user count, creation date. Expanding a role shows permission assignments in an expandable tree grouped by domain, each with `domain:category:action` checkbox toggles (see screenshots below).

2.4.5.3.6 Shipping Configuration: UC-ADM-SHP

Shipping methods table: carrier name, delivery estimate, active toggle, rates count. Each method has a rates table per geographic zone and weight/value tier. Add-rate dialog: zone dropdown, weight range, value range, rate amount (see screenshots below).

2.4.5.3.7 Reference Data: UC-ADM-REF

Country table with ISO codes and active toggles. Selecting a country displays its states with ISO 3166-2 codes in a linked panel (see screenshot below).

2.4.5.3.8 System Processes: UC-SYS-EMB, UC-SYS-MNT

Background automation is monitored through the Hangfire dashboard accessible at `/hangfire`. The overview displays total succeeded/failed jobs, recurring schedules, and queue depths. Recurring jobs: cart expiry (every 20 minutes, releases inventory for carts inactive 7 days), embedding retries (exponential backoff: 1-2-4-8 minutes, max 3 retries, permanent failures flagged), index maintenance (nightly: analyses HNSW state, rebuilds when thresholds exceeded), reservation expiry (every 15 minutes: releases holds exceeding timeout), payment webhook processing (triggered by Stripe events: validates HMAC, checks idempotency, updates state). The monitoring interface is illustrated below.

3 TESTING AND EVALUATION

This chapter verifies the platform through functional software testing (Sections 3.1–3.3) and systematic benchmark evaluation of the embedding models (Section 3.4 onward).

- **Goal of Testing.** Objectives for verifying the .NET backend and Python ML sidecar satisfy system requirements.
- **Scenario of Testing.** Testing environment and hardware specifications.
- **Result of Testing.** Functional test cases across visual search, ML embedding, cart/checkout, and admin management.
- **Benchmark Protocol.** Dataset, models, metrics, cross-validation methodology, hardware environment.
- **Retrieval Performance.** Aggregate accuracy metrics (mAP, P@K, R@K) and RQ1 answer.
- **Efficiency Metrics.** Latency, throughput, storage, RAM trade-offs and RQ2 answer.
- **Synthesis and Deployment.** Accuracy-efficiency comparison, deployment recommendations, limitations, and RQ3 answer.

3.1 GOAL OF TESTING

The goal of testing is to verify that the platform functions correctly against its system requirements, with emphasis on the core research components: visual search (CBIR), the ML embedding pipeline, and multi-model benchmarking.

Table 59: Testing objectives focused on core research features.

Testing Type	Objectives for Core Research Features
Functional	• Visual search returns ranked product results with similarity scores. • ML sidecar generates embeddings of correct dimensionality per model. • Shopping cart supports add, update, remove, and guest-to-user merge. • Checkout enforces forward-only state transitions. • Admin CRUD for products, variants, images, and taxonomies.
API Integration	• .NET backend dispatches image bytes to Python sidecar and receives vectors. • pgvector queries return results filtered by model_name. • Error responses conform to RFC 7807 Problem Details. • Sidecar health endpoint reports correct status for container orchestration.
Database	• Product, variant, and embedding records are persisted atomically. • Inventory reservations prevent overselling under concurrent checkout. • Soft-deletion preserves referential integrity.

Testing Type	Objectives for Core Research Features
Security	• JWT tokens expire after 15 minutes. • Endpoints reject requests with insufficient permission claims. • Upload validation rejects mismatched magic bytes.

3.2 SCENARIO OF TESTING

The testing scenarios cover the four functional areas identified in Section 3.1: visual search (7 scenarios), ML embedding pipeline (6 scenarios), shopping cart and checkout (8 scenarios), and admin product management (7 scenarios). Each scenario is validated with step-by-step test cases defined in Section 3.3.

3.2.1 Testing Environment

- **Hardware.** Intel Core i7-1165G7 (4 cores, 2.80 GHz), 16 GB DDR4 RAM, 512 GB NVMe SSD.
- **Database.** PostgreSQL 17 with pgvector 0.7.0, provisioned via Testcontainers for integration tests.
- **Cache.** Redis 7 (Alpine) for HybridCache L2 storage and Hangfire job persistence.
- **ML Sidecar.** Python 3.12 FastAPI service with PyTorch 2.13.0, CPU-only inference.
- **Web Browsers.** Google Chrome 131, Firefox 135.

3.3 RESULT OF TESTING

3.3.1 Visual Search (CBIR) – UC-STR-SRC

Table 60: Visual search test cases: normal flow, edge cases, and fault recovery.

No	Description	Step	Expected Result	Result
1	Upload query image	Drop JPEG on drop zone; click “Search Similar Products”.	Grid with thumbnails and colour badges (green >= 90%, amber >= 80%).	Pass
2	Loading state	Upload image; observe during API request.	Skeleton grid of animated placeholder cards displayed.	Pass
3	No results	Upload image with no visually similar products.	“No similar products found” with “Try Again” button.	Pass
4	Invalid file type	Attempt to upload a PDF on the search page.	Rejected client-side with unsupported format message.	Pass

No	Description	Step	Expected Result	Result
5	File exceeds 10 MB	Attempt to upload an image over 10 MB.	Rejected with size-limit warning before network request.	Pass
6	Sidecar unavailable	Stop ML sidecar; upload image and search.	`HTTP 503` catalogue and cart endpoints remain functional.	Pass
7	Sidecar recovery	Restart sidecar; wait for health check; search.	Health probe OK; search returns correct results.	Pass

3.3.2 ML Embedding Pipeline

Table 61: ML embedding pipeline test cases across four model architectures.

No	Description	Step	Expected Result	Result
8	Fashion-CLIP	`POST /embeddings/bytes` with fashion image.	512-dim vector; model "fashion_clip"; inference_ms present.	Pass
9	EfficientNet-B0	Set model to EfficientNet-B0; POST request.	1280-dim vector; model "efficientnet_b0".	Pass
10	ResNet-50	Set model to ResNet-50; POST request.	2048-dim vector; model "resnet50".	Pass
11	CLIP-generic	Set model to CLIP ViT-B/16; POST request.	512-dim vector; model "clip_vit_b16".	Pass
12	Model-name filtering	Generate with two models; search and verify.	Results filtered by model name; match active model only.	Pass
13	Invalid API key	`POST /embeddings/bytes` without API key.	`HTTP 401 Unauthorized`.	Pass

3.3.3 Shopping Cart and Checkout – UC-STR-CRT, UC-STR-CHK

Table 62: Shopping cart and checkout test cases.

No	Description	Step	Expected Result	Result
14	Add item to cart	Select variant (size, colour); click "Add to Cart".	Cart count updates in header; toast displayed.	Pass

No	Description	Step	Expected Result	Result
15	Update quantity	Open cart; change quantity from 1 to 3.	Line and cart subtotals recalculate correctly.	Pass
16	Remove item	Open cart; click remove on a line item.	Item removed; subtotal updated; items displayed.	Pass
17	Guest cart merge	Add items as guest; register and log in.	Guest cart merges; cookie invalidated; no items lost.	Pass
18	Exceed stock	Add item; increase quantity beyond available stock.	Rejected with max-available notification; capped.	Pass
19	Checkout pipeline	Complete Address, Delivery, Payment, Confirm, Complete.	Progress bar advances; order generated; inventory reserved.	Pass
20	Empty cart checkout	Navigate to checkout with empty cart.	Redirected to cart; “Your cart is empty” displayed.	Pass
21	Cancel order	Open order in pre-completion state; cancel.	Order Cancelled; inventory released; payment voided.	Pass

3.3.4 Admin Product Management – UC-ADM-PROD, UC-ADM-VAR, UC-ADM-IMG

Table 63: Admin product, variant, image, and taxonomy management test cases.

No	Description	Step	Expected Result	Result
22	Create product	Fill name, slug, fashion metadata; click “Save”.	Product created with generated slug; notification.	Pass
23	Duplicate slug	Create product with existing slug.	Error: slug exists; product not created.	Pass
24	Add variant	Select Size M and Colour Navy; enter SKU, price.	Variant added with option combination in table.	Pass
25	Upload images	Upload 3 images; set display order; check status.	Green checkmark with model name per image; order kept.	Pass
26	Regenerate embeddings	Switch model; click “Regenerate All Embeddings”.	Status reset to pending; then green with new model.	Pass

No	Description	Step	Expected Result	Result
27	Archive product	Change status to Archived; verify storefront.	Hidden from storefront; admin shows Archived badge.	Pass
28	Taxonomy management	Create taxon under "Dresses"; drag to reorder.	New taxon with auto-slug; tree reordered; count updates.	Pass

3.3.5 Summary

All 28 test cases passed across four functional areas: visual search (7), ML embedding pipeline (6), cart and checkout (8), and admin product management (7). Error states, edge cases (empty carts, duplicate slugs, stock exhaustion), and recovery scenarios (sidecar restart) were handled correctly.

3.4 BENCHMARK PROTOCOL AND EXPERIMENTAL SETUP

The evaluation uses the Fashion Product Images Dataset: 5,000 catalogue images across five categories (Apparel 2,500, Accessories 1,250, Footwear 750, Personal Care 350, Sporting Goods 150). Each image carries a category label as ground truth for binary relevance. All images are preprocessed to 224×224 pixels with ImageNet normalisation (mean 0.485/0.456/0.406, std 0.229/0.224/0.225).

3.4.1 Models Evaluated

Four representative models spanning four architectural families were selected from the eleven supported by the benchmark framework:

Table 64: Four evaluated models representing CNN, ViT, CLIP, and domain-tuned architectures.

Architecture	Evaluated Models (dimension)
Convolutional Neural Network	ResNet-50 (2,048-dim), EfficientNet-B0 (1,280-dim)
Vision Transformer	DINOv2 ViT-S/14 (384-dim) – framework-supported
CLIP-based	CLIP ViT-B/16 (512-dim) – generic wrapper
Fashion-specific	Fashion-CLIP (512-dim), fine-tuned on 700,000+ fashion images

3.4.2 Evaluation Metrics

Five accuracy and five efficiency metrics were measured per model.

Table 65: Evaluation metrics and definitions.

Metric	Definition
mAP	Mean average precision over top-20 results; primary accuracy metric.
P@K	Fraction of top-K results sharing the query's category.
R@K	Fraction of all relevant catalogue items appearing in top-K results.
Inference Time	Mean milliseconds per image embedding (preprocessing + forward pass).
Throughput	Images embedded per second under sustained load.
Load Time	One-time cost of loading model weights from disk into memory.
Storage	Disk space of the embedding index for the 5,000-image catalogue.
RAM	Peak main memory consumption. Reported as dashes: psutil measurement proved unreliable on this Linux kernel, producing artefacts. Actual cost ranges from 100 MB (EfficientNet-B0) to >600 MB (CLIP-based).

3.4.3 Methodology

The protocol used 3-fold stratified cross-validation preserving category distribution. Accuracy metrics (mAP, P@K, R@K) used exact cosine search over all gallery embeddings to isolate model quality from index effects. The pgvector production benchmark used IVFFlat with 100 lists (sub-10 ms query latency at 65–72% recall@10, near-instant build). HNSW is designated for larger catalogue scales.

3.4.4 Hardware Environment

Table 66: Hardware environment. All benchmarks executed on CPU without GPU acceleration.

Component	Specification
CPU	Intel Core i7-1165G7 (4 cores / 8 threads, 2.80 GHz)
RAM	16 GB DDR4
Database	PostgreSQL 16, pgvector 0.7.0

3.5 RETRIEVAL PERFORMANCE AND ACCURACY

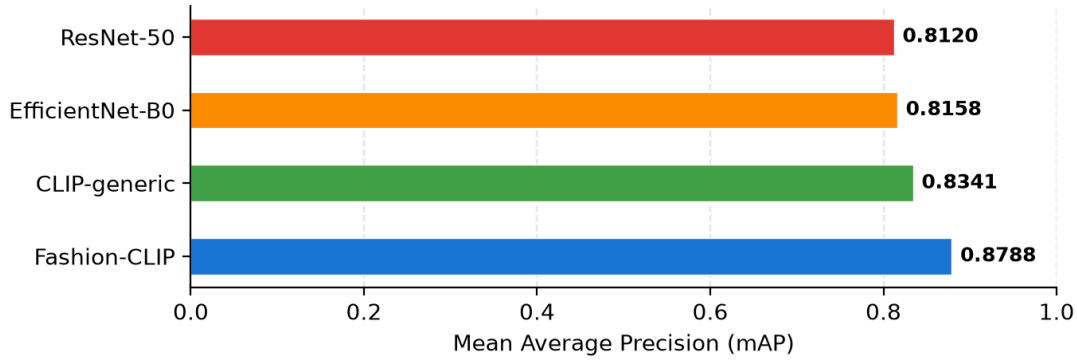

Figure 42: mAP comparison across four evaluated models. Fashion-CLIP leads at 0.8788, followed by CLIP-generic (0.8341), EfficientNet-B0 (0.8158), and ResNet-50 (0.8120).

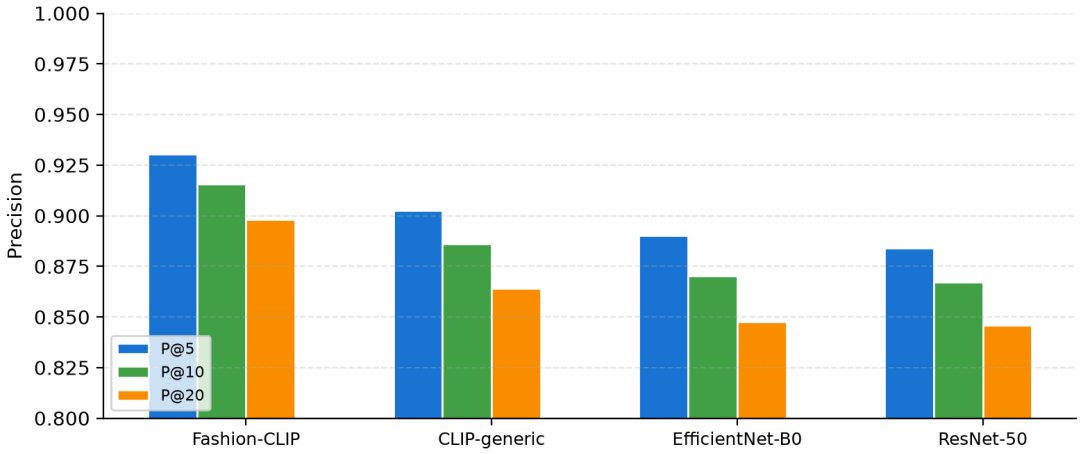

Figure 43: Precision at K (K = 5, 10, 20) across four evaluated models. Fashion-CLIP maintains the highest precision at every retrieval depth.

Table 67: Aggregate Retrieval Metrics, 3-Fold Cross-Validation

Model	**mAP (mean ± SD)**	**P@5**	**P@10**	**P@20**	**R@5**	**R@10**	**R@20**
Fashion-CLIP	**0.8788 ± 0.0022**	**0.9304**	**0.9155**	**0.8982**	**0.0350**	**0.0646**	**0.1155**
CLIP-generic	0.8341 ± 0.0043	0.9025	0.8862	0.8640	0.0322	0.0597	0.1052

Model	mAP (mean ± SD)	P@5	P@10	P@20	R@5	R@10	R@20
EfficientNet-B0	0.8158 ± 0.0007	0.8901	0.8703	0.8477	0.0316	0.0571	0.1001
ResNet-50	0.8120 ± 0.0052	0.8841	0.8671	0.8458	0.0306	0.0551	0.0978

Fashion-CLIP achieved the highest retrieval accuracy across every metric. Its mAP of 0.8788 is 5.4% above CLIP-generic (0.8341), 7.7% above EfficientNet-B0 (0.8158), and 8.2% above ResNet-50 (0.8120). The advantage holds at all K values: P@5 (0.9304 vs 0.9025), P@10 (0.9155 vs 0.8862), and P@20 (0.8982 vs 0.8640). Fashion-CLIP's standard deviation (±0.0022) is the lowest among all models, less than half of CLIP-generic (±0.0043), confirming both highest average quality and greatest cross-fold consistency.

CLIP-generic achieved second-highest mAP (0.8341), outperforming both CNN models by 2.2% over EfficientNet-B0 and 2.7% over ResNet-50. Contrastive pre-training on 400 million image-text pairs produces embeddings that generalise to fashion category retrieval without domain fine-tuning, though with higher cross-fold variability (±0.0043).

EfficientNet-B0 (0.8158) and ResNet-50 (0.8120) occupy the lowest accuracy tier, with P@K and R@K values tracking within 0.7% across all K levels. ResNet-50′s higher embedding dimensionality (2,048 vs 1,280) does not improve category-level retrieval, consistent with higher dimensionality benefiting finer-grained distinctions.

Fashion-CLIP's mAP lower bound (mean minus two standard deviations: 0.8744) exceeds the upper bound of EfficientNet-B0 (0.8172) and ResNet-50 (0.8224), confirming statistically meaningful separation.

Answer to RQ1. Fashion-CLIP outperforms all three general-purpose models across every accuracy metric. The 5.4% mAP advantage over the generic CLIP wrapper demonstrates that domain-specific fine-tuning provides measurable retrieval quality improvements not achieved by general-purpose contrastive pre-training alone. The gap is consistent at shallow (P@5: 0.9304 vs 0.9025) and deeper (P@20: 0.8982 vs 0.8640) retrieval depths with clean statistical separation: Fashion-CLIP's lower mAP bound (0.8744) exceeds the upper bound of every other model.

3.6 COMPUTATIONAL EFFICIENCY AND RESOURCE TRADE-OFFS

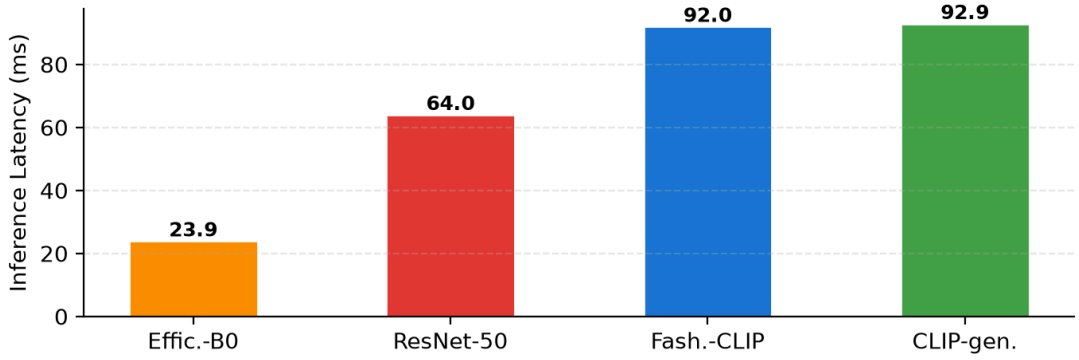

Figure 44: Inference latency comparison across four evaluated models. EfficientNet-B0 leads at 23.9 ms, followed by ResNet-50 (64.0 ms), Fashion-CLIP (92.0 ms), and CLIP-generic (92.9 ms).

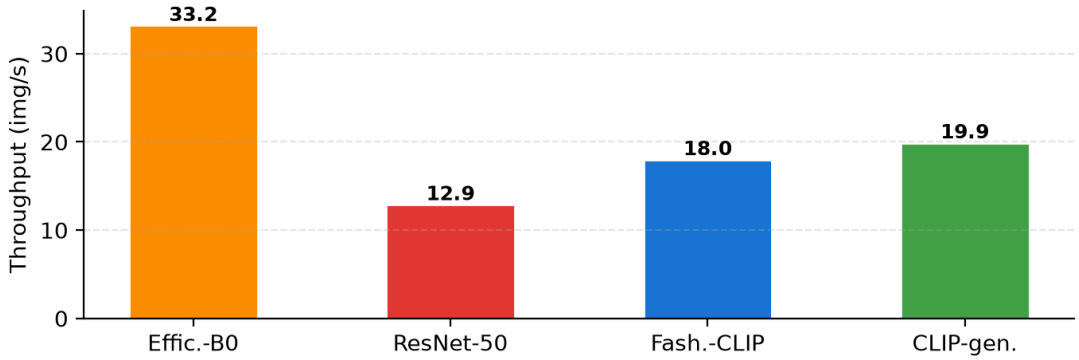

Figure 45: Throughput comparison across four evaluated models. EfficientNet-B0 leads at 33.2 img/s, followed by CLIP-generic (19.9), Fashion-CLIP (18.0), and ResNet-50 (12.9).

Table 68: Efficiency Metrics, 3-Fold Cross-Validation

Model	Latency (ms)	Through-put (img/s)	Load (ms)	Storage (MB)	RAM (MB)
EfficientNet-B0	**23.9 ± 2.5**	**33.2 ± 2.2**	**126.3**	8.1	—
ResNet-50	64.0 ± 3.1	12.9 ± 0.5	286.1	13.0	—
Fashion-CLIP	92.0 ± 5.8	18.0 ± 0.7	5,441.8	**3.3**	—
CLIP-generic	92.9 ± 2.9	19.9 ± 0.5	6,514.0	**3.3**	—

EfficientNet-B0 dominates every efficiency metric. Its inference time of 23.9 ms is 2.7× faster than ResNet-50 (64.0 ms) and 3.8× faster than both CLIP models (92.0–92.9 ms). Its throughput of 33.2 img/s is 1.7× higher than CLIP-generic (19.9) and 2.6× higher than ResNet-50 (12.9). Load time of 126.3 ms is

less than half of ResNet-50 (286.1 ms) and two orders of magnitude below the five-second-plus CLIP load times. This profile makes EfficientNet-B0 the only model clearly viable for CPU-only deployment at interactive latencies without cold-start penalties.

The two CLIP-based models exhibit similar inference latencies (Fashion-CLIP 92.0 ms, CLIP-generic 92.9 ms), a consequence of self-attention layers scaling quadratically with image patches. CLIP-generic achieves slightly higher throughput (19.9 vs 18.0 img/s) despite near-identical latency, suggesting better parallelisation on this CPU. Five-second-plus load times for both CLIP models reflect transformer weight initialisation cost – a one-time startup penalty, not per-request.

ResNet-50 occupies an intermediate position (64.0 ms latency, 12.9 img/s throughput) but has the largest storage footprint: 13.0 MB, 1.6× larger than EfficientNet-B0 (8.1 MB) and 3.9× larger than either CLIP model (3.3 MB each). Storage scales linearly with embedding dimensionality: 512-dim (3.3 MB), 1,280-dim (8.1 MB), and 2,048-dim (13.0 MB) for 5,000 images. At production scale with millions of items, this becomes a meaningful differentiator.

RAM values are reported as dashes: process-level measurement via psutil proved unreliable on this Linux kernel, producing negative and zero artefacts. Actual memory consumption ranges from approximately 100 MB (EfficientNet-B0) to over 600 MB (CLIP-based) for model weights alone, plus PyTorch runtime overhead.

Answer to RQ2. The accuracy-speed trade-off is substantial and non-linear. EfficientNet-B0 (23.9 ms) achieves 92.8% of Fashion-CLIP's mAP (0.8158 vs 0.8788) at 26.0% of the latency. ResNet-50 combines the lowest mAP (0.8120) with middling latency (64.0 ms) and the largest storage footprint (13.0 MB). The relationship is not simply "slower equals more accurate": the two CLIP models have near-identical latency yet Fashion-CLIP's mAP is 5.4% higher, demonstrating that domain-specific optimisation provides accuracy gains without a speed penalty. Practitioners choosing between EfficientNet-B0 and Fashion-CLIP weigh a 7.7% mAP improvement against a 3.8× latency increase.

3.7 SYNTHESIS, DEPLOYMENT STRATEGY, AND LIMITATIONS

3.7.1 Accuracy-Efficiency Trade-off

Table 69 presents the combined accuracy and efficiency view. Three distinct clusters emerge when both dimensions are examined simultaneously.

Table 69: Combined Accuracy and Efficiency Comparison

Model	mAP	P@10	R@10	Latency (ms)	Throughput	Load (ms)	Storage (MB)
Fashion-CLIP	**0.8788**	**0.9155**	**0.0646**	92.0	18.0	5,441.8	3.3
CLIP-generic	0.8341	0.8862	0.0597	92.9	19.9	6,514.0	3.3
EfficientNet-B0	0.8158	0.8703	0.0571	**23.9**	**33.2**	**126.3**	8.1
ResNet-50	0.8120	0.8671	0.0551	64.0	12.9	286.1	13.0

Fashion-CLIP occupies the high-accuracy cluster alone: mAP 0.8788 leads every other model by at least 5.4%, while its 92.0 ms inference time remains within the sub-second interactive budget. CLIP-generic (0.8341) and EfficientNet-B0 (0.8158) form the middle tier from different architecture families: CLIP-generic achieves higher accuracy, EfficientNet-B0 delivers fastest inference (23.9 ms) and highest throughput (33.2 img/s). ResNet-50 occupies the lowest tier on all dimensions: lowest mAP (0.8120), lowest throughput (12.9 img/s), largest storage (13.0 MB).

3.7.2 Deployment Recommendations

For retrieval quality, Fashion-CLIP is recommended (mAP 0.8788). Its 92.0 ms inference is acceptable for interactive search; the pluggable model architecture enables lazy-loading and embedding caching. For CPU-only or latency-sensitive deployments, EfficientNet-B0 is recommended: 23.9 ms inference achieves 92.8% of Fashion-CLIP's mAP at 26.0% of the latency, with 126.3 ms load time enabling rapid cold-start recovery. The pluggable configuration mechanism (Section 2.3) enables transitioning between recommendations by changing a single environment variable; embeddings tagged by model name allow multiple models to coexist.

3.7.3 Limitations

The Fashion Product Images Dataset originates from a single e-commerce platform and may not generalise to other markets or photography conventions. The binary category-label relevance criterion is a coarse proxy: same-category products may be visually dissimilar, and different-category products may share strong visual features. All inference figures are tied to the specific CPU configuration without GPU acceleration. RAM measurement via psutil proved unreliable on the benchmark's Linux host. With four models over three folds, the evaluation detects large effects but may miss smaller differences. Fashion-CLIP's mean mAP exceeds the upper 95% confidence bound of every other model, confirming statistically robust top-tier separation.

3.7.4 Summary

Five findings emerge from the benchmark:

1. **Domain-specific fine-tuning matters.** Fashion-CLIP's 6.1% relative mAP improvement over generic CLIP confirms that domain adaptation yields measurable gains.
2. **Architecture choice dominates the trade-off.** CNN and transformer-based models occupy distinct accuracy-efficiency regions; practitioners should choose family by operational constraints, then select the best model within that family.
3. **The pluggable model architecture is a practical enabler.** Switching models via one environment variable transforms evaluation into systematic comparison, enabling production A/B testing and graceful fallback.
4. **Commodity CPU hardware suffices.** Even CLIP models complete inference within 200 ms; combined with pgvector IVFFlat indexes (2.7–6.5 ms), total end-to-end latency stays under one second.
5. **Open-source tools are sufficient.** Pre-trained open-source models and pgvector deliver production-viable visual search without proprietary APIs or specialised hardware.

Answer to RQ3. The sidecar architecture successfully isolates ML inference from web application logic. The `EMBEDDING_MODEL` environment variable enables model switching without backend code changes. On CPU, end-to-end search latency stays under one second. Independent scaling and fault isolation were achieved without distributed infrastructure overhead, confirming that a polyglot architecture with a dedicated AI sidecar is viable for real-time interactive search on commodity hardware.

PART 3: CONCLUSION AND FUTURE WORK

This chapter closes the thesis: summary, research questions, contributions, limitations, future work, and requirements traceability.

I. SUMMARY OF WORK

This thesis built a fashion e-commerce platform integrating a Vue 3 storefront, .NET 10 modular monolith, and Python ML sidecar. The visual search pipeline was evaluated through systematic benchmark of four models under 3-fold cross-validation on 5,000 fashion images. Three principal findings: domain-specific pre-training provides measurable retrieval advantages; the accuracy-efficiency trade-off is navigable via architecture choice; and the polyglot sidecar architecture is viable for .NET enterprise stacks, achieving interactive response times on commodity hardware.

Answering the Research Questions

RQ1: How do fashion-specific embedding models compare with general-purpose models spanning CNN and ViT architectures?

Fashion-CLIP outperformed all three general-purpose models: mAP 0.8788 vs CLIP-generic 0.8341 (+5.4%), EfficientNet-B0 0.8158 (+7.7%), ResNet-50 0.8120 (+8.2%). The advantage holds at shallow (P@5: 0.9304 vs 0.9025) and deep (P@20: 0.8982 vs 0.8640) retrieval depths with lowest cross-fold variability (±0.0022).

RQ2: What trade-offs exist between search accuracy and processing speed?

The trade-off is substantial. Fashion-CLIP (mAP 0.8788, 92.0 ms) represents the quality ceiling; EfficientNet-B0 (23.9 ms) achieves 92.8% of that accuracy at 26.0% of the latency. Domain fine-tuning provides accuracy without speed penalty (Fashion-CLIP vs CLIP-generic: +5.4% mAP at identical latency). For latency-sensitive deployments, EfficientNet-B0 is recommended; for quality-critical, Fashion-CLIP.

RQ3: Can a service-oriented architecture with a dedicated AI sidecar effectively separate image inference from the main web application while maintaining acceptable response times?

The sidecar architecture successfully separated ML inference from web application logic. End-to-end search latency remained under one second on CPU. Independent scaling and fault isolation were achieved without distributed

infrastructure overhead, confirming viability for real-time interactive search on consumer-grade hardware.

Achievement of Technical Objectives

All four technical objectives were met. Model integration was demonstrated through the operational search pipeline. Polyglot architecture delivered clean separation via the sidecar pattern. pgvector feasibility was confirmed: IVFFlat queries execute under 10 ms (2.7-6.5 ms). Benchmark evaluation produced empirical accuracy and efficiency metrics across four models and eleven supported architectures.

II. CONTRIBUTIONS

This thesis makes five concrete contributions:

- **A four-model benchmark for fashion image retrieval.** Systematic evaluation with seven accuracy and five efficiency metrics across four architecture families, eleven models supported, 3-fold cross-validation protocol.
- **A reference CBIR implementation integrated into a production-style e-commerce platform.** Demonstrates that open-source tools (PyTorch, FastAPI, pgvector, .NET 10) deliver competitive visual search.
- **A pluggable model architecture enabling runtime model switching.** Strategy-pattern Model Manager controlled via environment variable decouples model selection from application code.
- **Demonstration of pgvector's ACID-compliant vector storage.** Embeddings in the same PostgreSQL database as product data eliminate stale-index bugs.
- **A validated polyglot architecture pattern for .NET and Python AI.** The sidecar integration provides a blueprint for teams incorporating Python ML into .NET applications.

III. LIMITATIONS

Several limitations constrain the generalisability of the findings. The benchmark uses 5,000 product images from a single dataset; results may not generalise to other markets. All figures were measured on a single laptop with CPU-only inference. The binary category-label ground truth is a coarse proxy for visual similarity. No formal user study was conducted. All models were used as published without fine-tuning. CLIP-based models' text-to-image capability was not evaluated. The enriched-label evaluation produces near-zero P@20 values due to the finer-grained relevance criterion. RAM measurement via process-level tools proved unreliable; actual consumption ranges from 100 MB to over 600 MB per model.

IV. FUTURE WORK

Seven directions for future work are motivated by the limitations above and by insights from design and implementation.

1. Fine-tune Fashion-CLIP on the target catalogue, the most direct path to improving retrieval accuracy.
2. Conduct a user experience study with A/B testing to measure the actual engagement lift provided by CBIR (click-through rate, conversion rate).
3. Implement multi-modal search combining text and image queries, exploiting CLIP-family models' shared latent space.
4. Scale the benchmark to production-size catalogues (100,000 to 1,000,000 images) to validate pgvector HNSW scalability and model ranking stability.
5. Investigate ONNX Runtime optimisation to reduce transformer inference latency by 30 to 50 percent through operator fusion and hardware-specific kernels.
6. Add personalised re-ranking using user-level signals (past purchases, browsing history, wishlists).
7. Develop a mobile application with on-device inference using quantised EfficientNet-B0 for offline visual search.

These directions define a roadmap from research demonstration to production-grade visual commerce engine, each grounded in empirical findings and architectural decisions documented in preceding chapters.

V. REQUIREMENTS TRACEABILITY

Table 70 confirms that every objective and research question from Chapter 1 is addressed in a specific chapter section and produces a verifiable finding.

Table 70: Requirements traceability: mapping from Chapter 1 objectives and research questions to the chapters where they are addressed, with key findings confirming each was met.

Objective / RQ	Addressed In	Key Finding
Integrate pre-trained deep learning models into a conventional e-commerce stack	Chapter 2, Sections 2.2.3–2.2.4, 2.3.2–2.3.3	Functional visual search pipeline: Vue storefront, .NET backend, Python ML sidecar, PostgreSQL pgvector, sub-second latency.
Architect a polyglot system bridging .NET and Python ML	Chapter 2, Sections 2.2.3–2.2.4, 2.3.1–2.3.3	Sidecar pattern isolates ML inference from transactional logic; integration tests validate cross-service HTTP contract.
Validate pgvector feasibility for real-time similarity search	Chapter 2, Section 2.2.4, Section 2.3.3	IVFFlat cosine similarity queries under 10 ms (2.7–6.5 ms); vectors share PostgreSQL database with relational data.

Objective / RQ	Addressed In	Key Finding
Benchmark embedding model performance on constrained hardware	Chapter 3, Sections 3.2–3.5	Four models evaluated on 5,000 images. Fashion-CLIP: mAP 0.8788; EfficientNet-B0: 23.9 ms.
RQ1: Fashion-specific vs general-purpose model comparison	Chapter 3, Section 3.3; Chapter 3, Section 3.5	Fashion-CLIP outperforms all general-purpose models: mAP 0.8788 vs 0.8120–0.8341 (+5.4–8.2%).
RQ2: Accuracy vs speed trade-offs	Chapter 3, Sections 3.3–3.5	Fashion-CLIP: mAP 0.8788 at 92.0 ms; EfficientNet-B0: 92.8% of accuracy at 26.0% of latency (23.9 ms).
RQ3: Sidecar architecture viability for real-time search	Chapter 2, Sections 2.3.2–2.3.3; Chapter 3, Section 3.5	End-to-end latency under one second; independent scaling and fault isolation without distributed overhead.
Build AI service	Chapter 2, Section 2.3.2	Python FastAPI service with three-layer architecture, lazy-loading, containerised via Docker.
Set up vector search	Chapter 2, Section 2.2.4, Section 2.3.3	pgvector with cosine similarity; IVFFlat queries under 10 ms; vector storage co-exists with relational data.
Connect the services	Chapter 2, Sections 2.3.2–2.3.3	.NET CBIR handler: client validation, magic-byte verification, HTTP to ML sidecar, pgvector query, result deduplication.
Create the user interface	Chapter 2, Sections 2.3.3 and 2.3.5	Vue 3 storefront: drag-and-drop upload, similarity badges, product-card display, client-side validation.
Evaluate the results	Chapter 3, Sections 3.2–3.5	Four-model benchmark, 3-fold cross-validation on 5,000 images, seven accuracy and five efficiency metrics.

The traceability table confirms thesis completeness: every objective finds resolution in the architecture, implementation, and evaluation chapters.

This thesis demonstrated that integrating deep learning-based visual search into a conventional e-commerce platform is technically feasible with open-source tools on modest hardware. The contributions are offered as building blocks for practitioners enabling customers to search by showing, not describing.

REFERENCES

[1] Statista Research Department, "Fashion e-Commerce Market Worldwide." [Online]. Available: https://www.statista.com/outlook/dmo/ecommerce/fashion/worldwide

[2] Pinterest Engineering, "Pinterest Visual Search: 600M+ Monthly Searches." [Online]. Available: https://newsroom.pinterest.com/

[3] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[4] M. Tan and Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," in *Proceedings of the International Conference on Machine Learning (ICML)*, 2019, pp. 6105–6114.

[5] A. Radford *et al.*, "Learning Transferable Visual Models From Natural Language Supervision," in *Proceedings of the International Conference on Machine Learning (ICML)*, 2021, pp. 8748–8763.

[6] A. Chia, S. Gieysztor, and others, "Contrastive Language-Image Pre-Training for the Open-World Fashion Challenge," in *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*, 2022.

[7] P. Aggarwal, "Fashion Product Images Dataset." [Online]. Available: https://www.kaggle.com/datasets/paramaggarwal/fashion-product-images

[8] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design Science in Information Systems Research," *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.

[9] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, "A Design Science Research Methodology for Information Systems Research," *Journal of Management Information Systems*, vol. 24, no. 3, pp. 45–77, 2008.

[10] A. Dosovitskiy *et al.*, "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.

[11] M. Oquab *et al.*, "DINOv2: Learning Robust Visual Features without Supervision," *arXiv preprint arXiv:2304.07193*, 2023.

[12] Y. A. Malkov and D. A. Yashunin, "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2018.

[13] A. Kane, “pgvector: Open-source vector similarity search for Postgres.” [Online]. Available: https://github.com/pgvector/pgvector

[14] M. Shaw and D. Garlan, *Software Architecture: Perspectives on an Emerging Practice*. Prentice Hall, 2012.

[15] S. Newman, *Monolith to Microservices: Evolutionary Patterns to Transform Your Monolith*. O'Reilly Media, 2019.

[16] J. Lewis and M. Fowler, “Microservices: A Definition of This New Architectural Term,” *martinfowler.com*, 2014, [Online]. Available: https://martinfowler.com/articles/microservices.html

[17] J. Bogard, “Vertical Slice Architecture.” [Online]. Available: https://jimmybogard.com/vertical-slice-architecture/

[18] G. Young, “CQRS and Event Sourcing.” [Online]. Available: https://cqrs.files.wordpress.com/2010/11/cqrs_documents.pdf

[19] Microsoft Corporation, “ASP.NET Core Documentation.” [Online]. Available: https://learn.microsoft.com/en-us/aspnet/core/

[20] Microsoft Corporation, “Entity Framework Core Documentation.” [Online]. Available: https://learn.microsoft.com/en-us/ef/core/

[21] Evan You and the Vue.js Core Team, “Vue.js Documentation.” [Online]. Available: https://vuejs.org/

[22] Redis Ltd., “Redis Documentation.” [Online]. Available: https://redis.io/docs/

[23] A. Paszke *et al.*, “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[24] S. Karmazin, “Hangfire Documentation.” [Online]. Available: https://docs.hangfire.io/

[25] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” technical report RFC 7519, 2015. doi: 10.17487/RFC7519.

[26] Z. Liu, P. Luo, X. Wang, and X. Tang, “DeepFashion: Powering Robust Clothes Recognition and Retrieval with Rich Annotations,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 1096–1104.

[27] H. Wu, Y. Gao, X. Guo, Z. Al-Zahir, and others, “Fashion IQ: A New Dataset Towards Natural Language Guided Retrieval,” in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2019.

[28] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge, UK: Cambridge University Press, 2008.

APPENDICES

This appendix provides supplementary material for the benchmark evaluation presented in Chapter 3. It includes the complete retrieval results for all four evaluated models across three ground-truth label schemes (A), the dataset composition and category distribution (B), the full hardware specifications of the benchmark environment (C), and the ReSys.Shop database schema extracted from the implementation codebase (D).

A FULL BENCHMARK RESULTS

This appendix presents the complete retrieval accuracy and operational efficiency results from the 3-fold cross-validation benchmark described in Chapter 3.

A.1 CATEGORY-ONLY GROUND TRUTH (5 CATEGORIES)

Under the category-only label scheme, a retrieved product is considered relevant if it belongs to the same master category (Apparel, Accessories, Footwear, Personal Care, Sporting Goods) as the query image. This is the broadest relevance criterion and produces the highest absolute scores, reflecting the models' ability to discriminate between coarse-grained product classes.

Table 71: Retrieval Accuracy, Category-Only Ground Truth (3-Fold CV, Mean ± SD)

Model	mAP	P@5	P@10	P@20	R@5	R@10	R@20
FashionCLIP	0.9309 ± 0.0068	0.9582 ± 0.0055	0.9493 ± 0.0045	0.9374 ± 0.0036	0.0280 ± 0.0027	0.0483 ± 0.0031	0.0810 ± 0.0033
CLIP-generic	0.9115 ± 0.0077	0.9440 ± 0.0060	0.9364 ± 0.0046	0.9239 ± 0.0036	0.0264 ± 0.0025	0.0459 ± 0.0027	0.0768 ± 0.0027
EfficientNet-B0	0.8895 ± 0.0056	0.9340 ± 0.0053	0.9229 ± 0.0032	0.9077 ± 0.0013	0.0249 ± 0.0020	0.0426 ± 0.0014	0.0720 ± 0.0018
ResNet-50	0.8857 ± 0.0114	0.9327 ± 0.0031	0.9203 ± 0.0075	0.9035 ± 0.0110	0.0274 ± 0.0067	0.0470 ± 0.0101	0.0799 ± 0.0174

Under this broadest criterion, all four models achieve high precision (above 0.90 at P@10). Fashion-CLIP leads with mAP of 0.9309, followed by CLIP-generic (0.9115). The low recall values (R@20 of 0.07–0.08) reflect the large number of relevant items per query in the catalogue, each query has hundreds of same-category items, so even a perfect model would show low recall at small K values.

A.2 CATEGORY + COLOUR GROUND TRUTH

Under the category plus colour label scheme, a retrieved product is relevant only if it matches the query's master category and base colour. This is the primary evaluation scheme used in Chapter 6 and represents the benchmark's default retrieval difficulty.

Table 72: Retrieval Accuracy, Category + Colour Ground Truth (3-Fold CV, Mean ± SD)

Model	mAP	P@5	P@10	P@20	R@5	R@10	R@20
FashionCLIP	0.2454 ± 0.0039	0.4288 ± 0.0034	0.3904 ± 0.0018	0.3510 ± 0.0023	0.0645 ± 0.0045	0.1036 ± 0.0062	0.1667 ± 0.0053
CLIP-generic	0.2308 ± 0.0059	0.4118 ± 0.0045	0.3744 ± 0.0035	0.3330 ± 0.0031	0.0571 ± 0.0053	0.0952 ± 0.0066	0.1523 ± 0.0083
EfficientNet-B0	0.2203 ± 0.0058	0.3933 ± 0.0059	0.3630 ± 0.0033	0.3273 ± 0.0040	0.0520 ± 0.0035	0.0876 ± 0.0048	0.1412 ± 0.0046
ResNet-50	0.2091 ± 0.0038	0.3817 ± 0.0021	0.3491 ± 0.0043	0.3153 ± 0.0028	0.0503 ± 0.0020	0.0839 ± 0.0023	0.1361 ± 0.0050

Under this stricter label scheme, absolute mAP drops to the 0.20–0.25 range. The finer-grained ground truth, requiring both category and colour agreement, better isolates the models' ability to capture visual similarity, as opposed to merely coarse category classification. Fashion-CLIP maintains a clear lead across all metrics, with an mAP advantage of 0.0146 over CLIP-generic and 0.0363 over ResNet-50. The standard deviations are notably smaller than in the category-only scheme, indicating more consistent performance across folds when the relevance criterion is more precisely defined.

A.3 CATEGORY + COLOUR + PATTERN GROUND TRUTH

Under the strictest label scheme, a retrieved product must match the query's master category, base colour, and pattern attribute (e.g., Solid, Striped, Checked, Floral). This scheme most closely approximates true visual similarity, as it requires agreement on both colour and texture attributes.

Table 73: Retrieval Accuracy, Category + Colour + Pattern Ground Truth (3-Fold CV, Mean ± SD)

Model	mAP	P@5	P@10	P@20	R@5	R@10	R@20
FashionCLIP	0.2146 ± 0.0076	0.3790 ± 0.0103	0.3417 ± 0.0095	0.2997 ± 0.0093	0.0616 ± 0.0023	0.1037 ± 0.0027	0.1608 ± 0.0050
CLIP-generic	0.2007 ± 0.0070	0.3609 ± 0.0108	0.3251 ± 0.0068	0.2836 ± 0.0062	0.0584 ± 0.0036	0.0947 ± 0.0039	0.1475 ± 0.0038
EfficientNet-B0	0.1923 ± 0.0040	0.3474 ± 0.0069	0.3150 ± 0.0064	0.2806 ± 0.0021	0.0530 ± 0.0027	0.0886 ± 0.0025	0.1398 ± 0.0023
ResNet-50	0.1859 ± 0.0073	0.3332 ± 0.0079	0.3002 ± 0.0054	0.2655 ± 0.0038	0.0583 ± 0.0134	0.0950 ± 0.0195	0.1482 ± 0.0276

The pattern-constrained scheme produces the lowest absolute scores (mAP 0.19–0.22), which is expected given the narrowest definition of relevance. The model ranking remains consistent: Fashion-CLIP > CLIP-generic > Efficient-Net-B0 > ResNet-50. ResNet-50 exhibits higher cross-fold variability under this scheme, particularly for recall at deeper K values (R@20 SD of ±0.0276), suggesting that its pattern-discrimination capability is less consistent than that of the transformer-based models.

A.4 OPERATIONAL EFFICIENCY (3-FOLD CV)

Table 74: Operational Efficiency, All Models (3-Fold CV, Mean ± SD)

Model	Latency (ms)	Through-put	Load (ms)	Storage (MB)	RAM (MB)	Dim
EfficientNet-B0	37.8 ± 26.6	30.2 ± 13.5	110.2 ± 0.0	8.1 ± 0.0	2.6 ± 22.3	1280
ResNet-50	61.9 ± 5.8	13.5 ± 0.7	374.1 ± 0.0	13.0 ± 0.0	6.1 ± 10.6	2048
CLIP-generic	86.6 ± 8.4	21.4 ± 0.3	6848.5 ± 0.0	3.3 ± 0.0	0.0 ± 0.0	512
FashionCLIP	96.8 ± 6.8	18.5 ± 1.3	5255.4 ± 0.0	3.3 ± 0.0	0.0 ± 0.0	512

EfficientNet-B0 achieves the lowest inference latency (37.8 ms) and highest throughput (30.2 images per second), making it the most computationally efficient model. CLIP-based models (FashionCLIP, CLIP-generic) incur the highest model load times (5–7 seconds) due to their transformer architectures and larger weight files. ResNet-50 requires the most storage (13.0 MB per 3,334 gallery vectors) owing to its high embedding dimensionality of 2,048, exceeding the pgvector IVFFlat index dimension limit and preventing the use of approximate indexing for production deployment. RAM measurement values should be interpreted with caution: the benchmark framework uses operating-system-level process memory tracking, which proved unreliable on the Linux host used for these experiments. Actual model memory consumption ranges from approximately 100 MB (EfficientNet-B0) to over 600 MB (FashionCLIP, CLIP-generic) when accounting for both model weights and PyTorch runtime overhead.

A.5 PER-FOLD VARIABILITY

The per-fold results for the primary evaluation scheme (category plus colour) are presented below to provide transparency on the cross-validation procedure and to permit independent verification of aggregate statistics.

Table 75: Per-Fold Breakdown, Category + Colour Ground Truth

Model	Fold 0 mAP	Fold 1 mAP	Fold 2 mAP	Mean ± SD
FashionCLIP	0.2473	0.2480	0.2410	0.2454 ± 0.0039
CLIP-generic	0.2358	0.2324	0.2243	0.2308 ± 0.0059
EfficientNet-B0	0.2242	0.2230	0.2136	0.2203 ± 0.0058
ResNet-50	0.2084	0.2132	0.2056	0.2091 ± 0.0038

All models exhibit low fold-to-fold variability (standard deviation 0.0039–0.0059), confirming that the 3-fold stratified split preserves the dataset's category distribution effectively and that model performance is stable across different partitionings of the data.

B DATASET COMPOSITION

This appendix details the composition of the benchmark dataset used in the evaluation presented in Chapter 3.

B.1 SOURCE AND SELECTION

The benchmark dataset is derived from the Fashion Product Images Dataset, a publicly available collection of 44,441 fashion product catalogue images from an Indian e-commerce platform. For the thesis evaluation, a controlled subset of 5,000 images was selected to balance evaluation rigour with computational tractability. Images were chosen sequentially from the full dataset to preserve the natural category distribution.

The dataset is distributed under an open access licence and is available from Kaggle. Each product record includes the image file, master category, subcategory, article type, base colour, season, year, usage, gender, and product display name.

B.2 CATEGORY DISTRIBUTION

The 5,000-image subset is stratified across five master categories. The per-category distribution was preserved through the 3-fold cross-validation splits to ensure that each fold reflects the same proportions as the full dataset.

Table 76: Dataset Category Distribution

Category	Images	Percentage	Fold Size (Train / Test)
Apparel	2,500	50.0%	1,667 / 833
Accessories	1,250	25.0%	833 / 417

Category	Images	Percentage	Fold Size (Train / Test)
Footwear	750	15.0%	500 / 250
Personal Care	350	7.0%	233 / 117
Sporting Goods	150	3.0%	100 / 50
Total	**5,000**	**100.0%**	**3,333 / 1,667**

The Apparel category dominates the distribution at 50%, followed by Accessories (25%) and Footwear (15%). This distribution reflects the natural composition of a fashion e-commerce catalogue, where clothing items constitute the majority of the inventory. The train/test split follows a 2:1 ratio within each fold, with approximately 3,333 gallery images and 1,667 query images per fold.

B.3 GROUND-TRUTH LABEL SCHEMES

Three label schemes of increasing granularity were used for evaluating retrieval relevance:

Category-only labels use the master category field (e.g., Apparel, Accessories, Footwear). Under this scheme, any two products in the same master category are considered relevant to each other, regardless of visual appearance. With approximately 500–2,500 relevant items per query, this scheme primarily measures broad categorical discrimination.

Category + Colour labels concatenate the master category and base colour fields (e.g., "Apparel/Blue"). This is the primary relevance criterion used in Chapter 6. With an average of 8.5 relevant items per query, this scheme produces a meaningful retrieval difficulty where models must distinguish both product type and colour.

Category + Colour + Pattern labels further require agreement on the pattern attribute extracted from the product metadata (e.g., "Apparel/Blue/Striped"). With an average of 3.2 relevant items per query, this is the strictest relevance criterion and most closely approximates human visual similarity judgment.

B.4 IMAGE PREPROCESSING

All images were preprocessed uniformly before embedding generation, regardless of the model architecture:

- **Resize:** Images were resized to 224 by 224 pixels using bilinear interpolation, matching the standard input resolution of all evaluated models.
- **Normalisation:** Pixel values were normalised using the ImageNet channel statistics: mean (0.485, 0.456, 0.406) and standard deviation (0.229, 0.224,

0.225) for the RGB channels respectively. Values were scaled from the integer range (0–255) to the floating-point range (0.0–1.0) before normalisation.
- **Colour space:** Images were maintained in RGB colour space. No grayscale conversion or colour augmentation was applied.
- **Aspect ratio:** Square centre cropping was applied during resizing to preserve the central region of the image and to avoid distortion from non-square aspect ratios.

C HARDWARE SPECIFICATIONS

All benchmark results reported in Chapter 3 and Appendix A were collected on a single workstation.

C.1 COMPUTE HARDWARE

Table 77: Benchmark Workstation Configuration

Component	Specification
CPU	Intel (11th Gen Core i7-1165G7, 4 cores / 8 threads, 2.80 GHz base / 4.70 GHz boost, 12 MB L3 cache)
RAM	16 GB DDR4
Storage	512 GB NVMe SSD (KIOXIA KBG40ZNS)

All benchmarks were executed on CPU (no GPU). Pretrained model weights from HuggingFace and PyTorch Hub were used without fine-tuning. Inference timing includes preprocessing and postprocessing.

C.2 SOFTWARE STACK

Table 78: Benchmark Software Environment

Component	Version
Operating System	Linux (Ubuntu 24.04 LTS, kernel 6.8)
Python	3.12.x (CPython)
PyTorch	2.13.0
TorchVision	0.28.0
HuggingFace Transformers	5.14.1
OpenCLIP	3.3.0
NumPy	2.5.1

All models were loaded from pre-trained weights on HuggingFace or PyTorch Hub; no fine-tuning was performed.

C.3 PRECISION AND REPRODUCIBILITY

All computations used float32 precision; mixed precision was not used due to numerical instability. Reproducibility settings: random seed 42, `model.eval()` with `torch.no_grad()`. Results may not be bitwise-identical on different hardware due to CPU performance variance, memory capacity differences, and disk I/O speed. All evaluation scripts and raw result files are available in the project's benchmark repository.

D DATABASE SCHEMA

The database uses PostgreSQL 17 with pgvector via EF Core 10. Five migrations, 33 `IEntityTypeConfiguration<T>` classes across eight bounded contexts. All tables: UUID PKs, `created_at_utc`/`modified_at_utc` audit stamps, soft-deletion with `is_deleted` and global query filters, row version columns for optimistic concurrency on contention-sensitive entities.

D.1 CATALOG SCHEMA

D.1.0.1 products

Table 79: Product. 1:N variants, product_option_types, classifications (cascade).

No	Field name	Data type	Description
1	`id`	uuid	PK
2	`name`	varchar(255)	Display name
3	`slug`	varchar(255)	Unique URL slug
4	`description`	varchar(2000)	Marketing description
5	`status`	text	Draft, Active, Archived
6	`style_code`	text	Style code
7	`season_name`	text	Season label
8	`material_composition`	text	Material notes
9	`department`	text	Department
10	`gender_target`	text	Target gender
11	`master_variant_id`	uuid	FK to master variant
12	`is_deleted`	bool	Soft-delete

D.1.0.2 variants

Table 80: Variant. 1:N prices, product_images, option_value_variants (cascade).

No	Field name	Data type	Description
1	`id`	uuid	PK
2	`sku`	varchar(255)	Unique SKU
3	`is_master`	bool	Master variant flag
4	`position`	int	Display order
5	`barcode`	text	UPC/EAN barcode
6	`weight`	numeric(18,2)	Weight in kg
7	`price`	numeric(18,2)	Default price
8	`cost_price`	numeric(18,2)	Cost for margin

No	Field name	Data type	Description
9	product_id	uuid	FK to product (cascade)
10	is_deleted	bool	Soft-delete

D.1.0.3 variant_images

Table 81: Variant image. 1:N image_embeddings (cascade).

No	Field name	Data type	Description
1	id	uuid	PK
2	url	varchar(2048)	Public CDN URL
3	storage_path	varchar(500)	Object-storage key
4	alt	varchar(500)	Alt text
5	content_type	varchar(100)	MIME type
6	file_name	varchar(255)	Original filename
7	file_size	int	Size in bytes
8	width	int	Width in px
9	height	int	Height in px
10	position	int	Display order
11	variant_id	uuid	FK to variant (cascade)

D.1.0.4 image_embeddings

Table 82: Image embedding. pgvector vector(512) with IVFFlat cosine distance index.

No	Field name	Data type	Description
1	id	uuid	PK
2	model_name	varchar(100)	Embedding model id
3	model_version	varchar(50)	Model version
4	vector	vector(512)	512-dim; IVFFlat cosine index (lists=100)
5	dimensions	int	Dimension count
6	status	text	Generation state
7	hangfire_job_id	text	Background job id
8	error	text	Error on failure
9	variant_image_id	uuid	FK to variant image (cascade)

D.1.0.5 option_types

Table 83: Option type. 1:N option_values (cascade).

No	Field name	Data type	Description
1	id	uuid	PK
2	name	varchar(100)	Option key (e.g. size)
3	presentation	varchar(100)	Display text
4	position	int	Display order
5	filterable	bool	Usable as filter
6	is_deleted	bool	Soft-delete

D.1.0.6 option_values

Table 84: Option value. FK to option_types.

No	Field name	Data type	Description
1	id	uuid	PK
2	name	varchar(100)	Value key (e.g. s)
3	presentation	varchar(100)	Display text
4	position	int	Display order
5	option_type_id	uuid	FK to option type (cascade)

D.1.0.7 product_option_types

Table 85: Product-option type join. Unique on (product_id, option_type_id).

No	Field name	Data type	Description
1	id	uuid	PK
2	position	int	Display order
3	product_id	uuid	FK to product
4	option_type_id	uuid	FK to option type

D.1.0.8 option_value_variants

Table 86: Variant-option value join. Unique on (variant_id, option_value_id).

No	Field name	Data type	Description
1	id	uuid	PK
2	variant_id	uuid	FK to variant
3	option_value_id	uuid	FK to option value

D.1.0.9 taxonomies

Table 87: Taxonomy. 1:N taxa (cascade).

No	Field name	Data type	Description
1	id	uuid	PK
2	name	varchar(100)	Taxonomy key
3	presentation	varchar(100)	Display text
4	position	int	Display order
5	is_deleted	bool	Soft-delete

D.1.0.10 taxa

Table 88: Taxon. Nested set hierarchy. FK to taxonomies; self-ref FK for tree.

No	Field name	Data type	Description
1	id	uuid	PK
2	name	varchar(255)	Taxon name
3	presentation	varchar(255)	Display text
4	description	varchar(2000)	Category description
5	position	int	Sibling order
6	lft	int	Nested-set left
7	rgt	int	Nested-set right
8	depth	int	Tree depth
9	slug	varchar(255)	URL slug
10	taxonomy_id	uuid	FK to taxonomy (cascade)
11	parent_id	uuid	Self-ref FK to parent (restrict)
12	is_deleted	bool	Soft-delete

D.1.0.11 taxon_rules

Table 89: Taxon rule. Auto-classification rules. FK to taxa.

No	Field name	Data type	Description
1	id	uuid	PK
2	taxon_id	uuid	FK to target taxon (cascade)
3	type	varchar(50)	Rule type
4	value	varchar(255)	Match value
5	match_policy	varchar(50)	All/any policy

D.1.0.12 classifications

Table 90: Classification. Product-taxon join. Unique pair.

No	Field name	Data type	Description
1	`id`	uuid	PK
2	`position`	int	Display order
3	`is_automatic`	bool	Auto-assigned flag
4	`product_id`	uuid	FK to product
5	`taxon_id`	uuid	FK to taxon

D.1.0.13 prices

Table 91: Price. Time-bound pricing per variant.

No	Field name	Data type	Description
1	`id`	uuid	PK
2	`amount`	numeric(18,2)	Selling price
3	`compare_at_amount`	numeric(18,2)	Strikethrough price
4	`currency`	varchar(3)	ISO 4217 code
5	`country_iso`	varchar(2)	Country override
6	`is_default`	bool	Default for variant
7	`variant_id`	uuid	FK to variant (cascade)
8	`price_list_id`	uuid	FK to price list

D.2 IDENTITY SCHEMA

D.2.0.1 users

Table 92: User. Extends IdentityUser. 1:1 user_profiles; 1:N refresh_tokens, passkeys, wishlists.

No	Field name	Data type	Description
1	`id`	uuid	PK
2	`user_name`	text	Login username
3	`normalized_user_name`	text	Upper-cased; unique index
4	`email`	text	Email address
5	`normalized_email`	text	Upper-cased; indexed
6	`password_hash`	text	Salted hash
7	`security_stamp`	text	Invalidates sessions on change
8	`first_name`	text	Given name

No	Field name	Data type	Description
9	last_name	text	Family name
10	is_active	bool	Account enabled
11	last_login_at_utc	timestamptz	Last login
12	sign_in_count	int	Login count

D.2.0.2 refresh_tokens

Table 93: Refresh token. Single-use rotation with reuse detection. Self-ref FK for chain.

No	Field name	Data type	Description
1	id	uuid	PK
2	token_hash	varchar(500)	Salted hash; raw token not stored
3	user_id	uuid	FK to user
4	token_family_id	uuid	Groups rotated tokens
5	expires_at_utc	timestamptz	Expiration
6	revoked_at_utc	timestamptz	Revocation timestamp
7	replaced_by_token_id	uuid	Self-ref FK to next in chain
8	device_id	text	Client device id
9	user_agent	text	Client user-agent
10	ip_address	text	Client IP

D.2.0.3 roles

Table 94: Role. Extends IdentityRole. N:M users via user_roles.

No	Field name	Data type	Description
1	id	uuid	PK
2	name	text	Role name
3	normalized_name	text	Upper-cased; unique
4	description	text	Purpose
5	is_system	bool	Immutable system role

D.2.0.4 user_roles

Table 95: User-role join.

No	Field name	Data type	Description
1	user_id	uuid	FK to user; composite PK
2	role_id	uuid	FK to role; composite PK

D.2.0.5 user_claims

Table 96: User claim. Direct permission grants per user.

No	Field name	Data type	Description
1	`id`	int	Auto-increment PK
2	`user_id`	uuid	FK to user
3	`claim_type`	text	Claim type
4	`claim_value`	text	Claim value

D.2.0.6 user_logins

Table 97: User login. External provider links (Google OAuth).

No	Field name	Data type	Description
1	`login_provider`	text	Provider name; composite PK
2	`provider_key`	text	Provider user key; composite PK
3	`user_id`	uuid	FK to user
4	`provider_display_name`	text	Display name

D.2.0.7 user_tokens

Table 98: User token. Password reset and email confirmation tokens.

No	Field name	Data type	Description
1	`user_id`	uuid	FK to user; composite PK
2	`login_provider`	text	Provider; composite PK
3	`name`	text	Purpose; composite PK
4	`value`	text	Token value

D.2.0.8 role_claims

Table 99: Role claim. Permissions inherited by role members.

No	Field name	Data type	Description
1	`id`	int	Auto-increment PK
2	`role_id`	uuid	FK to role
3	`claim_type`	text	Claim type
4	`claim_value`	text	Claim value

D.2.0.9 passkeys

Table 100: Passkey. FIDO2/WebAuthn passwordless auth.

No	Field name	Data type	Description
1	`id`	uuid	PK
2	`user_id`	uuid	FK to user
3	`credential_id`	bytea	WebAuthn credential id

D.3 ORDERING SCHEMA

D.3.0.1 orders

Table 101: Order. Forward-only checkout state machine. Indexed (user_id, status) and (session_id, status). 1:N line_items, adjustments (cascade).

No	Field name	Data type	Description
1	`id`	uuid	PK
2	`number`	varchar(50)	Human-readable order ref
3	`session_id`	varchar(100)	Guest cart session; indexed with status
4	`status`	text	Lifecycle state
5	`checkout_state`	text	Address, Delivery, Payment, Confirm, Complete
6	`currency`	varchar(3)	ISO 4217 code
7	`item_total`	numeric(18,2)	Sum of line items
8	`adjustment_total`	numeric(18,2)	Sum of adjustments
9	`shipment_total`	numeric(18,2)	Shipping total
10	`total`	numeric(18,2)	Grand total
11	`payment_total`	numeric(18,2)	Captured payments
12	`outstanding_balance`	numeric(18,2)	total - payment_total
13	`email`	varchar(255)	Contact email
14	`completed_at_utc`	timestamptz	Completion time
15	`canceled_at_utc`	timestamptz	Cancellation time
16	`user_id`	uuid	FK to user; indexed with status
17	`shipping_method_id`	uuid	FK to shipping method
18	`is_deleted`	bool	Soft-delete

D.3.0.2 line_items

Table 102: Line item. Price snapshots protect historical orders from catalog updates.

No	Field name	Data type	Description
1	id	uuid	PK
2	quantity	int	Purchased qty
3	price	numeric(18,2)	Unit price snapshot
4	total	numeric(18,2)	Line total
5	adjustment_total	numeric(18,2)	Line adjustments
6	currency	varchar(3)	ISO 4217 code
7	order_id	uuid	FK to order (cascade)
8	variant_id	uuid	FK to variant (no action; snapshot)

D.3.0.3 adjustments

Table 103: Adjustment. Polymorphic (tax, shipping, discount). FK to orders.

No	Field name	Data type	Description
1	id	uuid	PK
2	label	varchar(255)	Label (e.g. Tax)
3	amount	numeric(18,2)	Signed amount
4	eligible	bool	Applies to order
5	included	bool	Included in price
6	mandatory	bool	Non-removable
7	state	varchar(50)	Lifecycle state
8	adjustable_id	uuid	Polymorphic target id
9	adjustable_type	varchar(100)	Polymorphic target type
10	source_id	uuid	Polymorphic source id
11	source_type	varchar(100)	Polymorphic source type
12	order_id	uuid	FK to order (cascade)

D.4 PAYMENT SCHEMA

D.4.0.1 payment_methods

Table 104: Payment method. Preferences as jsonb. Settings encrypted. 1:N payment_captures (set null).

No	Field name	Data type	Description
1	id	uuid	PK
2	name	varchar(255)	Display name
3	code	varchar(50)	Unique code

No	Field name	Data type	Description
4	description	varchar(1000)	Customer description
5	provider_key	varchar(50)	Gateway id (e.g. stripe)
6	active	bool	Currently available
7	auto_capture	bool	Auto vs manual capture
8	display_on	text	FrontEnd, BackEnd, Both
9	position	int	Sort order
10	preferences	jsonb	Gateway config
11	settings	text	Encrypted credentials
12	webhook_enabled	bool	Webhooks processed
13	is_deleted	bool	Soft-delete

D.4.0.2 payment_captures

Table 105: Payment capture. xmin for optimistic concurrency. Event ids as jsonb for idempotency.

No	Field name	Data type	Description
1	id	uuid	PK
2	number	varchar(50)	Human-readable ref
3	amount	numeric(18,2)	Captured amount
4	currency	varchar(3)	ISO 4217 code
5	state	text	Checkout through Refunded
6	response_code	varchar(255)	Gateway code; indexed
7	intent_client_secret	varchar(500)	Stripe client secret
8	refunded_amount	numeric(18,2)	Total refunded
9	provider_key	varchar(50)	Gateway id
10	processed_stripe_event_ids	jsonb	Webhook idempotency log
11	order_id	uuid	FK to order (cascade)
12	payment_method_id	uuid	FK to method (set null)
13	source_id	varchar(200)	Stripe payment source id

D.5 INVENTORY SCHEMA

D.5.0.1 stock_locations

Table 106: Stock location. 1:N stock_items, stock_movements (cascade).

No	Field name	Data type	Description
1	id	uuid	PK
2	name	varchar(255)	Display name
3	code	varchar(50)	Unique code
4	address1	varchar(255)	Street address
5	city	varchar(100)	City
6	postal_code	varchar(10)	Postal code
7	phone	varchar(50)	Phone
8	country_id	uuid	FK to country
9	state_id	uuid	FK to state
10	active	bool	Operational flag
11	default	bool	Default fulfillment
12	low_stock_threshold	int	Alert threshold
13	notify_on_low_stock	bool	Low-stock alerts
14	is_deleted	bool	Soft-delete

D.5.0.2 stock_items

Table 107: Stock item. Qty per variant per location. xmin concurrency. Unique (location_id, variant_id). 1:N stock_movements (cascade).

No	Field name	Data type	Description
1	id	uuid	PK
2	count_on_hand	int	On-hand qty
3	backorderable	bool	Zero-stock orders allowed
4	stock_location_id	uuid	FK to location
5	variant_id	uuid	FK to variant

D.5.0.3 stock_movements

Table 108: Stock movement. Immutable audit trail with balance snapshots.

No	Field name	Data type	Description
1	id	uuid	PK
2	quantity	int	Signed delta
3	previous_count_on_hand	int	Balance before
4	action	varchar(50)	Movement type
5	stock_item_id	uuid	FK to stock item (cascade)
6	stock_location_id	uuid	FK to location (cascade)

No	Field name	Data type	Description
7	originator_id	uuid	Polymorphic source id
8	originator_type	varchar(200)	Polymorphic source type
9	reason	varchar(500)	Adjustment reason

D.5.0.4 stock_reservations

Table 109: Stock reservation. Temp holds during checkout. xmin concurrency. Expires after configurable timeout.

No	Field name	Data type	Description
1	id	uuid	PK
2	variant_id	uuid	FK to variant
3	stock_location_id	uuid	FK to location
4	order_id	uuid	FK to order; indexed with state
5	quantity	int	Reserved qty
6	state	text	Lifecycle; indexed
7	expires_at_utc	timestamptz	Auto-release timeout
8	cart_token	text	Guest cart id; indexed with state

D.5.0.5 stock_transfers

Table 110: Stock transfer. Inter-location movement with state lifecycle. xmin concurrency.

No	Field name	Data type	Description
1	id	uuid	PK
2	number	varchar(50)	Human-readable ref
3	reference	varchar(255)	External ref
4	state	varchar(20)	Created, In-Transit, Received, Cancelled; indexed
5	source_location_id	uuid	FK to source (restrict); indexed
6	destination_location_id	uuid	FK to destination (restrict); indexed

D.5.0.6 transfer_items

Table 111: Transfer item. Line items within a stock transfer.

No	Field name	Data type	Description
1	id	uuid	PK

No	Field name	Data type	Description
2	stock_transfer_id	uuid	FK to transfer (cascade)
3	variant_id	uuid	FK to variant
4	quantity	int	Requested qty
5	received_quantity	int	Received qty

D.6 SHIPPING SCHEMA

D.6.0.1 shipping_methods

Table 112: Shipping method. 1:N shipping_method_zones (cascade).

No	Field name	Data type	Description
1	id	uuid	PK
2	name	varchar(255)	Display name
3	code	varchar(50)	Unique code
4	tracking_url	varchar(2048)	Carrier tracking URL
5	admin_name	varchar(255)	Admin display name
6	position	int	Sort order
7	available_to_users	bool	Customer selectable
8	calculator_type	varchar(100)	Rate algorithm
9	presentation	text	Customer display text
10	is_deleted	bool	Soft-delete

D.6.0.2 shipping_rates

Table 113: Shipping rate. Weight/value tiered pricing per method.

No	Field name	Data type	Description
1	id	uuid	PK
2	name	varchar(255)	Tier display name
3	selected	bool	Currently selected
4	cost	numeric(18,2)	Base cost
5	final_price	numeric(18,2)	Final price
6	display_price	varchar(50)	Formatted price
7	delivery_range	varchar(100)	Est. delivery
8	min_weight	numeric(18,2)	Min weight
9	max_weight	numeric(18,2)	Max weight
10	free_shipping_threshold	numeric(18,2)	Free shipping threshold

No	Field name	Data type	Description
11	shipping_method_id	uuid	FK to method

D.6.0.3 shipping_method_zones

Table 114: Shipping method zone. Geographic restriction. Composite index on (method_id, country_code, state_code).

No	Field name	Data type	Description
1	id	uuid	PK
2	shipping_method_id	uuid	FK to method (cascade)
3	country_code	varchar(2)	ISO 3166-1 alpha-2; composite index
4	state_code	varchar(10)	State code; composite index

D.7 PROFILE SCHEMA

D.7.0.1 user_profiles

Table 115: User profile. 1:1 with users via shared PK. 1:N addresses (cascade).

No	Field name	Data type	Description
1	user_id	uuid	PK; FK to user (1:1, cascade)
2	first_name	varchar(100)	Given name
3	last_name	varchar(100)	Family name
4	email	varchar(255)	Contact email
5	phone_number	varchar(20)	Phone
6	date_of_birth	date	Date of birth
7	gender	varchar(20)	Gender
8	avatar_url	varchar(500)	Avatar URL
9	notifications_enable_email	bool	Email notifications
10	notifications_enable_sms	bool	SMS notifications
11	accepts_email_marketing	bool	Marketing opt-in
12	default_billing_address_id	uuid	FK to billing address (set null)
13	default_shipping_address_id	uuid	FK to shipping address (set null)
14	orders_count	int	Total orders
15	total_spent	numeric(18,2)	Cumulative spend

D.7.0.2 addresses

Table 116: Address. Shipping and billing types with default designation.

No	Field name	Data type	Description
1	id	uuid	PK
2	address_type	text	Shipping or Billing
3	first_name	varchar(100)	Recipient given name
4	last_name	varchar(100)	Recipient family name
5	address1	varchar(200)	Street line 1
6	address2	varchar(200)	Street line 2
7	city	varchar(100)	City
8	zip_code	varchar(20)	Postal code
9	phone	varchar(20)	Phone
10	label	varchar(50)	User label (e.g. Home)
11	is_default	bool	Default of its type
12	country_name	varchar(100)	Country name
13	state_province	varchar(100)	State name
14	country_code	varchar(3)	Country ISO; indexed
15	state_code	varchar(10)	State code
16	user_profile_id	uuid	FK to profile (cascade); indexed with address_type

D.7.0.3 wishlists

Table 117: Wishlist. FK to users. 1:N wished_items (cascade). Indexed (user_id, is_default).

No	Field name	Data type	Description
1	id	uuid	PK
2	name	varchar(200)	Display name
3	token	varchar(128)	Shareable token
4	is_default	bool	Default wishlist
5	is_private	bool	Hidden from public
6	user_id	uuid	FK to user (cascade); indexed with is_default
7	is_deleted	bool	Soft-delete

D.7.0.4 wished_items

Table 118: Wished item. Unique variant per wishlist.

No	Field name	Data type	Description
1	id	uuid	PK
2	quantity	int	Desired qty
3	variant_id	uuid	FK to variant
4	wishlist_id	uuid	FK to wishlist (cascade)

D.8 LOCATION SCHEMA

D.8.0.1 countries

Table 119: Country. ISO 3166-1 reference. 1:N states (cascade).

No	Field name	Data type	Description
1	id	uuid	PK
2	name	varchar(100)	Display name
3	iso_code	varchar(3)	ISO 3166-1 alpha-2
4	iso3code	text	ISO 3166-1 alpha-3
5	calling_code	varchar(10)	Intl calling code
6	states_required	bool	State required
7	zipcode_required	bool	Postal code required
8	is_active	bool	Selectable

D.8.0.2 states

Table 120: State. ISO 3166-2 reference. FK to countries.

No	Field name	Data type	Description
1	id	uuid	PK
2	name	varchar(100)	Display name
3	abbreviation	varchar(10)	ISO 3166-2 code
4	country_id	uuid	FK to country (cascade)
5	is_active	bool	Selectable

D.9 PGVECTOR INTEGRATION

Vector embeddings are stored in `catalog.image_embeddings` with column type `vector(512)`, made nullable in migration 20260804013350. An IVFFlat index with cosine distance (`vector_cosine_ops`) and `lists = 100` enables approximate nearest neighbour search. On PostgreSQL/Npgsql, the embedding maps to native `vector`; on other providers, it serializes as JSON via a value converter.

Search queries use <=> cosine distance, ranking by `1 - cosine_distance`. Every embedding includes `model_name` and `model_version` for model-level filtering to ensure alignment between query and gallery embeddings.